



Cruzr S2 优必选 SDK 二次开发文档【对外】6.24

版本管理

修订记录				
N o.	日期	修订版本	修改描述	撰写人
1	2025/11/19	V1.0	第一版对外公开文档	SDK 项目组
2	2026/4/29	V2.0	新增【大寰夹爪适配】+【模块 6-上层应用接口】+【模块 7-场景 demo 案例】	SDK 项目组
3	2026/6/23	v2.1	新增模型与标定参数、优化快速指南描述、开发模式进入调整、新增运控板、视觉版说明、调整细节描述、调整代码编译库	SDK 项目组



1. 关于优必选 Cruzr S2

Cruzr S2 是优必选新一代通用轮式人形机器人，基于全栈式人形机器人技术开发，融合多模态规划大模型等具身智能关键技术，具备仿人形态、高负载、自主导航及多场景适配特性。

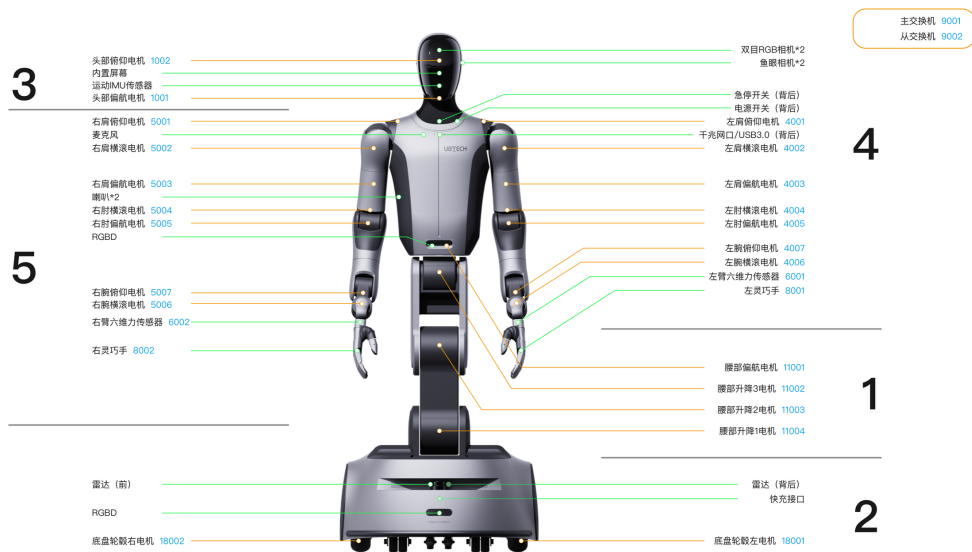
Cruzr S2 机器人整机（不含灵巧手）共 22 个自由度，由 22 个一体化关节电机驱动，实现精确运动控制。其结构包括：头部 2 个自由度，单臂 7 个自由度（肩、肘、腕），腰部 1 个自由度，折叠机构 3 个自由度，轮式底盘 2 个自由度。

1.1 总体规格参数

参数类别	具体参数
身高	176cm
臂展	177cm（不含灵巧手）
重量	188kg（不含灵巧手，满配重）
头部自由度	2
单臂自由度（不含灵巧手）	7

腰部自由度	1
折叠机构自由度	3
轮式底盘自由度	2
负载能力	双臂最大负载 15kg
越障高度	≤20mm
越沟宽度	<50mm
爬坡角度	≤5°
算力配置	X86 + NVIDIA Jetson Orin（275TOPS）
传感器配置	深度相机、六维力传感器、IMU、鱼眼相机
显示交互	4 寸圆形交互屏、电池状态指示灯、麦克风阵列、扬声器
供电方式	锂电池 DC 48V
电池充电电压/电流	电压：DC 54V 电流：20A
综合续航	8h
充电时长	≤3h
智能 OTA 升级	有
操作系统	Ubuntu + ROSA 2.0

1.2 关节参数



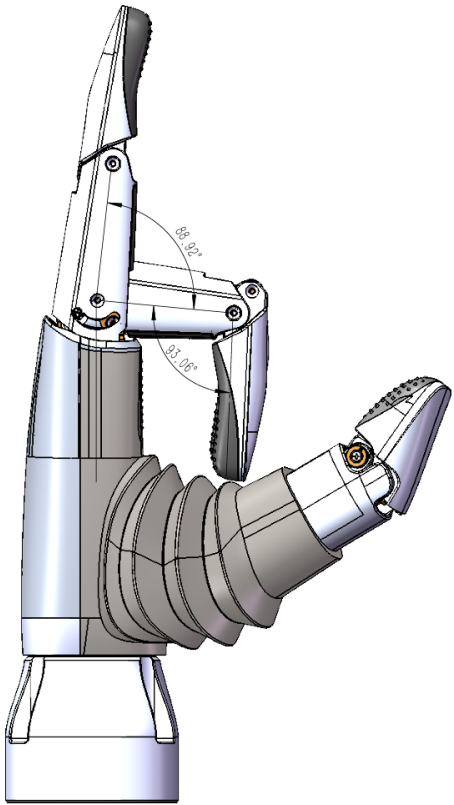
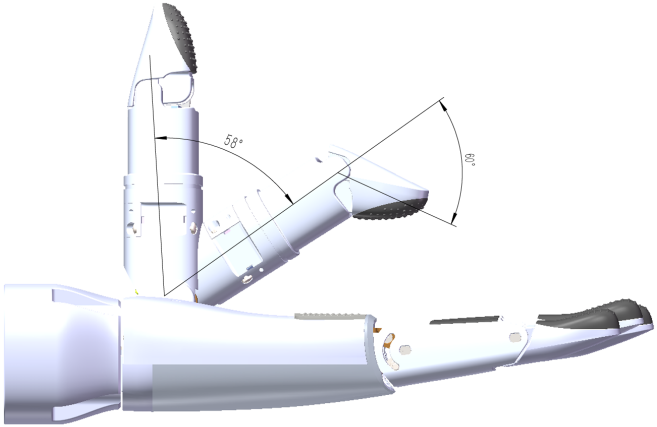
所属部位	名称	运动范围 (rad)	运动范围 (°)	额定转速 (rpm)	最大转速 (rpm)	额定力矩 (Nm)	峰值力矩 (Nm)
头部	头部俯仰电机 head_pitch_joint	lower: -0.65 upper: 0.51	lower: -37.24° upper: 29.22°	30	50	3	4.5
	头部偏航电机 head_yaw_joint	lower: -1.65 upper: 1.65	lower: -94.54° upper: 94.54°	30	50	3	4.5
左臂	左肩俯仰电机 L_shoulder_pitch_joint	lower: -2.83 upper: 2.83	lower: -162.14° upper: 162.14°	20	30	27	80
	左肩横滚电机 L_shoulder_roll_joint	lower: -1.86 upper:	lower: -106.57° upper:	20	30	27	80

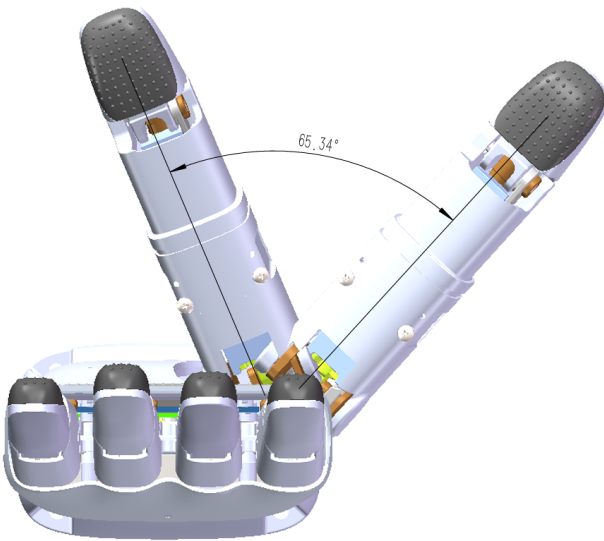
		0.08	4.58°				
	左肩偏航电机 L_shoulder_yaw_joint	lower: - 2.92 upper: 2.92	lower: - 167.30° upper: 167.30°	20	30	17	45
	左肘横滚电机 L_elbow_roll_joint	lower: - 2.6 upper: 0.02	lower: - 148.97° upper: 1.15°	20	30	17	45
	左肘偏航电机 L_elbow_yaw_joint	lower: - 2.88 upper: 2.88	lower: - 165.02° upper: 165.02°	20	30	17	20
	左腕俯仰电机 L_wrist_pitch_joint	lower: - 1.57 upper: 1.6	lower: - 89.95° upper: 91.67°	20	30	17	20
	左腕横滚电机 L_wrist_roll_joint	lower: - 1.98 upper: 1.98	lower: - 113.45° upper: 113.45°	20	30	17	20
右臂	右肩俯仰电机 R_shoulder_pitch_joint	lower: - 2.83 upper: 2.83	lower: - 162.14° upper: 162.14°	20	30	27	80
	右肩横滚电机 R_shoulder_roll_joint	lower: - 1.86 upper: 0.08	lower: - 106.57° upper: 4.58°	20	30	27	80
	右肩偏航电机 R_shoulder_yaw_joint	lower: - 2.92 upper: 2.92	lower: - 167.30° upper: 167.30°	20	30	17	45

	右肘横滚电机 R_elbow_roll_joint	lower: - 2.6 upper: 0.02	lower: - 148.97° upper: 1.15°	20	30	17	45
	右肘偏航电机 R_elbow_yaw_joint	lower: - 2.88 upper: 2.88	lower: - 165.02° upper: 165.02°	20	30	17	20
	右腕俯仰电机 R_wrist_pitch_joint	lower: - 1.6 upper: 1.57	lower: - 91.67° upper: 89.95°	20	30	17	20
	右腕横滚电机 R_wrist_roll_joint	lower: - 1.98 upper: 1.98	lower: - 113.45° upper: 113.45°	20	30	17	20
腰	腰部俯仰电机 waist_yaw_joint	lower: - 2.8 upper: 2.8	lower: - 160.43° upper: 160.43°	20	37	35	85
折叠机构自由度	腰部升降 1 电机 lifter_pitch_1_joint	lower: - 1.5 upper: 1.5	lower: - 85.94° upper: 85.94°	15	19	170	480
	腰部升降 2 电机 lifter_pitch_2_joint	lower: - 2.5 upper: 2.5	lower: - 143.24° upper: 143.24°	15	19	170	480
	腰部升降 3 电机 lifter_pitch_3_joint	lower: - 1.5 upper: 1.5	lower: - 85.94° upper: 85.94°	10	15	106	265

轮式底盘自由度	底盘轮毂左电机 wheel_left_bottom_motor	/	/	150	180	20	60
	底盘轮毂右电机 wheel_right_bottom_motor	/	/	150	180	20	60
三代手 (注 1: 左右手相同 指尖是被动关节无电机) (注 2: 驱动是直线, 转速和力矩-分别对应直线速度和推力)	四指指腹弯曲电机 L_index_mcp_joint L_middle_mcp_joint L_ring_mcp_joint L_little_mcp_joint	lower: 0.0 upper: 1.46	+0.00~+8 3.66	25m m/s	39m m/s	额定 扭矩 下推 杆推 力为 45.5 3N	峰值 扭矩 下推 杆推 力为 100N
	大拇指弯曲电机 L_thumb_mcp_joint	lower: 0.0 upper: 1.04	+0.00~+5 9.59				
	大拇指旋转电机 L_thumb_swing_joint	lower: 0.0 upper: 0.96	+0.00~+5 5.00				

关节部位	运动范围 (°)	图例说明
------	-------------	------

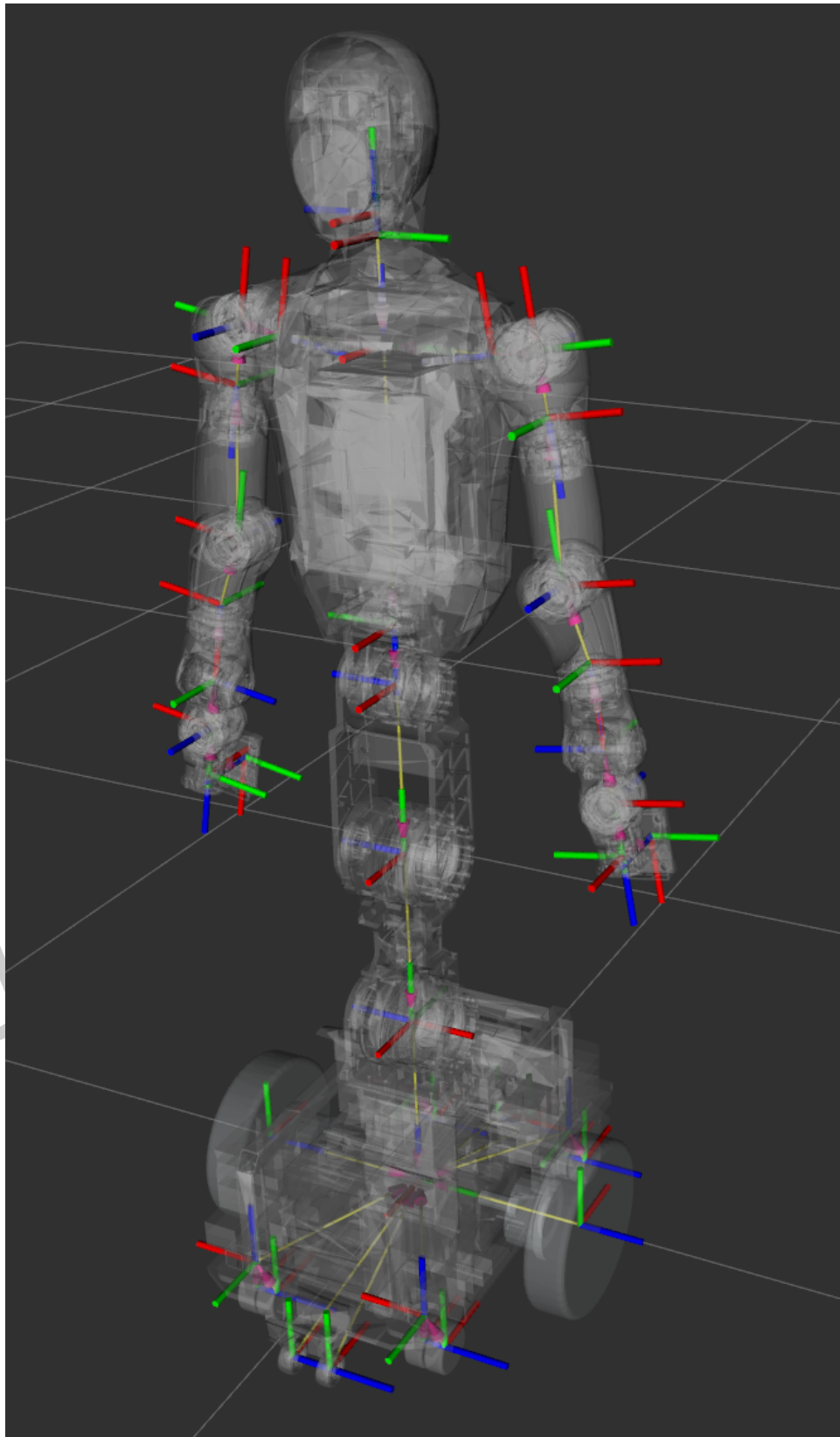
小拇指 无名指 中指 食指	指腹旋转 角度 88.92° (1.52 rad), 指 尖被动旋 转 93.06° (1.62 rad)	
大拇指弯 曲角度	拇指指腹 角度 58° (1.01 rad), 指 尖被动旋 转 60° (1.05 rad)	

大拇指旋 转角度	大拇指侧 摆角度 65.34° (1.14 rad)	 A 3D CAD model of a prosthetic hand. The thumb is shown in a side-swing position. A curved arrow indicates the angle between the vertical axis and the thumb's axis, labeled as 65.34°. The hand is purple with black textured tips on the fingers and thumb.
-------------	---	---

1.3 坐标系，关节旋转轴与关节零点

当各个关节均为零度时，各坐标系如下图。红色为 x 轴，绿色为 y 轴，蓝色为 z 轴。

优必选
UBTECH | WALKER



1.4 模型与标定参数

1.4.1 模型参数

1.4.1.1 刚度阻尼参数

电机	id	pos_ kp	pos_ ki	pos_ kd	vel_ p	vel_ i	vel_ d
head_yaw_motor	1001	600	0	0	400	5000 0	0
head_pitch_motor	1002	600	0	0	400	5000 0	0
lifter_pitch_1_motor	1100 4	635	0	0	399	5000 0	0
lifter_pitch_2_motor	1100 3	635	0	0	399	5000 0	0
lifter_pitch_3_motor	1100 2	635	0	0	399	5000 0	0
waist_yaw_motor	1100 1	600	0	0	400	5000 0	0
left_shoulder_pitch_motor	4001	600	0	0	400	5000 0	0
left_shoulder_roll_motor	4002	500	0	0	400	5000 0	0
left_shoulder_yaw_motor	4003	600	0	0	400	5000 0	0
left_elbow_roll_motor	4004	500	0	0	400	5000 0	0
left_elbow_yaw_motor	4005	600	0	0	400	5000 0	0
left_wrist_pitch_motor	4006	600	0	0	400	5000 0	0
left_wrist_roll_motor	4007	600	0	0	400	5000	0

r						0	
right_shoulder_pitch_motor	5001	600	0	0	400	5000 0	0
right_shoulder_roll_motor	5002	500	0	0	400	5000 0	0
right_shoulder_yaw_motor	5003	600	0	0	400	5000 0	0
right_elbow_roll_motor	5004	500	0	0	400	5000 0	0
right_elbow_yaw_motor	5005	600	0	0	400	5000 0	0
right_wrist_pitch_motor	5006	600	0	0	400	5000 0	0
right_wrist_roll_motor	5007	600	0	0	400	5000 0	0

1.4.2 传感器外参数

注意旋转顺序为 ZYX

传感器名称	parent link	x 世界系方向	y 世界系方向	z 世界系方向	roll (degree)	pitch (degree)	yaw (degree)
胯前相机	waist_yaw_link	0.0775 4007	0	0.0231 9591	0	51	0
底盘前方相机 (RGBD)	base_link	0.3053 3127	0.00128 866	0.1553 86	0	-10	0

前方雷达	base_link	0.28125	0.00002351	0.096			
后方雷达	base_link	-0.27975	0.0000235	0.096			
头前双目左侧	head_pitch_link	0.10675004	0.03018686	0.13200105	-90	0	0
头前双目右侧	head_pitch_link	0.10675004	-0.02981314	0.13200105	-90	0	0
头左鱼眼相机	head_pitch_link	0.02681314	0.07925004	0.10490105	-90	90	0
头右鱼眼相机	head_pitch_link	0.02718686	-0.07925004	0.10490105	-90	-90	0
头部 imu	head_pitch_link	-0.007	0	0.081			
左手末端六维力	L_sixforce_link	0	0	0	0	0	0
右手末端六维力	R_sixforce_link	0	0	0	0	0	0

2. 快速操作指南

2.1 风险提示

- 机器人运动过程中，确保机器人运动范围内无障碍物，避免碰撞
- 机器人运动过程中，肢体不要触碰机器人

2.2 开机上电

1. 上电前确保机器人：背部急停按钮处于松开状态。

2. 按下底盘金属按钮，按键变为绿色。若按下没有反应，请连接电源适配器激活，底盘电源关机超过 3 个小时会进入休眠，需要连接电源适配器激活才可按下启动（图-1）

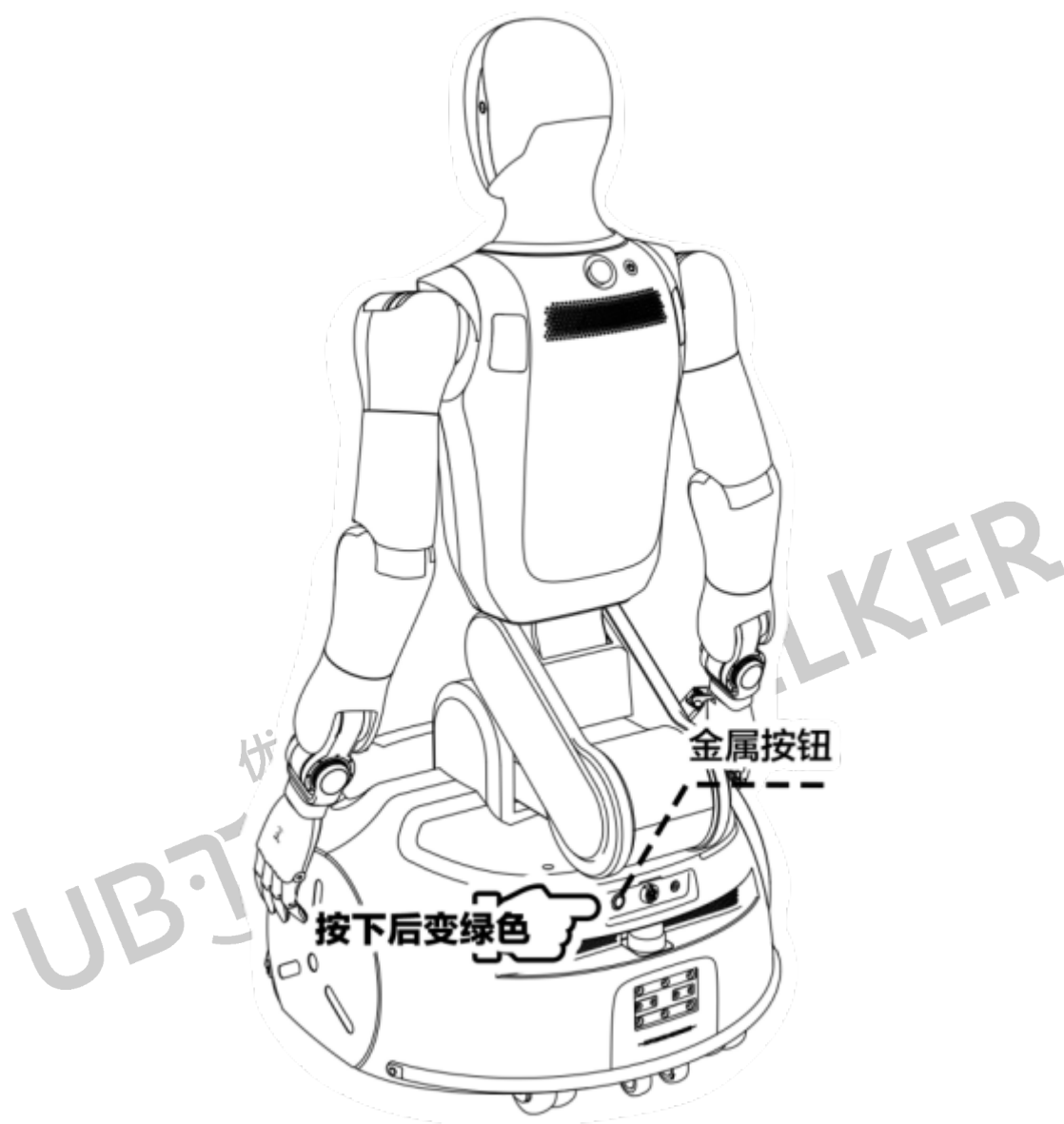


图-1

3. 按下机器人背部电源开关。（图-2）

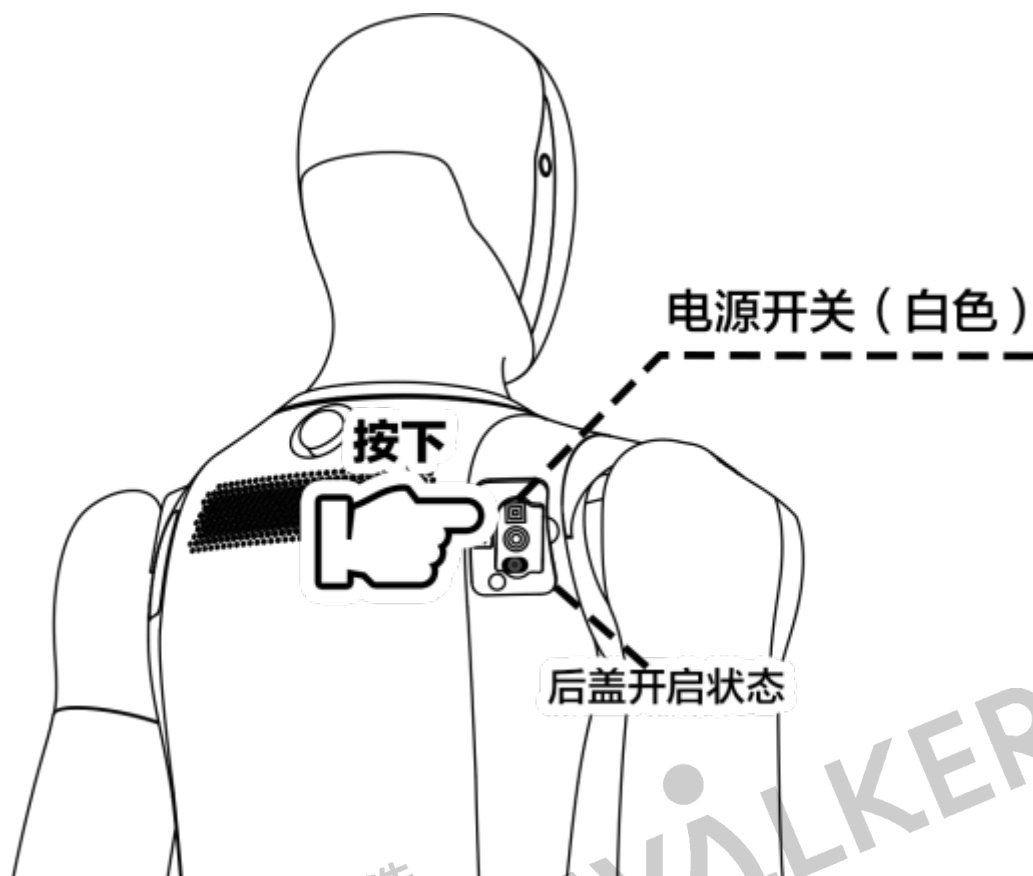


图-2

4. 长按机器人背部的启动键，约 5s，看到机器人头部显示屏亮起后表示机器人已开始启动，启动时间约 2 分钟。(图-3)

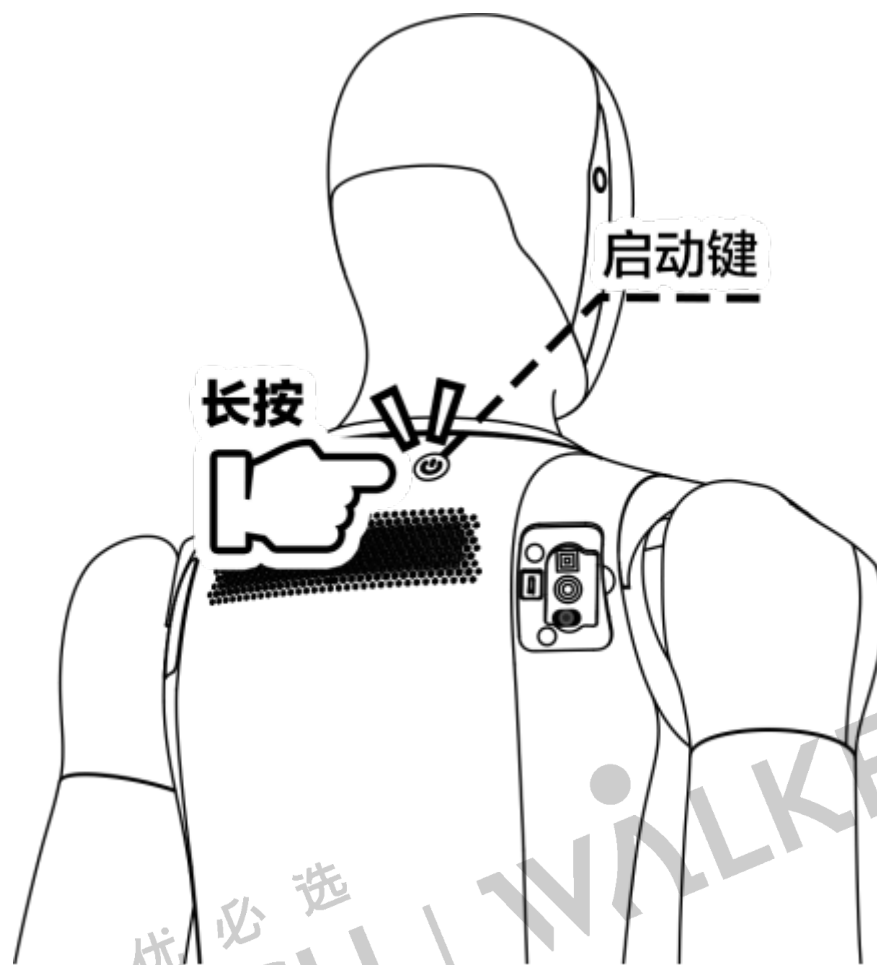
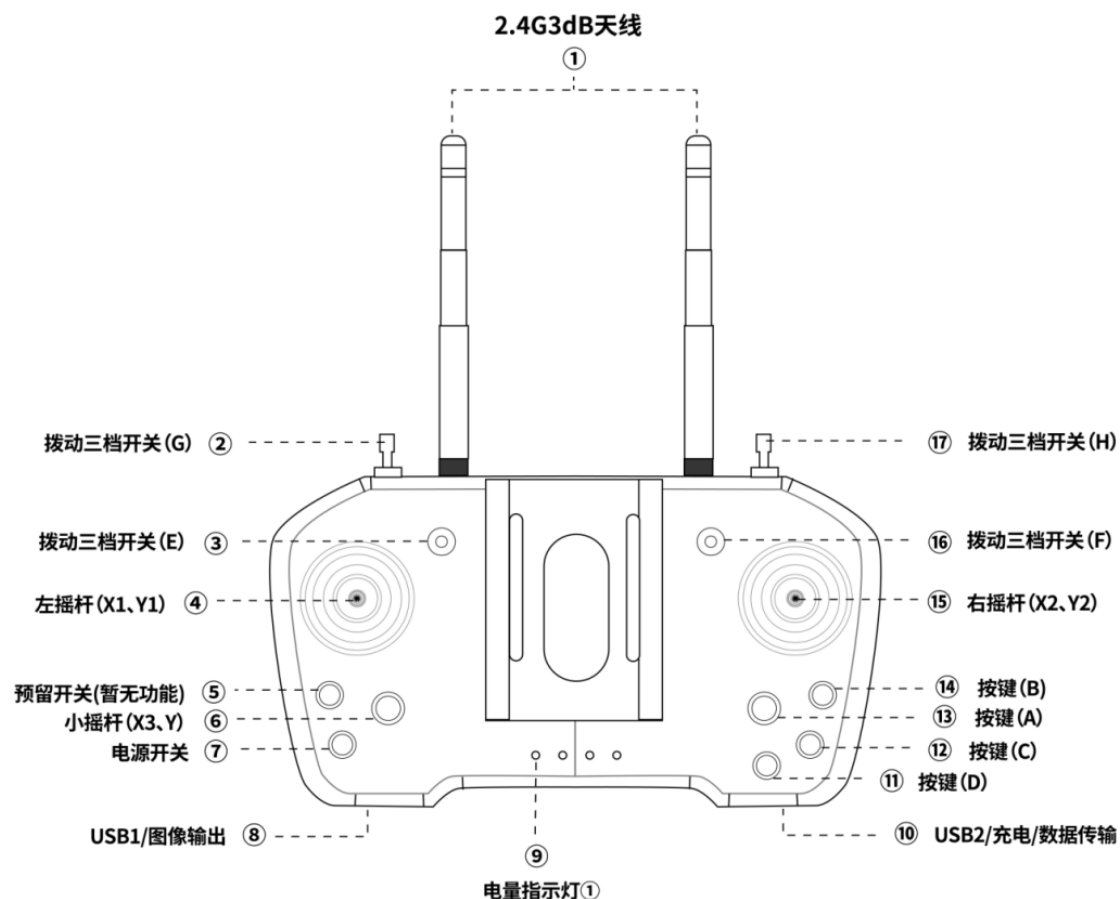


图-3

2.3 连接机器人

1. 以太网线连接 Cruzr S2 机器人：使用网线与机器人以太网口连接，接着打开网络设置，找到机器人所连接的网卡，进入 IPv4，将 IPv4 方式改为手动，地址设置为 192.168.11.99，子网掩码设置为 255.255.255.0，完成后点击应用，等待网络重新连接。
2. 通过 SSH，在 PC1 或 PC2 登录 ROS 2 容器并运行 Demo 程序或者自定义程序 (ssh ubt@192.168.11.2/3 -p 2222 密码 Ubtubt@9880)



2. 遥控器按键说明

Utars 单按键定义如下:

1	A	控制模式 确认切换工作模式	1. 控制模式: 配合 H 键使用, 当 H 在中间, 按下 A 键可切换到控制模式, 处于工作模式选择时按键失效 2. 确认切换工作模式: 工作模式切换状态下, 当 G 处于左边, 按下 A 键可以确认切换工作模式
2	B		暂无
3	C	锁位模式 解除锁位模式	1. 锁位模式 (软急停): 配合 F 键使用, 当 F 处于下边, 按下 C 键即可切换到锁位模式。处于工作模式选择时按键失效 2. 解除锁位模式: 配合 F 键使用, 当 F 处于上边, 按下 C 键即可切换到解除锁位模式。处于工作模式选择时按键失效

4	D	启动运控	启动运控：配合 F 键使用，当 F 处于下边，按下 D 键触发启动运控。处于工作模式选择时按键失效
5	E	切换任务	处于工作模式选择时按键失效 上拨：上一个任务 下拨：下一个任务
6	F	切腿部模式 启运控	1. 切腿部模式 F 上拨：配合按下 C 键，切换到解除锁位模式 F 下拨：配合按下 C 键，切换到锁位模式 2. 启运控 F 下拨：配合按下 D 键，启动运控
7	G	选择工作模式 开始执行任务	1. 开始选择工作模式：G 往左拨。此状态下只有工作模式切换相关的按键起作用 2. 结束工作模式选择：G 键从左边拨到中间 3. 开始执行任务：G 往右拨。此时无法遥控行走，继续遥控行走必须切换到控制模式。处于工作模式选择时按键失效
8	H	控制模式	配合 A 键使用，当 H 处于中间，按下 A 键
9	X1	移动	Vy 方向（不起作用），处于工作模式选择时按键失效
10	Y1	移动	Vx 方向，前进或后退，处于工作模式选择时按键失效
11	X2	移动	Vz 方向，左转或右转，处于工作模式选择时按键失效
12	Y2	折叠升降	1、折叠升降：处于工作模式选择时按键失效

		选择工作模式	Y2 上拨：上升 Y2 下拨：下降 2、切换工作模式：工作模式切换状态下， 即当 G 处于左边时 Y2 上拨：上一个工作模式 Y2 下拨：下一个工作模式 （切换完成可配合按下 A 键确认切换工作 模式）
--	--	--------	--

Utars 组合键定义如下：

G 键左拨	开始选择工作模式
G 键从左往中间拨	结束工作模式选择
G 键处于左边 + Y2 下拨	下一个工作模式
G 键处于左边 + Y2 上拨	上一个工作模式
G 键处于左边 + A 键按下	确认切换工作模式
F 键处于下边 + D 键按下	启运控
F 键处于下边 + C 键按下	锁位模式（软急停）
F 键处于上边 + C 键按下	解除锁位模式
H 键处于中间 + A 键按下	控制模式
E 键上拨	上一个任务
E 键下拨	下一个任务
G 键右拨	开始执行任务
Y2 上拨	折叠上升
Y2 下拨	折叠下降

X2 左拨（右拨）	左转向（右转向）
Y1 上拨（下拨）	前进（后退）
X1 左拨（右拨）	暂无

2.5 回零/启动控制器

1. 确认机器人急停按钮松开
2. 遥控器回零/控制器启动(操作方法请见章节 2.4)
3. 回零成功/控制器启动成功后，此时可以通过遥控器进入开发者模式，并通过 SDK 读取各类传感器数值

2.6 进入开发者模式并获取电机控制权限

在开发者模式中,将屏蔽遥控器指令.此时可以调用高层 API,如果需要取得电机的直接控制权限并接管全部电机的控制,可以通过 `cc_api` 向机器人申请该控制权限(章节 2.5,第 4 点)

1. 确保机器人控制器已经回零/启动
2. 自检通过后,遥控器选择切换至开发者模式
3. 接下来可通过,SSH 远程进入 ROS 2 Docker 容器开发环境,部署相关的自建程序

主机	架构	地址	用户名@密码
PC1	x86	192.168.11.2:2222	ubt@Ubtubt@9880
PC2	arm	192.168.11.3:2222	ubt@Ubtubt@9880

开发者容器内置文件路径:

- `/opt/demo`: 文档所提到的底层 demo 程序
- `/opt/walker/`: 开发者模式功能包(ik 逆解,详情请见 5.11.1)
- `/opt/demo`: 其他 cc api 相关文件,如 demo.pip 安装包等

```
C++
ssh -p 2222 ubt@192.168.11.2/3
# 输入密码: Ubtubt@9880
```

4. 进入开发者模式后,可以通过 `cc_api` 请求获取电机控制权限,具体操作如下,具体 Api 库请看章节 6:

```
Python
#!/usr/bin/env python3
"""
Example 3rd: Switch MC Mode Demo
切换运控模式（开发中）
"""

import ubt_robot
import time

def main():
    print("== UbtApi Demo ==")

    # 创建 API 实例
    api = ubt_robot.Api()

    # 配置连接参数
    config = ubt_robot.Config()
    config.address = "0.0.0.0:51000" # 指定机器人 Api 的服务地址
    config.api_id = "demo"           # 填入 ApiId
    config.token = ""                # 填入 Token

    # 初始化和启动 API
    api.initialize(config)
    api.start()

    # 等待连接完成
    api.wait_until_connected()

    # 切换运控模式
    # mode: "sdk"      -> 主控将伺服的控制权释放给 SDK
    #       "normal"  -> 主控收回所有伺服的控制权
    params = {
        "mode": "sdk"
    }

    def on_switch_mode_response(response):
        if not response.is_success:
            # 常见错误码:
```

```

        # -32602: 参数错误
        # 105: 重复操作
        print(f"Switch MC mode failed: code = {response.error_code}, message = {response.error_message}")
    else:
        # 成功返回: {"status": "accept"}
        print(f"Switch MC mode success: result = {response.result}")

    api.request("cc.api.sdk.switch_mc_mode", params, on_switch_mode_response)

    # 等待异步响应
    print("Waiting for async response (3 seconds)...")
    time.sleep(3)

    # 停止 API 连接服务
    api.stop()

    print("== End ==")

if __name__ == "__main__":
    main()

```

5. 开发者模式介绍:

当前版本当开机之后,会进入一个设定模式,例如(遥控模式,开发者模式,遥操模式),不同模式可以通过遥控器进行切换,在本章节主要介绍开发者模式:

- 当通过遥控器进入开发者模式后,默认进入 **cc_api** 模式,此时遥控器指令将屏蔽,用户可以调用内置的运控算法通过 **cc_api** 对机器人进行移动等调用,并通过 **cc_api** 控制机器人的运动
- 用户可以通过 2.5.4 的代码向机器人请求伺服电机控制权,为了避免指令冲突,一旦请求控制权,内置的运动控制算法将失效,用户将接管所有电机的控制
- 当用户自建程序使得机器人发生崩溃/或者其他错误,机器人将 **播报:SDK 模式运行错误, 请按下急停后退出 SDK 模式或重启运控程序自动进入** 此时用户需要拍下急停,并且在确保安全的情况下重新启动机器人,机器人运控重启后,将自动获取电机控制权限给开发者,无需重复获取

获取电机控制权限注意事项!:

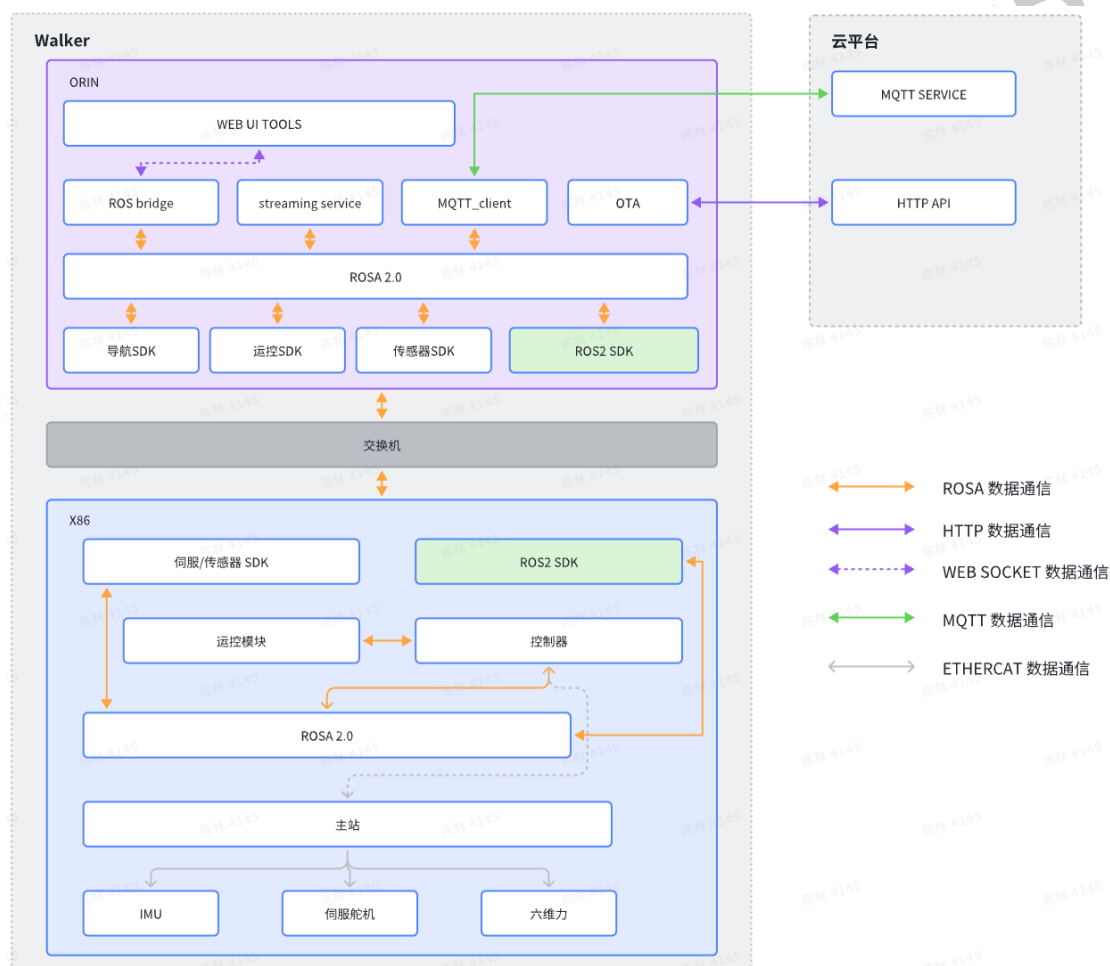
在开发者模式中,一旦获取了电机控制权限,机器人运动控制相关模块将退出,机器人平衡将不能得到保证,开发者需要对所有关节进行接管控制,假如不需要直接对机器人电机进行控制,我们建议不需要向机器人请求电机控制权限,可以通过高层 API 对机器人进行运动控制,以满足开发者的业务需求

不建议在 vision(192.168.11.3)或者外部 PC 对关节进行实时控制,通信实时性将不能得到保证

3. 应用开发

3.1 软件架构说明

整机分为 ORIN 上层计算单元、X86 底层运动控制单元两大核心硬件模块,二者通过交换机完成数据互通;整机可通过网络对接外部云平台,形成“端 - 边 - 云”完整体系。



系统架构图

- **X86 底层运动控制单元 (运控板):** 负责实时性要求极高的底层运动控制。

- **ORIN 上层计算单元（计算板）：**负责上层应用、AI 推理与视觉算法计算。

两块板卡并非传统的上下位机关系，而是作为两个独立的 ROS2 节点，通过共享同一个 `ROS_DOMAIN_ID` 实现主题和服务的透明互通。

3.1.1 核心硬件分工表

板卡名称	硬件架构	网络地址	角色定位	核心运行内容
PC1 - 运控板	x86	192.168.11.2 :2222	底层运动控制（实时域）	t800_mc_server 运控服务、关节电机控制、IMU/六维力状态发布、
PC2 - 计算板	ARM (Orin)	192.168.11.3 :2222	上层应用/AI（计算域）	ubt_ros2_demo 容器、VLA 推理、视觉算法、大模型、SDK 二开程序 cc_api 服务

3.1.2 开发环境选择指南

针对二次开发，强烈建议开发者主要在 PC2（11.3 Orin 计算板）上进行程序构建与运行。PC1（11.2 运控板）在出厂状态下已配置完毕，通常无需额外安装或构建 demo 程序。

开发需求	推荐开发板	推荐工具/接口	说明
快速测试手臂/头部/手部控制	PC2 (11.3/2)	ros2 run ubt_ros2_example pub_arm_command	使用 SDK 自带 demo 验证功能
订阅 IMU / 六维力 / 关节状态	PC2 (11.3/2)	ros2 run ubt_ros2_example sub_robot_state	状态主题跨板可见，可直接订阅

编写自定义 C++/Python 控制节点	PC2 (11.2)	ROS2 + SDK low_level 接口	在 11.2 运行保证时候实时性
使用 Python/cc_api 高层控制	PC2 (11.3)	<code>import ubt_robot + cc_api</code>	需在开发者模式下运行
接入 VLA / 大模型推理	PC2 (11.3)	VLA 模型 + ROS2 通信	推理在 11.3，运控容器在 11.2 接收执行
部署底层运控算法 / 500Hz+ 实时控制	PC1 (11.2)	<code>rosa_controllers</code>	需登录运控板底层，确保运控指令的实时性

3.1.3 ROSA 2.0

优必选独立研发的机器人操作系统应用框架 ROSA 2.0，将机器人的各项功能与控制进行了深度整合，为机器人在多种场景下实现灵活应用提供了坚实支撑，保证底层算法的自主可控和安全性。

3.2 ROS 2 SDK 概述

优必选 SDK 由优必选机器人公司开发，提供了丰富的接口，涵盖头部、臂部、肘部和手部的电机控制，以及六维力、IMU（惯性测量单元）和相机的使用，用于编写和部署机器人应用程序，旨在帮助开发人员快速灵活地构建自己的应用程序来精确控制和使用机器人，以满足在不同应用场景下的需求。您可以按照我们提供的接口和例程，配套本开发指南，完成对 Cruzr S2 的二次开发。

3.3 ROS 2 SDK 获取

现阶段需要 ROS SDK demo 程序，请通过优必选公司对接人联系 jialong.kuang@ubtrobot.com 提供。未来，我们会在官网上附公开的下载链接。

3.3.1 ROS 2 SDK 构建

- 确保正确的网络连接后，进入机器人 ROS 2 容器

```
C++
ssh -p 2222 ubt@192.168.11.3
# 输入密码: Ubtubt@9880
```

- 在你可以下载并构建我们的 ROS 2 demo 程序

YAML

#将 demo 程序复制到/debug 目录下

```
scp -P 2222 -r ubt_ros2_demo ubt@192.168.11.3:/debug/
```

```
cd /debug/ubt_ros2_demo
```

```
source /opt/ros/humble/setup.bash
```

#构建 ubt_ros2demo 程序

```
colcon build
```

- 在该项目结构中
 - example 存储 SDK 的示例 demo 代码
 - low_level 为低层 SDK 调用，common 为相关通用接口，如音频播放等

3.3.2 运行并测试

在完成上述代码构建后输入

YAML

```
source /debug/ubt_ros2_demo/install/setup.bash
```

请您在运控版（ubt@192.168.11.2 -p 2222）运行与电机控制以及状态订阅相关的程序，并通过 `sudo su` 进入 root 用户

您可以运行我们的 demo 程序，同时也可以打开终端，并输入

YAML

```
ros2 topic list
```

您可以看到以下主题，可以通过 `ros2 topic echo` 打印对应的数据

```
/rosout
/sensor/camera/chassis_front_rgbdc/color/info
/sensor/camera/chassis_front_rgbdc/color/raw
/sensor/camera/chassis_front_rgbdc/depth/info
/sensor/camera/chassis_front_rgbdc/depth/raw
/sensor/camera/chassis_front_rgbdc/mix/raw
/sensor/camera/fisheye_left/image/raw
/sensor/camera/fisheye_right/image/raw
/sensor/camera/stereo/april_tag/results
/sensor/camera/stereo/depth/info
/sensor/camera/stereo/pointcloud/raw
/sensor/camera/stereo_left/image/raw
/sensor/camera/stereo_right/image/raw
/sensor/camera/waist_front_rgbdc/april_tag/results
/sensor/camera/waist_front_rgbdc/color/info
/sensor/camera/waist_front_rgbdc/color/raw
/sensor/camera/waist_front_rgbdc/depth/info
/sensor/camera/waist_front_rgbdc/depth/raw
/sensor/camera/waist_front_rgbdc/mix/raw
/sensor/gps
/sensor/imu/x86
/sensor/lidar/back
/sensor/lidar/front
/sensor/lslidar_point_cloud/top/raw
```

- demo 程序示例

示例源代码位于/example/src 文件夹下,我们提供如下示例程序

- audio_player 利用 Cruzr S2 内置喇叭播放固定音频文件
 - powerboardstate_subscriber 读取 Cruzr S2 电源版状态信息
 - batterystate_subscriber 订阅 Cruzr S2 电源状态信息
 - pub_arm_command 发布手臂控制
 - pub_hand_command 发布灵巧手控制
 - pub_head_command 发布头部控制
 - sub_hands_state 订阅手部状态信息
 - sub_robot_state 订阅机器人状态信息(IMU,六维力,关节电机等)
 - camera_subscriber 为各类相机订阅示例程序
- 运行示例代码

- a. 以 imu 订阅 demo 为例，打开终端并输入：

```
YAML
ros2 run ubt_ros2_example imu_subscriber
```

- b. 就可以获取 IMU(Orin)状态信息

```
ubuntu@ubuntu:~/debug$ ros2 run ubt_ros2_example imu_subscriber
[INFO] [1758543179.788544882] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.788983814] [imu_subscriber]: Angular velocity: [x=0.000, y=-0.001, z=0.002] rad/s
[INFO] [1758543179.789053326] [imu_subscriber]: Linear acceleration: [x=0.222, y=-5.080, z=8.381] m/s^2
[INFO] [1758543179.798115075] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.798156451] [imu_subscriber]: Angular velocity: [x=-0.001, y=0.001, z=-0.000] rad/s
[INFO] [1758543179.798166355] [imu_subscriber]: Linear acceleration: [x=0.238, y=-5.072, z=8.376] m/s^2
[INFO] [1758543179.799182744] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.799280806] [imu_subscriber]: Angular velocity: [x=0.001, y=0.001, z=-0.001] rad/s
[INFO] [1758543179.799304555] [imu_subscriber]: Linear acceleration: [x=0.215, y=-5.072, z=8.395] m/s^2
[INFO] [1758543179.809380421] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.809581697] [imu_subscriber]: Angular velocity: [x=0.001, y=-0.002, z=-0.002] rad/s
[INFO] [1758543179.809593327] [imu_subscriber]: Linear acceleration: [x=0.229, y=-5.064, z=8.361] m/s^2
[INFO] [1758543179.814352345] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.814390257] [imu_subscriber]: Angular velocity: [x=0.001, y=-0.002, z=0.002] rad/s
[INFO] [1758543179.814399827] [imu_subscriber]: Linear acceleration: [x=0.207, y=-5.062, z=8.358] m/s^2
[INFO] [1758543179.817806568] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.817883934] [imu_subscriber]: Angular velocity: [x=-0.000, y=0.001, z=0.001] rad/s
[INFO] [1758543179.817897652] [imu_subscriber]: Linear acceleration: [x=0.222, y=-5.053, z=8.368] m/s^2
[INFO] [1758543179.823348195] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.823418228] [imu_subscriber]: Angular velocity: [x=0.000, y=-0.002, z=0.001] rad/s
[INFO] [1758543179.823501905] [imu_subscriber]: Linear acceleration: [x=0.228, y=-5.055, z=8.374] m/s^2
[INFO] [1758543179.828256738] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.828298224] [imu_subscriber]: Angular velocity: [x=-0.001, y=0.002, z=0.001] rad/s
[INFO] [1758543179.828315086] [imu_subscriber]: Linear acceleration: [x=0.231, y=-5.063, z=8.382] m/s^2
[INFO] [1758543179.831956306] [imu_subscriber]: Orientation: [w=0.963, x=-0.268, y=-0.012, z=0.000]
[INFO] [1758543179.831980577] [imu_subscriber]: Angular velocity: [x=0.000, y=0.001, z=0.000] rad/s
```

- 其它详细示例见-模块 5

3.4 环境依赖

3.4.1 系统环境配置

为了确保最佳的开发体验和兼容性，建议在自建程序在机器人本体的 ROS 2 容器内运行。当前暂不支持在 Mac 和 Windows 系统下开发。

Cruzer S2 框架环境

平台	地址	系统版本	ROS 2 容器
Cruzer S2 x86	192.168.11.2	Ubuntu 22.04	humble
Cruzer S2 Orin	192.168.11.3	Ubuntu 22.04	humble

4. 全局值说明

- 说明：优必选 Cruzr S2 头部、手部、腰折叠机构、轮式底盘的电机命名 ID 定义

如下

Bash

```
{9001, "switch main", "主交换机"},

// IMU
{10000, "motion_imu", "运动 IMU 传感器"}, // EVT4 取消运控 IMU

// 腰
{11001, "waist_yaw_motor", "腰部偏航电机"},
{11002, "lifter_pitch_3_motor", "腰部升降 3 电机"},
{11003, "lifter_pitch_2_motor", "腰部升降 2 电机"},
{11004, "lifter_pitch_1_motor", "腰部升降 1 电机"},

// 底盘轮毂电机,使用一个 EtherCAT 转 CAN 分开左右两路轮毂电机 18001 和
18002, 对于运控 SDK 来看是两个电机
{18000, "ethercat2can", "Ecat2CAN 模块"}, // 对用户不可见
{18001, "wheel_left_botton_motor", "底盘轮毂左电机"},
{18002, "wheel_right_botton_motor", "底盘轮毂右电机"},

{9002, "switch sub", "从交换机"},

// 头部
{1001, "head_yaw_motor", "头部偏航电机"},
{1002, "head_pitch_motor", "头部俯仰电机"},

// 左臂
{4001, "left_shoulder_pitch_motor", "左肩俯仰电机"},
{4002, "left_shoulder_roll_motor", "左肩横滚电机"},
{4003, "left_shoulder_yaw_motor", "左肩偏航电机"},
{4004, "left_elbow_roll_motor", "左肘横滚电机"},
{4005, "left_elbow_yaw_motor", "左肘偏航电机"},
{4006, "left_wrist_roll_motor", "左腕横滚电机"},
{4007, "left_wrist_pitch_motor", "左腕俯仰电机"},
{6001, "left_arm_ft", "左臂六维力传感器"},
{8001, "left_hand", "左手（可能未接灵巧手，实际为夹板）"},

// 右臂
{5001, "right_shoulder_pitch_motor", "右肩俯仰电机"},
{5002, "right_shoulder_roll_motor", "右肩横滚电机"},
{5003, "right_shoulder_yaw_motor", "右肩偏航电机"},
```

```
{5004, "right_elbow_roll_motor", "右肘横滚电机"},
{5005, "right_elbow_yaw_motor", "右肘偏航电机"},
{5006, "right_wrist_roll_motor", "右腕横滚电机"},
{5007, "right_wrist_pitch_motor", "右腕俯仰电机"},
{6002, "right_arm_ft", "右臂六维力传感器"},
{8002, "right_hand", "右手（可能未接灵巧手，实际为夹板）"}
```

5. 接口说明-底层服务

本文提到的底层服务 demo 源码,可以在开发者容器/opt/demo 中看到

5.1 运控控制

5.1.1 运控状态状态获取接口

- 说明：获取运动控制相关的状态信息，其中包含电机的当前位置、IMU 和六维力的信息。
- 控制方式：topic
- 话题名称：/mc/sdk/robot_state
- 数据定义位置：mc_state_msgs::msg::robot_state
- 数据格式：

```
Plain Text
std_msgs/Header header
sensor_msgs/JointState joint_states
sensor_msgs/Imu[] imu_states
geometry_msgs/WrenchStamped[] ft_states
```

- 代码示例：

```
C++
#include "rclcpp/rclcpp.hpp"
#include "mc_state_msgs/msg/robot_state.hpp"
#include "std_msgs/msg/string.hpp"
#include <iostream>
#include <memory>

class RobotStateSubscriber : public rclcpp::Node
```

```

{
public:
    RobotStateSubscriber() : Node("sub_robot_state")
    {
        // 创建 QoS 配置，使用系统默认的传感器数据 QoS
        rclcpp::QoS qos_settings(10);
        qos_settings.reliability(RMW_QOS_POLICY_RELIABILITY_BEST_EFFORT);
        qos_settings.durability(RMW_QOS_POLICY_DURABILITY_VOLATILE);
        qos_settings.history(RMW_QOS_POLICY_HISTORY_KEEP_LAST);

        subscription_ = this-
>create_subscription<mc_state_msgs::msg::RobotState>(
            "/mc/sdk/robot_state",
            qos_settings,
            std::bind(&RobotStateSubscriber::topic_callback, this,
std::placeholders::_1));
    }

private:
    void topic_callback(const
mc_state_msgs::msg::RobotState::SharedPtr msg) const
    {
        RCLCPP_INFO(this->get_logger(),
"=====");

        for (size_t i = 0; i < msg->joint_states.name.size(); i++) {
            std::cout << "Joint: " << msg->joint_states.name[i]
                << " Joint position: " << msg-
>joint_states.position[i] << std::endl;
        }

        for (const auto& item : msg->imu_states) {
            std::cout << "Imu: " << item.header.frame_id
                << ", acc:" << item.linear_acceleration.x << " "
                << item.linear_acceleration.y << " " <<
item.linear_acceleration.z
                << ", gyro:" << item.angular_velocity.x << " "
                << item.angular_velocity.y << " "
                << item.angular_velocity.z << std::endl;
        }

        for (const auto& item : msg->ft_states) {
            std::cout << "Ft: " << item.header.frame_id

```



```

        << ", force: " << item.wrench.force.x << " "
        << item.wrench.force.y << " " <<
item.wrench.force.z
        << ", torque: " << item.wrench.torque.x << " "
        << item.wrench.torque.y << " " <<
item.wrench.torque.z << std::endl;
    }
}

rclcpp::Subscription<mc_state_msgs::msg::RobotState>::SharedPtr
subscription_;
};

int main(int argc, char const* argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<RobotStateSubscriber>());
    rclcpp::shutdown();
    return EXIT_SUCCESS;
}

```

5.1.2 运动控制接口

- 说明：电机的位置控制接口，需要提供电机的名称、控制模式和目标位置。
- 控制方式：topic
- 话题名称：/mc/sdk/robot_command
- 数据定义位置：mc_task_msgs::msg::joint_cmd.hpp 以及 mc_task_msgs::msg::robot_command.hpp
- 数据格式：

RobotCommand

```

Plain Text
std_msgs/Header header
JointCmd[] joint_cmd

```

JointCmd

```

Go
int8 BEGIN=-1
int8 MODE_EFFORT=0
int8 MODE_VELOCITY=1

```

```

int8 MODE_POSITION=2
// 7/8/9 为自定义控制模式槽位，物理语义由各硬件驱动自行解释，配合
v1/v2/v3 额外参数使用
int8 CUSTOM_MODE_1 = 7 //为 PVT 力位混合模式,同时下发
pos/vel/effort, v1=Kp, v2=Kd,
int8 CUSTOM_MODE_2 = 8 //预留，暂无驱动实现，下发后会落入"不支持"分支
(伺服置为 BEGIN/不动作)。
int8 CUSTOM_MODE_3 = 9 // 预留，暂无驱动实现，下发后会落入"不支持"分
支(伺服置为 BEGIN/不动作)。

string name
int8 control_mode
float64 position
float64 velocity
float64 effort
float64 v1
float64 v2
float64 v3

```

- 代码示例:

```

C++
#include <mc_task_msgs/msg/robot_command.hpp>
#include <mc_task_msgs/msg/joint_cmd.hpp>

#include <chrono>
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/header.hpp>
#include <cmath>

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("pub_head_command");
    auto cmd_publisher_ = node-
>create_publisher<mc_task_msgs::msg::RobotCommand>("/mc/sdk/robot_
command", 10);
    rclcpp::Rate rate(500);
    double time_cnt = 0.0;

    while (rclcpp::ok()) {
        mc_task_msgs::msg::RobotCommand cmd;
        cmd.header.stamp = node->now();

        mc_task_msgs::msg::JointCmd head_yaw_cmd;

```

```

    head_yaw_cmd.name = "head_yaw_joint";
    head_yaw_cmd.control_mode =
mc_task_msgs::msg::JointCmd::MODE_POSITION;
    head_yaw_cmd.position = sin(time_cnt) * 0.5;
    cmd.joint_cmd.push_back(head_yaw_cmd);

    mc_task_msgs::msg::JointCmd head_pitch_cmd;
    head_pitch_cmd.name = "head_pitch_joint";
    head_pitch_cmd.control_mode =
mc_task_msgs::msg::JointCmd::MODE_POSITION;
    head_pitch_cmd.position = sin(time_cnt) * 0.5;
    cmd.joint_cmd.push_back(head_pitch_cmd);

    cmd_publisher_->publish(cmd);

    time_cnt += 0.002;
    rate.sleep();
}

rclcpp::shutdown();
return EXIT_SUCCESS;
}

```

注意：

- 保证机器处于零位状态后，再启动 demo 程序，控制信息需从零值开始发，不要发阶跃值

错误码信息，如下：

故障类别	故障码 (603F)	故障详细	处理机制
一级	0x1001	电机过温告警	只上报告警信息 >90
一级	0x1002	伺服过温告警	只上报告警信息 >80
一级	0x1003	伺服堵转告警	只上报告警信息
二级	0x2001	位置超限	超限后 0 力矩输出并且只响应反向指令；

二级	0x2002	转速超限	输出 0 力矩，故障恢复后自动恢复力矩输出；
二级	0x2003	过电流故障	输出 0 力矩，故障恢复后自动恢复力矩输出；
二级	0x2004	过电压故障	限制输出电流为满电流的 1/2，故障恢复后自动恢复；
二级	0x2005	欠电压故障	限制输出电流为满电流的 1/2，故障恢复后自动恢复；
二级	0x2006	堵转故障	伺服直接关动力输出，并且需要重启电源复位故障；
二级	0x2007	电机过温	限制输出电流为满电流的 1/2，故障恢复后自动恢复； > 95
二级	0x2008	伺服过温	限制输出电流为满电流的 1/2，故障恢复后自动恢复； > 85
二级	0x2009	通讯中断	通讯中断后自动锁位到当前位置，通讯恢复后恢复运行；
三级	0x3001	硬件过流	伺服直接关动力输出，并且需要重启电源复位故障；
三级	0x3002	电流检测故障	伺服直接关动力输出，并且需要重启电源复位故障；
三级	0x3003	EEPROM 故障	伺服直接关动力输出，并且需要重启电源复位故障；
三级	0x3004	内环编码器故障	伺服直接关动力输出，并且需要重启电源复位故障；
三级	0x3005	外环编码器故障	伺服直接关动力输出，并且需要重启电源复位故障；

三级	0x3006	三相输出异常	伺服直接关动力输出，并且需要重启电源复位故障；
三级	0x3007	电机过温	伺服直接关动力输出，温度恢复后自动清除故障； >100
三级	0x3008	伺服过温	伺服直接关动力输出，温度恢复后自动清除故障； >90

5.2 IMU orin

5.2.1 状态获取接口

- 说明：获取 orin 上的 IMU 传感器数据信息，其中包含加速度、角速度、位姿四元素。
- 控制方式：topic
- 话题名称：/sensor/imu/orin
- 数据类型：sensor_msgs::msg::Imu
- 数据格式：

Plain Text

```
sensor_msgs::msg::Imu
```

```
std_msgs/Header header          #消息头
```

```
geometry_msgs/Quaternion orientation      #姿态四元数
```

```
float64[9] orientation_covariance        #姿态的协方差矩阵
```

```
geometry_msgs/Vector3 angular_velocity    #角速度向量
```

```
float64[9] angular_velocity_covariance    #角速度的协方差矩阵
```

```
geometry_msgs/Vector3 linear_acceleration #线性加速度向量
```

```
float64[9] linear_acceleration_covariance #线性加速度的协方差矩阵
```

- 订阅话题：

Plain Text

```
ros2 run ubt_ros2_example imu_subscriber
```

- 示例代码

```
C++
#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/imu.hpp"

class ImuSubscriber : public rclcpp::Node
{
public:
    ImuSubscriber() : Node("imu_subscriber")
    {
        subscription_ = this-
>create_subscription<sensor_msgs::msg::Imu>(
        "/sensor/imu/orin", // IMU 话题名称
        rclcpp::QoS(rclcpp::KeepLast(10)).best_effort(),
        std::bind(&ImuSubscriber::topic_callback, this,
std::placeholders::_1));
    }

private:
    void topic_callback(const sensor_msgs::msg::Imu::SharedPtr msg)
    {
        RCLCPP_INFO(this->get_logger(),
            "Orientation: [w=%.3f, x=%.3f, y=%.3f, z=%.3f]",
            msg->orientation.w, msg->orientation.x, msg-
>orientation.y,
            msg->orientation.z);

        RCLCPP_INFO(this->get_logger(),
            "Angular velocity: [x=%.3f, y=%.3f, z=%.3f]
rad/s",
            msg->angular_velocity.x, msg->angular_velocity.y,
            msg->angular_velocity.z);

        RCLCPP_INFO(this->get_logger(),
            "Linear acceleration: [x=%.3f, y=%.3f, z=%.3f]
m/s^2",
            msg->linear_acceleration.x, msg-
>linear_acceleration.y,
            msg->linear_acceleration.z);
    }

    rclcpp::Subscription<sensor_msgs::msg::Imu>::SharedPtr
subscription_;
```

```
};

int main(int argc, char *argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<ImuSubscriber>());
    rclcpp::shutdown();
    return 0;
}
```

5.3 RGBD 相机

Cruzr S2 在底盘前以及腰前配置了 Gemini 2L 相机并提供如下话题及服务:



5.3.1 底盘奥比中光 Gemini 2L 相机

- 默认提供话题如下:

```
/sensor/camera/chassis_front_rgbd/color/info: sensor_msgs/msg/CameraInfo
/sensor/camera/chassis_front_rgbd/color/raw: shm_msgs/msg/Image1m
/sensor/camera/chassis_front_rgbd/depth/info: sensor_msgs/msg/CameraInfo
/sensor/camera/chassis_front_rgbd/depth/raw: shm_msgs/msg/Image1m
/sensor/camera/chassis_front_rgbd/mix/raw: shm_msgs/msg/Vec2Image1m
```

- 示例
 - 说明: 获取头部 Gemini 2L 数据
 - 话题名称: `/sensor/camera/chassis_front_rgbd/color/raw`
 - 类型: `shm_msgs::msg::Image1m`

- 数据格式

```
YAML
shm_msgs::msg::Image1m
shm_msgs/Header header #消息头
uint32 height           # 图像高度
uint32 width            # 图像宽度
shm_msgs/String encoding # 像素编码
uint8 is_bigendian      # 是否为大端序
uint32 step             # 行的总字节长度
uint8[1048576] data      # 实际矩阵数据
uint32 DATA_MAX_SIZE=1048576
```

```
Plain Text
shm_msgs/Header
  builtin_interfaces/Time stamp
  shm_msgs/String frame_id

shm_msgs/String
  char[256] data
  uint8 size 0
  uint8 MAX_SIZE=255
```

- 订阅话题:

```
Plain Text
ros2 run ubt_ros2_example headfrontcolor_subscriber #订阅
颜色话题
```

- 示例代码:

```
C++
#include "rclcpp/rclcpp.hpp"
#include "shm_msgs/msg/image1m.hpp"

class ChassisFrontRgbColorSubscriber : public
rclcpp::Node
{
public:
  ChassisFrontRgbColorSubscriber() :
Node("chassis_front_color_subscriber")
```



```

    {
        subscription_ = this-
>create_subscription<shm_msgs::msg::Image1m>(
            "/sensor/camera/chassis_front_rgbd/color/raw", //
话题名称
            rclcpp::QoS(rclcpp::KeepLast(10)).best_effort(),

std::bind(&ChassisFrontRgbdColorSubscriber::topic_callback
, this,
            std::placeholders::_1));
    }

private:
    void topic_callback(const
shm_msgs::msg::Image1m::SharedPtr msg)
    {
        RCLCPP_INFO(this->get_logger(), "Cruent Time: %d
sec %u nanosec",
            msg->header.stamp.sec, msg-
>header.stamp.nanosec);
        RCLCPP_INFO(this->get_logger(), "Frame id: %s",
            reinterpret_cast<char *>(msg-
>header.frame_id.data.data()));
        RCLCPP_INFO(this->get_logger(), "Height * Width: %u
* %u", msg->height,
            msg->width);
        RCLCPP_INFO(this->get_logger(), "Encoding: %s",
            reinterpret_cast<char *>(msg-
>encoding.data.data()));
        RCLCPP_INFO(this->get_logger(), "Bigendian: %u", msg-
>is_bigendian);
        RCLCPP_INFO(this->get_logger(), "Step: %u", msg-
>step);
        RCLCPP_INFO(this->get_logger(), "Matrix data
length: %zu",
            sizeof(msg->data) / sizeof(msg->data[0]));
    }

    rclcpp::Subscription<shm_msgs::msg::Image1m>::SharedPtr
subscription_;
};

int main(int argc, char *argv[])
{

```

```

    rclcpp::init(argc, argv);

    rclcpp::spin(std::make_shared<ChassisFrontRgbdColorSubscriber>());
    rclcpp::shutdown();
    return 0;
}

```

- 默认提供服务如下：

```

Bash
/chassis_front_rgbd/get_color_auto_white_balance:
orbdec_camera_msgs/srv/GetInt32
/chassis_front_rgbd/get_color_device_info:
orbdec_camera_msgs/srv/GetDeviceInfo
/chassis_front_rgbd/get_color_exposure:
orbdec_camera_msgs/srv/GetInt32
/chassis_front_rgbd/get_color_gain:
orbdec_camera_msgs/srv/GetInt32
/chassis_front_rgbd/get_color_sharpness:
orbdec_camera_msgs/srv/GetInt32
/chassis_front_rgbd/get_color_white_balance:
orbdec_camera_msgs/srv/GetInt32
/chassis_front_rgbd/get_depth_exposure:
orbdec_camera_msgs/srv/GetInt32
/chassis_front_rgbd/get_depth_gain:
orbdec_camera_msgs/srv/GetInt32
/chassis_front_rgbd/set_color_auto_exposure:
std_srvs/srv/SetBool
/chassis_front_rgbd/set_color_auto_white_balance:
std_srvs/srv/SetBool
/chassis_front_rgbd/set_color_exposure:
orbdec_camera_msgs/srv/SetInt32
/chassis_front_rgbd/set_color_gain:
orbdec_camera_msgs/srv/SetInt32
/chassis_front_rgbd/set_color_mirror: std_srvs/srv/SetBool
/chassis_front_rgbd/set_color_sharpness:
orbdec_camera_msgs/srv/SetInt32
/chassis_front_rgbd/set_color_white_balance:
orbdec_camera_msgs/srv/SetInt32
/chassis_front_rgbd/set_depth_auto_exposure:
std_srvs/srv/SetBool
/chassis_front_rgbd/set_depth_exposure:

```

```

orbdec_camera_msgs/srv/SetInt32
/chassis_front_rgbd/set_depth_gain:
orbdec_camera_msgs/srv/SetInt32
/chassis_front_rgbd/set_depth_mirror: std_srvs/srv/SetBool
/chassis_front_rgbd/toggle_color: std_srvs/srv/SetBool
/chassis_front_rgbd/toggle_depth: std_srvs/srv/SetBool

```

- 其中奥比相机服务接口数据格式如下，详情请见奥比官方 SDK:
https://github.com/orbbec/OrbbecSDK_ROS2/tree/v2-main

5.3.2 腰部奥比中光 Gemini 2L 相机

- 说明：获取腰部 Gemini 2L 数据
- 默认提供话题如下：

```

Bash
/sensor/camera/waist_front_rgbd/color/info:
sensor_msgs/msg/CameraInfo
/sensor/camera/waist_front_rgbd/color/raw:
shm_msgs/msg/Image1m
/sensor/camera/waist_front_rgbd/depth/info:
sensor_msgs/msg/CameraInfo
/sensor/camera/waist_front_rgbd/depth/raw:
shm_msgs/msg/Image1m
/sensor/camera/waist_front_rgbd/mix/raw:
shm_msgs/msg/Vec2Image1m

```

- 示例：
 - 话题名称： `/sensor/camera/waist_front_rgbd/color/raw`
 - 类型： `shm_msgs::msg::Image1m`
 - 数据格式

```

YAML
shm_msgs::msg::Image1m
shm_msgs/Header header #消息头
uint32 height           # 图像高度
uint32 width            # 图像宽度
shm_msgs/String encoding # 像素编码
uint8 is_bigendian      # 是否为大端序
uint32 step             # 行的总字节长度
uint8[1048576] data     # 实际矩阵数据

```

```
uint32 DATA_MAX_SIZE=1048576
```

Plain Text

```
shm_msgs/Header
  builtin_interfaces/Time stamp
  shm_msgs/String frame_id
```

```
shm_msgs/String
  char[256] data
  uint8 size 0
  uint8 MAX_SIZE=255
```

- 订阅话题:

Plain Text

```
ros2 run ubt_ros2_example waistfrontcolor_subscriber
```

- 默认提供服务如下:

Bash

```
/waist_front_rgbd/get_color_auto_white_balance:
orbbeccamera_msgs/srv/GetInt32
/waist_front_rgbd/get_color_device_info:
orbbeccamera_msgs/srv/GetDeviceInfo
/waist_front_rgbd/get_color_exposure:
orbbeccamera_msgs/srv/GetInt32
/waist_front_rgbd/get_color_gain:
orbbeccamera_msgs/srv/GetInt32
/waist_front_rgbd/get_color_sharpness:
orbbeccamera_msgs/srv/GetInt32
/waist_front_rgbd/get_color_white_balance:
orbbeccamera_msgs/srv/GetInt32
/waist_front_rgbd/get_depth_exposure:
orbbeccamera_msgs/srv/GetInt32
/waist_front_rgbd/get_depth_gain:
orbbeccamera_msgs/srv/GetInt32
/waist_front_rgbd/set_color_auto_exposure:
std_srvs/srv/SetBool
/waist_front_rgbd/set_color_auto_white_balance:
std_srvs/srv/SetBool
/waist_front_rgbd/set_color_exposure:
orbbeccamera_msgs/srv/SetInt32
```

```

/waist_front_rgbd/set_color_gain:
orbbeccamera_msgs/srv/SetInt32
/waist_front_rgbd/set_color_mirror: std_srvs/srv/SetBool
/waist_front_rgbd/set_color_sharpness:
orbbeccamera_msgs/srv/SetInt32
/waist_front_rgbd/set_color_white_balance:
std_srvs/srv/SetBool
/waist_front_rgbd/set_depth_auto_exposure:
std_srvs/srv/SetBool
/waist_front_rgbd/set_depth_exposure:
orbbeccamera_msgs/srv/SetInt32
/waist_front_rgbd/set_depth_gain:
orbbeccamera_msgs/srv/SetInt32
/waist_front_rgbd/set_depth_mirror: std_srvs/srv/SetBool
/waist_front_rgbd/toggle_color: std_srvs/srv/SetBool
/waist_front_rgbd/toggle_depth: std_srvs/srv/SetBool

```

5.4 双目相机

5.4.1 鱼眼相机接口

对于鱼眼相机以及双目相机,可能存在不同版本 image 差异请先通过 `ros2 topic info` 确定 msg 格式,再根据实际情况修改 msg 格式进行订阅

Plain Text

#例如(image 为 6m):

```
ros2 topic info /sensor/camera/stereo_left/image/raw
```

Type: shm_msgs/msg/Image6m

Publisher count: 1

Subscription count: 4

这时请将对应示例程序 image 格式修改为 6m

- 默认提供话题如下:

Plain Text

```
/sensor/camera/fisheye_left/image/raw: shm_msgs/msg/Image2m
```

```
/sensor/camera/fisheye_right/image/raw: shm_msgs/msg/Image2m
```

- 示例

- 话题名称: `/sensor/camera/fisheye_left/image/raw`
- 话题类型: `shm_msgs::msg::Image2m`
- 数据格式:

```
YAML
shm_msgs::msg::Image2m
shm_msgs/Header header #消息头
uint32 height           # 图像高度
uint32 width            # 图像宽度
shm_msgs/String encoding # 像素编码
uint8 is_bigendian      # 是否为大端序
uint32 step             # 行的总字节长度
uint8[2097152] data      # 实际矩阵数据

uint32 DATA_MAX_SIZE=2097152
```

- 示例代码

```
C++
代码块
#include "rclcpp/rclcpp.hpp"
#include "shm_msgs/msg/image2m.hpp"

class FishEyeLeftSubscriber : public rclcpp::Node
{
public:
    FishEyeLeftSubscriber() : Node("fish_eye_L_subscriber")
    {
        subscription_ = this-
>create_subscription<shm_msgs::msg::Image2m>(
        "/sensor/camera/fisheye_left/image/raw", // 话题名
        rclcpp::QoS(rclcpp::KeepLast(10)).best_effort(),
        std::bind(&FishEyeLeftSubscriber::topic_callback,
        this,
        std::placeholders::_1));
    }

private:
    void topic_callback(const
shm_msgs::msg::Image2m::SharedPtr msg)
```

```

    {
        RCLCPP_INFO(this->get_logger(), "Cruent Time: %d
sec %u nanosec",
                    msg->header.stamp.sec, msg-
>header.stamp.nanosec);
        RCLCPP_INFO(this->get_logger(), "Frame id: %s",
                    reinterpret_cast<char *>(msg-
>header.frame_id.data.data()));
        RCLCPP_INFO(this->get_logger(), "Height * Width: %u
* %u", msg->height,
                    msg->width);
        RCLCPP_INFO(this->get_logger(), "Encoding: %s",
                    reinterpret_cast<char *>(msg-
>encoding.data.data()));
        RCLCPP_INFO(this->get_logger(), "Bigendian: %u", msg-
>is_bigendian);
        RCLCPP_INFO(this->get_logger(), "Step: %u", msg-
>step);
        RCLCPP_INFO(this->get_logger(), "Matrix data
length: %zu",
                    sizeof(msg->data) / sizeof(msg->data[0]));
    }

    rclcpp::Subscription<shm_msgs::msg::Image2m>::SharedPtr
subscription_;
};

int main(int argc, char *argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<FishEyeLeftSubscriber>());
    rclcpp::shutdown();
    return 0;
}

```

5.4.2 stereo 相机接口

- 默认提供话题如下：

Bash

```

/sensor/camera/stereo/depth/info: sensor_msgs/msg/CameraInfo
/sensor/camera/stereo/pointcloud/raw: sensor_msgs/msg/PointCloud2
/sensor/camera/stereo_left/image/raw: shm_msgs/msg/Image2m

```

```
/sensor/camera/stereo_right/image/raw: shm_msgs/msg/Image2m
```

- 示例
 - 话题名称: `/sensor/camera/stereo_left/image/raw`
 - 话题类型: `shm_msgs::msg::Image2m`
 - 数据格式:

```
YAML
shm_msgs::msg::Image2m
shm_msgs/Header header #消息头
uint32 height          # 图像高度
uint32 width           # 图像宽度
shm_msgs/String encoding # 像素编码
uint8 is_bigendian     # 是否为大端序
uint32 step            # 行的总字节长度
uint8[2097152] data     # 实际矩阵数据

uint32 DATA_MAX_SIZE=2097152
```

- 示例代码

```
C++
代码块
#include "rclcpp/rclcpp.hpp"
#include "shm_msgs/msg/image2m.hpp"

class StereoLeftSubscriber : public rclcpp::Node
{
public:
    StereoLeftSubscriber() : Node("stereo_L_subscriber")
    {
        subscription_ = this-
>create_subscription<shm_msgs::msg::Image2m>(
        "/sensor/camera/stereo_left/image/raw", // 话题名
        rclcpp::QoS(rclcpp::KeepLast(10)).best_effort(),
        std::bind(&StereoLeftSubscriber::topic_callback,
        this,
        std::placeholders::_1));
    }
}
```



```

private:
    void topic_callback(const
shm_msgs::msg::Image2m::SharedPtr msg)
    {
        RCLCPP_INFO(this->get_logger(), "Cruent Time: %d
sec %u nanosec",
                    msg->header.stamp.sec, msg-
>header.stamp.nanosec);
        RCLCPP_INFO(this->get_logger(), "Frame id: %s",
                    reinterpret_cast<char *>(msg-
>header.frame_id.data.data()));
        RCLCPP_INFO(this->get_logger(), "Height * Width: %u
* %u", msg->height,
                    msg->width);
        RCLCPP_INFO(this->get_logger(), "Encoding: %s",
                    reinterpret_cast<char *>(msg-
>encoding.data.data()));
        RCLCPP_INFO(this->get_logger(), "Bigendian: %u", msg-
>is_bigendian);
        RCLCPP_INFO(this->get_logger(), "Step: %u", msg-
>step);
        RCLCPP_INFO(this->get_logger(), "Matrix data
length: %zu",
                    sizeof(msg->data) / sizeof(msg->data[0]));
    }

    rclcpp::Subscription<shm_msgs::msg::Image2m>::SharedPtr
subscription_;
};

int main(int argc, char *argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<StereoLeftSubscriber>());
    rclcpp::shutdown();
    return 0;
}

```

5.5 电池

5.5.1 电池状态

- 说明：获取电池状态信息。其中上报频率为 1Hz。
- 充电状态：charge_status idle 空闲状态，charging 充电中(目前仅有这两个状态)
- 控制方式：topic
- 话题名称：/emb/battery_state
- 类型：emb_task_msgs::msg::BatteryState
- 数据格式：

```
Go
emb_task_msgs::msg::BatteryState
BatteryInfo[] batteries_states
```

```
Go
emb_task_msgs::msg::BatteryInfo
string IDLE=idle
string CHARGING=charging
string DISCHARGING=discharging
string FULL=full
string charge_status          #充放电状态

float32  voltage              #电池总电压
float32  current              #电池总电流
float32  temperature          #电池温度
float32  maxdifvol            #最大压差
float32  batsoc                #电量剩余百分比
float32  remainchargetime     #剩余充电时间
uint16   healthstatus         #健康状态
float32  remainuselife        #电池剩余寿命(循环周期数)
```

- healthstatus：具体的健康状态如下列表

	序号	故障等级	故障码	故障描述
1	1	头部板一级故障	0x0000	无故障
2	2		0x0001	光线传感器故障
3	3		0x0002	参数设置错误
4	4		0x0003	通讯故障
5	5		0x0004	灯光故障
6	6		0x0005	接口故障
7	7		0x0006	发送故障
8	8		0x0007	CAN总线故障
9	9		0x0008	CAN物理层故障

点击图片可查看完整电子表格

- 代码示例

```
C++
#include "emb_task_msgs/msg/battery_state.hpp"
#include "rclcpp/rclcpp.hpp"

class BatterySubscriber : public rclcpp::Node
{
public:
    BatterySubscriber() : Node("battery_subscriber")
    {
        subscription_ = this->
        create_subscription<emb_task_msgs::msg::BatteryState>(
            "/emb/battery_state",
            rclcpp::QoS(rclcpp::KeepLast(10)).best_effort(),
            std::bind(&BatterySubscriber::topic_callback, this,
                std::placeholders::_1));
    }

private:
    void topic_callback(const
    emb_task_msgs::msg::BatteryState::SharedPtr msg)
    {
        int size = msg->batteries_states.size();
        RCLCPP_INFO(this->get_logger(), "-----
    ");
        for (int i = 0; i < size; i++)
        {
            RCLCPP_INFO(this->get_logger(), "-----Battery %d state:----
            -", i + 1);
        }
    }
};
```

```

        RCLCPP_INFO(this->get_logger(), " Charge status: %s",
                    msg->batteries_states[i].charge_status.c_str());
        RCLCPP_INFO(this->get_logger(), " Voltage: %.3f V",
                    msg->batteries_states[i].voltage);
        RCLCPP_INFO(this->get_logger(), " Current: %.3f A",
                    msg->batteries_states[i].current);
        RCLCPP_INFO(this->get_logger(), " Temperature: %.3f °C",
                    msg->batteries_states[i].temperature);
        RCLCPP_INFO(this->get_logger(), " Max voltage diff: %.3f
V",
                    msg->batteries_states[i].maxdifvol);
        RCLCPP_INFO(this->get_logger(), " Battery SOC: %.3f %%",
                    msg->batteries_states[i].batsoc);
        RCLCPP_INFO(this->get_logger(), " Remaining charge
time: %.3f s",
                    msg->batteries_states[i].remainchargetime);
        RCLCPP_INFO(this->get_logger(), " Health status: %u",
                    msg->batteries_states[i].healthstatus);
        RCLCPP_INFO(this->get_logger(), " Remaining use life: %.3f
times",
                    msg->batteries_states[i].remainuselife);
    }
}

rclcpp::Subscription<emb_task_msgs::msg::BatteryState>::SharedPtr
subscription_;
};

int main(int argc, char *argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<BatterySubscriber>());
    rclcpp::shutdown();
    return 0;
}

```

5.5.2 电源板状态

- 说明：获取电池状态信息。其中上报频率为 4.8Hz。
- 控制方式：topic
- 话题名称： `/emb/powerboard_innerdata`

- 类型: `emb_task_msgs::msg::InnerData`
- 数据格式:

```
Go
emb_task_msgs::msg::InnerData
float32    adc_orin_value
float32    adc_orin_ibus_value      #Orin 电流   A
float32    adc_arm_ibus_value
float32    adc_ibus_value
float32    adc_leftleg_ibus_value
float32    adc_rightleg_ibus_value
float32    adc_waist_ibus_value     #腰部电流  A
float32    adc_charge_det_value
float32    adc_vdc1_value
float32    adc_mos_temp
float32    adc_1v5
float32    temptature
float32    vrefint
float32    adc_x86_ibus_value       #x86 电流    A
float32    adc_rk_ibus_value        #rk 电流    A
float32    adc_3vref_value          #3v 参考电压  V
uint16     err_code

uint8 second      #秒
uint8 minute      #分
uint8 hour         #时
uint8 day          #日
uint8 month        #月
uint16 year        #年
```

```
C++
#include "emb_task_msgs/msg/inner_data.hpp"
#include "rclcpp/rclcpp.hpp"

class InnerDataSubscriber : public rclcpp::Node
{
public:
    InnerDataSubscriber() : Node("inner_data_subscriber")
    {
        subscription_ = this-
```

```

>create_subscription<emb_task_msgs::msg::InnerData>(
    "/emb/powerboard_innerdata", // 话题名称
    rclcpp::QoS(rclcpp::KeepLast(10)).best_effort(),
    std::bind(&InnerDataSubscriber::topic_callback, this,
        std::placeholders::_1));
}

private:
    void topic_callback(const
emb_task_msgs::msg::InnerData::SharedPtr msg)
    {
        RCLCPP_INFO(this->get_logger(), "----- Inner Data -----");
        RCLCPP_INFO(this->get_logger(), "Orin Voltage: %.3f V",
            msg->adc_orin_value);
        RCLCPP_INFO(this->get_logger(), "Orin Current: %.3f A",
            msg->adc_orin_ibus_value);
        RCLCPP_INFO(this->get_logger(), "Arm Current: %.3f A",
            msg->adc_arm_ibus_value);
        RCLCPP_INFO(this->get_logger(), "Total Current: %.3f A",
            msg->adc_ibus_value);
        RCLCPP_INFO(this->get_logger(), "Left Leg Current: %.3f A",
            msg->adc_leftleg_ibus_value);
        RCLCPP_INFO(this->get_logger(), "Right Leg Current: %.3f A",
            msg->adc_rightleg_ibus_value);
        RCLCPP_INFO(this->get_logger(), "Waist Current: %.3f A",
            msg->adc_waist_ibus_value);
        RCLCPP_INFO(this->get_logger(), "Charge Voltage: %.3f V",
            msg->adc_charge_det_value);
        RCLCPP_INFO(this->get_logger(), "Total Voltage: %.3f V",
            msg->adc_vdc1_value);
        RCLCPP_INFO(this->get_logger(), "MOSFET Temp: %.3f °C", msg-
>adc_mos_temp);
        RCLCPP_INFO(this->get_logger(), "5V Output Voltage: %.3f V",
msg->adc_1v5);
        RCLCPP_INFO(this->get_logger(), "Chip Temp: %.3f °C", msg-
>temptature);
        RCLCPP_INFO(this->get_logger(), "Reference Voltage: %.3f V",
msg->vrefint);
        RCLCPP_INFO(this->get_logger(), "X86 Current: %.3f A",
            msg->adc_x86_ibus_value);
        RCLCPP_INFO(this->get_logger(), "RK Current: %.3f A",
            msg->adc_rk_ibus_value);
        RCLCPP_INFO(this->get_logger(), "3V Reference Voltage: %.3f
V",

```

```

        msg->adc_3vref_value);
    RCLCPP_INFO(this->get_logger(), "Error Code: %u", msg-
>err_code);
    }

    rclcpp::Subscription<emb_task_msgs::msg::InnerData>::SharedPtr
subscription_;
};

int main(int argc, char *argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<InnerDataSubscriber>());
    rclcpp::shutdown();
    return 0;
}

```

5.6 音频接口

5.6.1 音频播放

- 说明: 通过喇叭播放已有的音频文件或通过 Tts 合成语音
- 控制方式: Action
- 音频播放: 通过 goal 控制, 当 type=1 为 Tts 语音合成播放, 当 type=0 为指定文件播放
- 话题名称: /sys/speech/tts
- 数据格式: sys_task_msgs::action::tts
- 格式要求: 仅支持 16kHz 格式的 wav

```

YAML
sys_task_msgs::action::tts

uint8 FILE = 0
uint8 TTS = 1
uint8 type 1

# all valid
bool is_break true #是否打断

```

```

# Only file is valid
string file_path "" #文件路径

# Only tts is valid
string text "" #Tts 合成语音文本
string speaker "male_01" #声优选择,默认 male_01
int32 speed 50 #Tts 语音速度
int32 volume 100 #Tts 语音音量
int32 pitch 50 #Tts 语音音调
string language "zh" #Tts 语言
string format "wav" #音频文件格式支持.wav 格式
bool need_save true #是否缓存

---
rosa_msgs/NodeState result
---
```

- 示例 Demo 运行

音频文件传输到/debug 目录下

YAML

```

ros2 run ubt_ros2_example audio_player --ros-args -p
file_path:=/path/to/your/file.wav
```

- 示例代码

C++

```

#include <rclcpp/rclcpp.hpp>
#include <rclcpp_action/rclcpp_action.hpp>
#include "sys_task_msgs/action/tts.hpp"
#include <future> // for std::promise / std::future

using namespace std::chrono_literals;

class SpeechClient : public rclcpp::Node
{
public:
    SpeechClient() : Node("simple_speech_client")
    {
        // 创建 Action 客户端
        action_client_ =
rclcpp_action::create_client<sys_task_msgs::action::Tts>(
```



```

        this, "/sys/speech/tts");

// 等待 Action Server 可用
if (!action_client_->wait_for_action_server(20s))
{
    RCLCPP_ERROR(this->get_logger(),
                  "Action server not available after waiting");
    return;
}

// 声明并获取参数
this->declare_parameter<std::string>("file_path", "");
this->get_parameter("file_path", file_path_);

if (file_path_.empty())
{
    RCLCPP_ERROR(this->get_logger(), "File path parameter not
provided");
    return;
}

// 发送目标请求
send_goal();
}

std::shared_future<void> get_result_future()
{
    return result_promise_.get_future();
}

private:
void send_goal()
{
    sys_task_msgs::action::Tts::Goal goal_msg;
    goal_msg.type = 0;
    goal_msg.is_break = true;
    goal_msg.file_path = file_path_;

    RCLCPP_INFO(this->get_logger(), "Sending goal with file
path: %s",
                file_path_.c_str());
}

rclcpp_action::Client<sys_task_msgs::action::Tts>::SendGoalOptions

```

```

options;

// Goal response 回调
options.goal_response_callback =
    [this](std::shared_ptr<rclcpp_action::ClientGoalHandle<
        sys_task_msgs::action::Tts>> goal_handle) {
        if (!goal_handle)
        {
            RCLCPP_ERROR(this->get_logger(), "Goal was rejected by
server");
            result_promise_.set_value(); // 提前结束
        }
        else
        {
            RCLCPP_INFO(this->get_logger(), "Goal accepted by
server");
        }
    };

// Feedback 回调（这里可以忽略或打印）
options.feedback_callback =
    [this](rclcpp_action::ClientGoalHandle<
        sys_task_msgs::action::Tts>::SharedPtr,
        const std::shared_ptr<const
sys_task_msgs::action::Tts::Feedback>
        feedback) {
        RCLCPP_INFO(this->get_logger(), "Feedback received...");
    };

// Result 回调
options.result_callback =
    [this](const rclcpp_action::ClientGoalHandle<
        sys_task_msgs::action::Tts>::WrappedResult &result)
    {
        switch (result.code)
        {
            case rclcpp_action::ResultCode::SUCCEEDED:
                RCLCPP_INFO(this->get_logger(), "Result received
successfully");
                break;
            case rclcpp_action::ResultCode::ABORTED:
                RCLCPP_ERROR(this->get_logger(), "Goal was
aborted");
                break;
        }
    };

```

```

        case rclcpp_action::ResultCode::CANCELED:
            RCLCPP_ERROR(this->get_logger(), "Goal was
canceled");
            break;
        default:
            RCLCPP_ERROR(this->get_logger(), "Unknown result
code");
            break;
    }
    result_promise_.set_value(); // 通知 main 可以退出
};

    action_client_->async_send_goal(goal_msg, options);
}

    std::string file_path_;
    rclcpp_action::Client<sys_task_msgs::action::Tts>::SharedPtr
action_client_;

    std::promise<void> result_promise_; // 用于退出控制
};

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<SpeechClient>();

    // 等待 result 回调通知
    auto future = node->get_result_future();
    rclcpp::spin_until_future_complete(node, future);

    rclcpp::shutdown();
    return 0;
}

```

- 错误码

YAML

```

#成功
int32 SUCCESS = 1001000
string SUCCESS_STR = "Success"

```

```

# 被打断
int32 INTERRUPTED = 1001001
string INTERRUPTED_STR = "Interrupted"

# websocket 无效
int32 WEBSOCKET_INVALID = 6001001
string WEBSOCKET_INVALID_STR = "Websocket cannot connect"

# file 无效
int32 FILE_INVALID = 5001002
string FILE_INVALID_STR = "Invalid file passed in"

# 大模型上传图片失败
int32 LLM_UPLOAD_IMAGE_FAILED = 5002003
string LLM_UPLOAD_IMAGE_FAILED_STR = "LLM upload image failed"

# 大模型 CHAT 超时
int32 LLM_CHAT_TIMEOUT = 5002004
string LLM_CHAT_TIMEOUT_STR = "LLM chat timeout"

# 大模型 CHAT 失败
int32 LLM_CHAT_FAILED = 5002005
string LLM_CHAT_FAILED_STR = "LLM chat failed"

```

5.6.2 麦克风数据获取

- 说明：获取 Cruzr S2 麦克风数据或者降噪后数据，原始数据为 8 通道，16KHz，降噪后数据为 1 通道，16KHz
- 获取方式：topic
- 话题名称： `/sys/speech/mic_source` 和 `/sys/speech/mic_denoise`
- 数据定义位置： `std_msgs::msg::Int16MultiArray`
- 数据格式：

```

Go
std_msgs::msg::Int16MultiArray
MultiArrayLayout layout      # specification of data layout
    #
    #
    #
    #
    #

```

```

        MultiArrayDimension[] dim #
            string label    #
            uint32 size      #
            uint32 stride    #
        uint32 data_offset    #
int16[]      data           # array of data

```

代码示例:

```

C++
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/int16_multi_array.hpp>
#include <fstream>
#include <vector>
#include <cstdint>

// WAV 文件头结构体
struct WavHeader
{
    char riff[4] = {'R', 'I', 'F', 'F'};
    uint32_t chunk_size;
    char wave[4] = {'W', 'A', 'V', 'E'};
    char fmt[4] = {'f', 'm', 't', ' '};
    uint32_t subchunk1_size = 16;
    uint16_t audio_format = 1;    // PCM
    uint16_t num_channels;
    uint32_t sample_rate;
    uint32_t byte_rate;
    uint16_t block_align;
    uint16_t bits_per_sample;
    char data[4] = {'d', 'a', 't', 'a'};
    uint32_t data_size;
};

class MicRecorder : public rclcpp::Node
{
public:
    MicRecorder()
        : Node("mic_recorder"),
          sample_rate_(16000),
          num_channels_(8)
    {
        sub_ = this->create_subscription<std_msgs::msg::Int16MultiArray>(

```

```

>create_subscription<std_msgs::msg::Int16MultiArray>(
    "/sys/speech/mic_source",
    10,
    std::bind(&MicRecorder::onMicData, this,
std::placeholders::_1));

    wav_file_.open("mic_output.wav", std::ios::binary);
    if (!wav_file_)
    {
        RCLCPP_ERROR(this->get_logger(), "Failed to open
output file");
        return;
    }

    // 先写一个占位的 WAV 头，之后会回填大小
    writeEmptyHeader();
    RCLCPP_INFO(this->get_logger(), "Recording started...");
}

~MicRecorder()
{
    finalizeWav();
    if (wav_file_.is_open())
        wav_file_.close();
    RCLCPP_INFO(this->get_logger(), "Recording stopped. Saved
to mic_output.wav");
}

private:
    void onMicData(const std_msgs::msg::Int16MultiArray::SharedPtr
msg)
    {
        // 将数据直接写入文件
        wav_file_.write(reinterpret_cast<const char *>(msg-
>data.data()),
                        msg->data.size() * sizeof(int16_t));
        data_bytes_ += msg->data.size() * sizeof(int16_t);
    }

    void writeEmptyHeader()
    {
        WavHeader header;
        header.chunk_size = 0;
        header.num_channels = num_channels_;
    }

```

```

        header.sample_rate = sample_rate_;
        header.bits_per_sample = 16;
        header.byte_rate = sample_rate_ * num_channels_ *
header.bits_per_sample / 8;
        header.block_align = num_channels_ *
header.bits_per_sample / 8;
        header.data_size = 0;

        wav_file_.write(reinterpret_cast<const char *>(&header),
sizeof(WavHeader));
    }

    void finalizeWav()
    {
        if (!wav_file_)
            return;

        wav_file_.seekp(0, std::ios::beg);

        WavHeader header;
        header.num_channels = num_channels_;
        header.sample_rate = sample_rate_;
        header.bits_per_sample = 16;
        header.byte_rate = sample_rate_ * num_channels_ *
header.bits_per_sample / 8;
        header.block_align = num_channels_ *
header.bits_per_sample / 8;
        header.data_size = data_bytes_;
        header.chunk_size = 36 + data_bytes_;

        wav_file_.write(reinterpret_cast<const char *>(&header),
sizeof(WavHeader));
    }

    rclcpp::Subscription<std_msgs::msg::Int16MultiArray>::SharedPtr
sub_;
    std::ofstream wav_file_;
    size_t data_bytes_ = 0;
    uint32_t sample_rate_;
    uint16_t num_channels_;
};

int main(int argc, char *argv[])

```

```
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<MicRecorder>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

如果需要降噪数据，请将示例 Demo 程序话题更改为/sys/speech/mic_denoise，并将通道数(num_channels_)设置为 1

5.7 灵巧手-三代手

5.7.1 状态获取接口

- 说明：获取灵巧手关节状态信息
- 获取方式：topic
- 话题名称：/mc/L_hand/joint_states 和 /mc/R_hand/joint_states
- 数据定义位置：sensor_msgs::msg::JointState
- 数据格式：

```
Go
std_msgs/Header header
    builtin_interfaces/Time stamp
        int32 sec
        uint32 nanosec
    string frame_id

string[] name
float64[] position
float64[] velocity
float64[] effort
```

- 代码示例：

```
C++
代码块
#include <iostream>
```



```

#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/joint_state.hpp"
#include "std_msgs/msg/string.hpp"

class HandsStateSubscriber : public rclcpp::Node {
public:
    HandsStateSubscriber() : Node("sub_hands_state") {
        // 创建 QoS 配置，使用系统默认的传感器数据 QoS
        rclcpp::QoS qos_settings(10);

        qos_settings.reliability(RMW_QOS_POLICY_RELIABILITY_BEST_EFFORT);
        qos_settings.durability(RMW_QOS_POLICY_DURABILITY_VOLATILE);
        qos_settings.history(RMW_QOS_POLICY_HISTORY_KEEP_LAST);

        // 订阅左手关节状态
        L_hand_subscription_ = this-
>create_subscription<sensor_msgs::msg::JointState>(
            "/mc/L_hand/joint_states", qos_settings,
            [this](const sensor_msgs::msg::JointState::SharedPtr msg)
        { this->L_hand_callback(msg); });

        // 订阅右手关节状态
        R_hand_subscription_ = this-
>create_subscription<sensor_msgs::msg::JointState>(
            "/mc/R_hand/joint_states", qos_settings,
            [this](const sensor_msgs::msg::JointState::SharedPtr msg)
        { this->R_hand_callback(msg); });
    }

private:
    void L_hand_callback(const
sensor_msgs::msg::JointState::SharedPtr msg) const {
        RCLCPP_INFO(this->get_logger(), "==== Left Hand Joint
States =====");

        for (size_t i = 0; i < msg->name.size(); i++) {
            std::cout << "Joint: " << msg->name[i] << " Position: " <<
msg->position[i] << std::endl;
        }
    }

    void R_hand_callback(const

```

```

sensor_msgs::msg::JointState::SharedPtr msg) const {
    RCLCPP_INFO(this->get_logger(), "==== Right Hand Joint
States =====");

    for (size_t i = 0; i < msg->name.size(); i++) {
        std::cout << "Joint: " << msg->name[i] << " Position: " <<
msg->position[i] << std::endl;
    }
}

rclcpp::Subscription<sensor_msgs::msg::JointState>::SharedPtr
L_hand_subscription_;
rclcpp::Subscription<sensor_msgs::msg::JointState>::SharedPtr
R_hand_subscription_;
};

int main(int argc, char const* argv[]) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<HandsStateSubscriber>());
    rclcpp::shutdown();
    return EXIT_SUCCESS;
}

```

5.7.2 控制接口

- 说明：控制三代灵巧手
- 控制方式：topic
- 话题名称：/mc/L_hand/command 或者 /mc/R_hand/command
- 类型：mc_task_msgs::msg::JointCommand
- 数据格式：

```

Go
std_msgs/Header header
    builtin_interfaces/Time stamp
        int32 sec
        uint32 nanosec
    string frame_id
int32[] mode
float64[] position
float64[] velocity
float64[] torque
float64[] acceleration

```

```

string[] names
float64[] kp
float64[] kd
int32 POSITION_MODE=1
int32 VELOCITY_MODE=2
int32 TORQUE_MODE=3
int32 RAW_POSITION_MODE=4

```

- 代码示例:

C++

代码块

```

#include <chrono>
#include <cmath>
#include <mc_task_msgs/msg/joint_command.hpp>
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/header.hpp>

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("pub_hand_command");

    // Create publishers for left and right hand controllers
    auto L_hand_publisher = node->create_publisher<mc_task_msgs::msg::JointCommand>("/mc/L_hand/command", 10);
    auto R_hand_publisher = node->create_publisher<mc_task_msgs::msg::JointCommand>("/mc/R_hand/command", 10);

    // Define joint names for left hand
    std::vector<std::string> L_joint_names = {"L_thumb_swing",
        "L_thumb_mcp", "L_index_mcp",
        "L_middle_mcp",
        "L_ring_mcp", "L_little_mcp"};

    // Define joint names for right hand
    std::vector<std::string> R_joint_names = {"R_thumb_swing",
        "R_thumb_mcp", "R_index_mcp",
        "R_middle_mcp",
        "R_ring_mcp", "R_little_mcp"};

    rclcpp::Rate rate(500);
    double time_cnt = 0.0;

```

```

static const double CONTROL_DURATION = 5.0; // 控制持续时间(秒)
static const double RESET_DURATION = 4.0;   // 回零持续时间(秒)

// 计算总关节数(左手+右手)
size_t total_joints = L_joint_names.size() +
R_joint_names.size();

while (rclcpp::ok()) {
    // 计算当前应该控制哪个关节
    size_t cycle_time =
        static_cast<size_t>(time_cnt) %
static_cast<size_t>(total_joints * (CONTROL_DURATION +
RESET_DURATION));
    size_t joint_index = cycle_time /
static_cast<size_t>(CONTROL_DURATION + RESET_DURATION);
    size_t phase_in_cycle = cycle_time %
static_cast<size_t>(CONTROL_DURATION + RESET_DURATION);

    bool is_control_phase = (phase_in_cycle < CONTROL_DURATION);

    // 初始化命令消息
    mc_task_msgs::msg::JointCommand L_cmd;
    L_cmd.header.stamp = node->now();
    L_cmd.names.resize(L_joint_names.size(), "");
    L_cmd.position.resize(L_joint_names.size(), 0.0);
    L_cmd.velocity.resize(L_joint_names.size(), 0.0);
    L_cmd.mode.resize(L_joint_names.size(), 0);

    mc_task_msgs::msg::JointCommand R_cmd;
    R_cmd.header.stamp = node->now();
    R_cmd.names.resize(R_joint_names.size(), "");
    R_cmd.position.resize(R_joint_names.size(), 0.0);
    R_cmd.velocity.resize(L_joint_names.size(), 0.0);
    R_cmd.mode.resize(R_joint_names.size(), 0);

    if (joint_index < L_joint_names.size()) {
        // 控制左手关节
        size_t idx = joint_index;
        L_cmd.names[idx] = L_joint_names[idx];
        L_cmd.mode[idx] = 5;

        if (is_control_phase) {
            // 控制阶段：设置目标位置
            L_cmd.position[idx] = sin(time_cnt * 2.0) * 0.3;

```

```

    } else {
        // 回零阶段：逐渐回到零位
        L_cmd.position[idx] = 0.0;
    }

    // 其他关节保持零位但需要填充
    for (size_t i = 0; i < L_joint_names.size(); i++) {
        if (i != idx) {
            L_cmd.names[i] = L_joint_names[i];
            L_cmd.position[i] = 0.0;
            L_cmd.mode[i] = 5;
        }
    }
} else {
    // 控制右手关节
    size_t idx = joint_index - L_joint_names.size();
    R_cmd.names[idx] = R_joint_names[idx];
    R_cmd.mode[idx] = 5;

    if (is_control_phase) {
        // 控制阶段：设置目标位置
        R_cmd.position[idx] = sin(time_cnt * 2.0) * 0.3;
    } else {
        // 回零阶段：逐渐回到零位
        R_cmd.position[idx] = 0.0;
    }

    // 其他关节保持零位但需要填充
    for (size_t i = 0; i < R_joint_names.size(); i++) {
        if (i != idx) {
            R_cmd.names[i] = R_joint_names[i];
            R_cmd.position[i] = 0.0;
            R_cmd.mode[i] = 5;
        }
    }
}

// 发布命令
L_hand_publisher->publish(L_cmd);
R_hand_publisher->publish(R_cmd);

time_cnt += 0.002;
rate.sleep();
}

```

```

    rclcpp::shutdown();
    return EXIT_SUCCESS;
}

```

- 注意：
1. HW_TYPE=cruze_s2_v1_sps
 2. 灵巧手的控制只支持位置控制，且 msg 中位置要按照 demo 中所给顺序赋值；
 3. 如只控制部分关节，则对于不控制的关节，控制量设置为 0，使其置于零位

5.8 雷达

- 说明：机器人配备两个雷达，前后各一个
- 控制方式：topic
- 话题名称：/sensor/lidar/front 和 /sensor/lidar/back
- 话题类型：sensor_msgs::msg::LaserScan
- 数据格式：

Python

```
sensor_msgs::msg::LaserScan
```

```
std_msgs/Header header # 消息头，包含时间戳和帧 id
```

```
float32 angle_min # 扫描的起始角度（弧度）
```

```
float32 angle_max # 扫描的结束角度（弧度）
```

```
float32 angle_increment # 每次测量的角度增量（弧度）
```

```
float32 time_increment # 每次测量的时间间隔（秒）
```

```
float32 scan_time # 扫描的时间间隔（秒）
```

```
float32 range_min # 最小测量距离（米）
```

```
float32 range_max # maximum range value [m]
```

```
float32[] ranges # 测量的距离数据（米）
```

```
float32[] intensities # 测量的强度数据
```

- 代码示例：

C++

```
#include "rclcpp/rclcpp.hpp"
```

```

#include "sensor_msgs/msg/laser_scan.hpp"

class LidarFrontSubscriber : public rclcpp::Node
{
public:
    LidarFrontSubscriber() : Node("lidar_front_subscriber")
    {
        subscription_ = this-
>create_subscription<sensor_msgs::msg::LaserScan>(
        "/sensor/lidar/front",
        rclcpp::QoS(rclcpp::KeepLast(10)).best_effort(),
        std::bind(&LidarFrontSubscriber::topic_callback, this,
        std::placeholders::_1));
    }

private:
    void topic_callback(const sensor_msgs::msg::LaserScan::SharedPtr
msg)
    {
        RCLCPP_INFO(this->get_logger(), "Cruent Time: %d sec %u
nanosec",
                    msg->header.stamp.sec, msg->header.stamp.nanosec);
        RCLCPP_INFO(this->get_logger(), "Frame id: %s",
                    msg->header.frame_id.c_str());
        RCLCPP_INFO(this->get_logger(), "Angle Min: %.3f rad; Angle
Max: %.3f rad",
                    msg->angle_min, msg->angle_max);
        RCLCPP_INFO(this->get_logger(), "Angle Increment: %.3f rad",
                    msg->angle_increment);
        RCLCPP_INFO(this->get_logger(), "Time Increment: %.3f s",
                    msg->time_increment);
        RCLCPP_INFO(this->get_logger(), "Scan Time: %.3f s", msg-
>scan_time);
        RCLCPP_INFO(this->get_logger(), "Range Min: %.3f m; Range
Max: %.3f m",
                    msg->range_min, msg->range_max);
    }

    rclcpp::Subscription<sensor_msgs::msg::LaserScan>::SharedPtr
subscription_;
};

int main(int argc, char *argv[])
{

```

```

rclcpp::init(argc, argv);
rclcpp::spin(std::make_shared<LidarFrontSubscriber>());
rclcpp::shutdown();
return 0;
}

```

5.9 灵巧手-四代手

5.9.1 状态获取接口

- 说明：获取灵巧手关节状态信息
- 获取方式：topic
- 话题名称：/mc/left_hand/joint_states 和 /mc/right_hand/joint_states
- 数据定义位置：sensor_msgs::msg::JointState
- 数据格式：

```

Go
std_msgs/Header header
    builtin_interfaces/Time stamp
        int32 sec
        uint32 nanosec
    string frame_id

string[] name
float64[] position
float64[] velocity
float64[] effort

```

- 代码示例：

```

C++
#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/joint_state.hpp"
#include "std_msgs/msg/string.hpp"
#include <iostream>
#include <memory>

class HandsStateSubscriber : public rclcpp::Node
{
public:

```



```

HandsStateSubscriber() : Node("sub_hands_state")
{
    // 创建 QoS 配置, 使用系统默认的传感器数据 QoS
    rclcpp::QoS qos_settings(10);

    qos_settings.reliability(RMW_QOS_POLICY_RELIABILITY_BEST_EFFORT);
    qos_settings.durability(RMW_QOS_POLICY_DURABILITY_VOLATILE);
    qos_settings.history(RMW_QOS_POLICY_HISTORY_KEEP_LAST);

    // 订阅左手关节状态
    left_hand_subscription_ = this-
>create_subscription<sensor_msgs::msg::JointState>(
    "/mc/left_hand/joint_states",
    qos_settings,
    [this](const sensor_msgs::msg::JointState::SharedPtr msg) {
        this->left_hand_callback(msg);
    });

    // 订阅右手关节状态
    right_hand_subscription_ = this-
>create_subscription<sensor_msgs::msg::JointState>(
    "/mc/right_hand/joint_states",
    qos_settings,
    [this](const sensor_msgs::msg::JointState::SharedPtr msg) {
        this->right_hand_callback(msg);
    });
}

private:
    void left_hand_callback(const
sensor_msgs::msg::JointState::SharedPtr msg) const
    {
        RCLCPP_INFO(this->get_logger(), "==== Left Hand Joint
States =====");

        for (size_t i = 0; i < msg->name.size(); i++) {
            std::cout << "Joint: " << msg->name[i]
                << " Position: " << msg->position[i] << std::endl;
        }
    }

    void right_hand_callback(const
sensor_msgs::msg::JointState::SharedPtr msg) const
    {

```

```

    RCLCPP_INFO(this->get_logger(), "==== Right Hand Joint
States =====");

    for (size_t i = 0; i < msg->name.size(); i++) {
        std::cout << "Joint: " << msg->name[i]
            << " Position: " << msg->position[i] << std::endl;
    }
}

rclcpp::Subscription<sensor_msgs::msg::JointState>::SharedPtr
left_hand_subscription_;
rclcpp::Subscription<sensor_msgs::msg::JointState>::SharedPtr
right_hand_subscription_;
};

int main(int argc, char const* argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<HandsStateSubscriber>());
    rclcpp::shutdown();
    return EXIT_SUCCESS;
}

```

5.9.2 控制接口

- 说明：控制四代灵巧手
- 控制方式：topic
- 话题名称：/mc/left_hand/command 或者 /mc/right_hand/command
- 类型：mc_task_msgs::msg::JointCommand
- 数据格式：

```

Go
std_msgs/Header header
    builtin_interfaces/Time stamp
        int32 sec
        uint32 nanosec
    string frame_id
int32[] mode
float64[] position
float64[] velocity
float64[] torque
float64[] acceleration

```

```

string[] names
float64[] kp
float64[] kd
int32 POSITION_MODE=1
int32 VELOCITY_MODE=2
int32 TORQUE_MODE=3
int32 RAW_POSITION_MODE=4

```

- 代码示例:

```

C++
#include <mc_task_msgs/msg/joint_command.hpp>

#include <chrono>
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/header.hpp>
#include <cmath>

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("pub_hand_command");

    // Create publishers for left and right hand controllers
    auto left_hand_publisher = node-
>create_publisher<mc_task_msgs::msg::JointCommand>(
    "/mc/left_hand/command", 10);
    auto right_hand_publisher = node-
>create_publisher<mc_task_msgs::msg::JointCommand>(
    "/mc/right_hand/command", 10);

    // Define joint names for left hand
    std::vector<std::string> left_joint_names = {
        "left_thumb_swing",
        "left_thumb_mcp",
        "left_thumb_pip",
        "left_index_mcp",
        "left_middle_mcp",
        "left_ring_mcp",
        "left_little_mcp"
    };

    // Define joint names for right hand
    std::vector<std::string> right_joint_names = {
        "right_thumb_swing",

```

```

    "right_thumb_mcp",
    "right_thumb_pip",
    "right_index_mcp",
    "right_middle_mcp",
    "right_ring_mcp",
    "right_little_mcp"
};

rclcpp::Rate rate(500);
double time_cnt = 0.0;

while (rclcpp::ok()) {
    // Publish joint commands for left hand
    mc_task_msgs::msg::JointCommand left_cmd;
    left_cmd.header.stamp = node->now();
    left_cmd.names.resize(left_joint_names.size());
    left_cmd.position.resize(left_joint_names.size());
    left_cmd.mode.resize(left_joint_names.size());

    for (size_t i = 0; i < left_joint_names.size(); i++) {
        left_cmd.names[i] = left_joint_names[i];
        left_cmd.position[i] = sin(time_cnt + i * 0.2) * 0.6; //
Add phase difference for each joint
        left_cmd.mode[i] = 5;
    }

    // Publish joint commands for right hand
    mc_task_msgs::msg::JointCommand right_cmd;
    right_cmd.header.stamp = node->now();
    right_cmd.names.resize(right_joint_names.size());
    right_cmd.position.resize(right_joint_names.size());
    right_cmd.mode.resize(right_joint_names.size());

    for (size_t i = 0; i < right_joint_names.size(); i++) {
        right_cmd.names[i] = right_joint_names[i];
        right_cmd.position[i] = sin(time_cnt + i * 0.2) * 0.6; //
Add phase difference for each joint
        right_cmd.mode[i] = 5;
    }

    // Publish commands
    left_hand_publisher->publish(left_cmd);
    right_hand_publisher->publish(right_cmd);
}

```

```

    time_cnt += 0.002;
    rate.sleep();
}

rclcpp::shutdown();
return EXIT_SUCCESS;
}

```

- 注意：

1. HW_TYPE=cruze_s2_v1_sps
2. 灵巧手的控制只支持位置控制，且 msg 中位置要按照 demo 中所给顺序赋值，注意不要设置为空值，且顺序要对应；
3. 如只控制部分关节，则对于不控制的关节，控制量设置为 0，使其置于零位

5.10 夹爪-大寰 PGC-140-50

5.10.1 状态获取接口

- 说明：获取大寰夹爪关节状态信息
- 获取方式：topic
- 话题名称：/ecat/left_grip/state 和 /ecat/right_grip/state
- 数据定义位置：ecat_task_msgs::msg::GripStatus
- 数据格式：

```

Go
shm_msgs/Header header #时间戳

uint16 init_state #1 初始化成功
uint16 grip_state #0:运动中 1:到位 2:夹持中 3:掉落
uint16 error_code #错误码
uint16 homed #回零状态，1: 已回零

float64 pos #实际位置，unit: M
float64 vel #实际速度，unit: M/s
float64 cur #实际电流，unit: A

```

- 代码示例：

C++

```

// Copyright 2025 UBTECH. All rights reserved.
//
// Demo: 订阅大寰(DH)夹爪状态信息 (ROS2 版本)
//
// 话题信息:
//   状态话题: /ecat/left_grip/state (左夹爪)
//             /ecat/right_grip/state (右夹爪)
//   消息类型: ecat_task_msgs::msg::GripStatus
//
// GripStatus 字段说明:
//   header      - 消息头 (shm_msgs::msg::Header)
//   init_state  - 初始化状态 (uint16): 1=初始化成功
//   grip_state  - 夹爪状态 (uint16): 0=运动中, 1=到位, 2=夹持中,
//   3=掉落
//   error_code  - 错误码 (uint16)
//   homed       - 回零状态 (uint16): 1=已回零
//   pos         - 实际位置 (double): 单位 m
//   vel         - 实际速度 (double): 单位 m/s
//   cur         - 实际电流 (double): 单位 A

#include <iomanip>
#include <iostream>
#include <memory>
#include <mutex>
#include <string>

#include <rclcpp/rclcpp.hpp>
#include <ecat_task_msgs/msg/grip_status.hpp>

using GripStatusMsg = ecat_task_msgs::msg::GripStatus;

// 将 grip_state 数值转为可读字符串
std::string GripStateToString(uint16_t state) {
    switch (state) {
        case 0: return "Moving ";
        case 1: return "Reached";
        case 2: return "Grip ";
        case 3: return "Dropped";
        default: return "Unknown";
    }
}

struct GripData {

```

```

    bool valid = false;
    uint16_t init_state = 0;
    uint16_t grip_state = 0;
    uint16_t error_code = 0;
    uint16_t homed = 0;
    double pos = 0.0;
    double vel = 0.0;
    double cur = 0.0;
};

std::mutex g_mutex;
GripData g_left, g_right;

void PrintState() {
    std::lock_guard<std::mutex> lock(g_mutex);
    std::cout << "\r";
    if (g_left.valid) {
        std::cout << "L[" << GripStateToString(g_left.grip_state)
            << " p=" << std::fixed << std::setprecision(4) <<
g_left.pos
            << " v=" << std::setprecision(3) << g_left.vel
            << " i=" << g_left.cur << "];";
    } else {
        std::cout << "L[--]";
    }
    std::cout << " ";
    if (g_right.valid) {
        std::cout << "R[" << GripStateToString(g_right.grip_state)
            << " p=" << std::fixed << std::setprecision(4) <<
g_right.pos
            << " v=" << std::setprecision(3) << g_right.vel
            << " i=" << g_right.cur << "];";
    } else {
        std::cout << "R[--]";
    }
    std::cout << " " << std::flush;
}

void LeftGripCallback(const GripStatusMsg::SharedPtr msg) {
    {
        std::lock_guard<std::mutex> lock(g_mutex);
        g_left.valid = true;
        g_left.init_state = msg->init_state;
        g_left.grip_state = msg->grip_state;
    }
}

```

```

        g_left.error_code = msg->error_code;
        g_left.homed = msg->homed;
        g_left.pos = msg->pos;
        g_left.vel = msg->vel;
        g_left.cur = msg->cur;
    }
    PrintState();
}

void RightGripCallback(const GripStatusMsg::SharedPtr msg) {
    {
        std::lock_guard<std::mutex> lock(g_mutex);
        g_right.valid = true;
        g_right.init_state = msg->init_state;
        g_right.grip_state = msg->grip_state;
        g_right.error_code = msg->error_code;
        g_right.homed = msg->homed;
        g_right.pos = msg->pos;
        g_right.vel = msg->vel;
        g_right.cur = msg->cur;
    }
    PrintState();
}

int main(int argc, char const* argv[])
{
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("sub_gripper_state");

    auto qos = rclcpp::QoS(10).best_effort();

    // 订阅左夹爪状态
    auto left_grip_sub = node->create_subscription<GripStatusMsg>(
        "/ecat/left_grip/state", qos,
        [] (GripStatusMsg::SharedPtr msg)
    { LeftGripCallback(msg); });

    // 订阅右夹爪状态
    auto right_grip_sub = node->create_subscription<GripStatusMsg>(
        "/ecat/right_grip/state", qos,
        [] (GripStatusMsg::SharedPtr msg)
    { RightGripCallback(msg); });

    std::cout << "=== DH Gripper State Subscriber Demo (ROS2) ==="

```



```

<< std::endl;
    std::cout << "Subscribing to gripper states from:" << std::endl;
    std::cout << "  Left:  /ecat/left_grip/state" << std::endl;
    std::cout << "  Right: /ecat/right_grip/state" << std::endl;
    std::cout << "Press Ctrl+C to stop." << std::endl;

    rclcpp::spin(node);

    std::cout << std::endl;
    rclcpp::shutdown();
    return EXIT_SUCCESS;
}

```

5.10.2 控制接口

- 说明：控制大寰 PGC-140-50 夹爪
- 控制方式：topic
- 话题名称：/ecat/left_grip/cmd 或者 /ecat/right_grip/cmd
- 类型：ecat_task_msgs::msg::GripCmd
- 数据格式：

Go

shm_msgs/Header header #时间戳

大寰夹爪使用方法

整机回零的时候大寰夹爪也会回零，然后给位置（行程）和力矩（夹持力）指令就行

位置范围[0-0.05] 单位米

力矩范围[0-100] 单位 N

速度范围[0.75] m/s

大寰夹爪使用方法

大寰电缸使用方法

整机回零的时候大寰电缸也会回零，

非推压模式：mode=0 给位置（行程）和力矩（推压力）指令 速度/加速度指令（不写就是默认）

推压模式：mode=10，给推压行程和力矩（推压力）指令 速度/加速度指令（不写就是默认）

备注：用 float64 cur -> 加速度 acc，给定 cur 就是给定 acc 值

位置范围[0-0.05] 单位米

```

### 力矩范围[41-100] 单位 N
### 速度范围[0-0.01] m/s
### 加速度范围[0-3] m/s2
### 大寰电缸使用方法 ###

uint16 init #1 有效
uint16 mode # position velocity torque mode
uint16 stop #1:停止 0: 可运动
uint16 reset #1 有效
uint16 homing #1 发送回零

float64 pos #unit: M
float64 vel #unit: M/s
float64 force #unit: N
float64 cur #unit: A

```

- 代码示例:

```

C++
// Copyright 2025 UBTECH. All rights reserved.
//
// Demo: 发布大寰(DH)夹爪控制指令 (ROS2 版本)
//
// 话题信息:
//  命令话题: /ecat/left_grip/cmd (左夹爪)
//           /ecat/right_grip/cmd (右夹爪)
//  消息类型: ecat_task_msgs::msg::GripCmd
//
// GripCmd 字段说明:
//  header - 消息头 (shm_msgs::msg::Header)
//  init    - 初始化标志 (uint16)
//  mode    - 运行模式 (uint16): 0=位置/力模式, 10=力推模式(仅气缸)
//  stop    - 停止标志 (uint16)
//  reset   - 复位标志 (uint16)
//  homing  - 回零标志 (uint16)
//  pos     - 目标位置 (double): [0 ~ 0.05] m
//  vel     - 目标速度 (double): [0 ~ 0.1] m/s
//  force   - 目标力 (double): [0 ~ 100] N
//  cur     - 目标电流 (double): [0 ~ 0.4] A

#include <chrono>
#include <cmath>

```

```

#include <iomanip>
#include <iostream>
#include <memory>

#include <rclcpp/rclcpp.hpp>
#include <ecat_task_msgs/msg/grip_cmd.hpp>

using GripCmd = ecat_task_msgs::msg::GripCmd;

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    auto node = rclcpp::Node::make_shared("pub_gripper_command");

    constexpr size_t kQueueSize = 10;

    // 创建左右夹爪的命令 Publisher
    auto left_grip_pub =
        node->create_publisher<GripCmd>("/ecat/left_grip/cmd",
kQueueSize);
    auto right_grip_pub =
        node->create_publisher<GripCmd>("/ecat/right_grip/cmd",
kQueueSize);

    // 默认参数
    constexpr double kDefaultVelocity = 0.1;    // m/s
    constexpr double kDefaultForce = 50.0;      // N
    constexpr double kDefaultCurrent = 0.2;     // A

    rclcpp::Rate rate(10); // 10 Hz
    double time_cnt = 0.0;

    std::cout << "=== DH Gripper Command Publisher Demo (ROS2) ==="
<< std::endl;
    std::cout << "Publishing gripper commands to:" << std::endl;
    std::cout << "  Left: /ecat/left_grip/cmd" << std::endl;
    std::cout << "  Right: /ecat/right_grip/cmd" << std::endl;
    std::cout << "Press Ctrl+C to stop." << std::endl;

    while (rclcpp::ok()) {
        // 构建左夹爪命令：用正弦波在 [0, 0.05]m 范围内来回运动
        GripCmd left_cmd;
        auto now = node->get_clock()->now();
        left_cmd.header.stamp.sec = now.seconds();
        left_cmd.header.stamp.nanosec = now.nanoseconds() %

```

```

1000000000;
    left_cmd.pos = (std::sin(time_cnt) + 1.0) / 2.0 * 0.05; // 映射到 [0, 0.05]
    left_cmd.vel = kDefaultVelocity;
    left_cmd.force = kDefaultForce;
    left_cmd.cur = kDefaultCurrent;
    left_cmd.mode = 0; // 位置/力模式

    // 构建右夹爪命令：与左夹爪反相
    GripCmd right_cmd;
    right_cmd.header.stamp.sec = now.seconds();
    right_cmd.header.stamp.nanosec = now.nanoseconds() %
1000000000;
    right_cmd.pos = (std::sin(time_cnt + M_PI) + 1.0) / 2.0 *
0.05;
    right_cmd.vel = kDefaultVelocity;
    right_cmd.force = kDefaultForce;
    right_cmd.cur = kDefaultCurrent;
    right_cmd.mode = 0;

    // 发布命令
    left_grip_pub->publish(left_cmd);
    right_grip_pub->publish(right_cmd);

    std::cout << "\r[t=" << std::fixed << std::setprecision(2) <<
time_cnt << "s] "
        << "L: " << std::setprecision(4) << left_cmd.pos <<
" m "
        << "R: " << right_cmd.pos << " m " << std::flush;

    time_cnt += 0.1; // 与 10Hz 对应
    rate.sleep();
}

std::cout << std::endl;
rclcpp::shutdown();
return EXIT_SUCCESS;
}

```

- 注意：

1. HW_TYPE=cruze_s2_v1_gripper

5.11 逆运动学求解

5.11.1 逆运动学求解算法

- 提供了一个 function 以供开发者进行求解调用,该方法的动态库安装在 /opt/walker/manipulation 下,可以通过 source/ opt/walker/mapulation/setup.bash 进行调用
- CMake 示例:

```
C++
# CMakeLists.txt 引用包示例

cmake_minimum_required(VERSION 3.16)
project(sdk_test)

# 基于 C++17
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)

find_package(manipulation_common)
find_package(manipulation_outline_sdk)
find_package(manipulation_kinematics)

add_executable(robot_kinematics_sdk_test
src/robot_kinematics_sdk_test.cpp)

ament_target_dependencies(
  robot_kinematics_sdk_test
  PUBLIC rclcpp
)

target_include_directories(
  robot_kinematics_sdk_test
  PUBLIC $<BUILD_INTERFACE:${PROJECT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:include>
)

target_link_libraries(
  robot_kinematics_sdk_test
  PUBLIC manipulation_outline_sdk::robot_kinematics_sdk
        manipulation_common::math_transform
        manipulation_kinematics::kinematics_base
        manipulation_kinematics::kinematics_factory
```

```

        manipulation_kinematics::manipulation_kinematics
    )

install(TARGETS
    robot_kinematics_sdk_test
    DESTINATION lib/${PROJECT_NAME})
ament_package()

```

- 功能函数
 - CheckComponentIsSupported

Plain Text

```
bool CheckComponentIsSupported(manipulation::kinematics::Model
model);
```

输入:

C++

代码块

```

// `model`: 模型，目前仅支持 Walker S2 臂模型(kS2Arm)/Cruzr S2 臂
模型(kS2Arm)/Astron 臂模型(kAstronArm)
manipulation::kinematics::Model model

```

输出:

C++

```

// 如果支持则返回`true`，否则返回`false`
bool

```

- CalcEndPose

Plain Text

```

std::optional<manipulation::math::RigidTransformd>
CalcEndPose(const Eigen::VectorXd& q, const Eigen::VectorXd&
zero_positions,
            manipulation::kinematics::Location location,
            manipulation::kinematics::Model model);

```

输入:

C++

代码块

```

// `q`: 需要计算的关节角，可以从全身关节状态接口

```

```
(/mc/sdk/robot_state)中读取当前关节角，
// 也可以自己输入，注意只需要单臂关节角（7 维），且注意顺序
(shoulder_pitch_joint->
// shoulder_roll_joint->shoulder_yaw_joint->elbow_roll_joint->
elbow_yaw_joint->
// wrist_pitch_joint->wrist_roll_joint)
const Eigen::VectorXd& q
// `zero_positions`: 零位偏置修正，若无标定参数修正，可以设置为 0，
注意只需要设置单臂（7 维）
const Eigen::VectorXd& zero_positions
// `location`: 位置，区分左右臂，即 location =
manipulation::kinematics::Location::kLeft,
// or location = manipulation::kinematics::Location::kRight
manipulation::kinematics::Location location
// `model`: 模型，目前仅支持 Walker S2 臂模型(kS2Arm)/Cruzr S2 臂
模型(kS2Arm)/Astron 臂模型(kAstronArm)
manipulation::kinematics::Model model
```

输出：

```
C++
// 如果成功计算出来则返回基于 Torso 的笛卡尔空间位姿，否则返回
`nullopt`
std::optional<manipulation::math::RigidTransformd>
```

◦ CalcJointAnglesForEndPose

Plain Text

```
std::optional<Eigen::VectorXd>
CalcJointAnglesForEndPose(const Eigen::VectorXd& initial_q,
                           const
manipulation::math::RigidTransformd& target_X,
                           const Eigen::VectorXd&
zero_positions,
                           manipulation::kinematics::Location
location,
                           manipulation::kinematics::Model
model);
```

输入：

```
C++
代码块
```

```

// `initial_q`: 初始关节角，作为参考关节角，可以从全身关节状态接口
(/mc/sdk/robot_state)
// 中读取当前关节角，也可以自己输入，注意只需要单臂关节角（7 维）
const Eigen::VectorXd& initial_q
// `target_X`: 基于 Torso 的目标笛卡尔空间位姿，可以通过上述
CalcEndPose 计算获取
const manipulation::math::RigidTransformd& target_X
// `zero_positions`: 零位偏置修正，若无标定参数修正，可以设置为 0，
注意只需要设置单臂（7 维）
const Eigen::VectorXd& zero_positions
// `location`: 位置，区分左右臂，即 location =
manipulation::kinematics::Location::kLeft,
// or location = manipulation::kinematics::Location::kRight
manipulation::kinematics::Location location
// `model`: 模型，目前仅支持 Walker S2 臂模型(kS2Arm)/Cruzr S2 臂
模型(kS2Arm)/Astron 臂模型(kAstronArm)
manipulation::kinematics::Model model

```

输出:

```

C++
// 如果成功计算出来则返回关节角，否则返回`nullopt`
std::optional<Eigen::VectorXd>

```

- CalcFloatingBasedKinematics

Plain Text

```

std::optional<Eigen::VectorXd>
CalcFloatingBasedKinematics(const Eigen::VectorXd& initial_q,
                             const
manipulation::math::RigidTransformd& X_base_world,
                             const
manipulation::math::RigidTransformd& X_lfoot_world,
                             const
manipulation::math::RigidTransformd& X_rfoot_world);

```

输入:

```

C++
// `initial_q`: 初始关节角，作为参考关节角，可以从全身关节状态接口
(/mc/sdk/robot_state)
// 中读取当前关节角，也可以自己输入，注意双腿关节角（12 维），左腿（6
维）+右腿（6 维）

```



```

const Eigen::VectorXd& initial_q
// `X_base_world`: 世界坐标系下浮动基的位姿
const manipulation::math::RigidTransformd& X_base_world
// `X_lfoot_world`: 世界坐标系下左脚踝处的位姿
const manipulation::math::RigidTransformd& X_lfoot_world
// `X_rfoot_world`: 世界坐标系下右脚踝处的位姿
const manipulation::math::RigidTransformd& X_rfoot_world

```

输出:

```

C++
// 如果成功计算出来则返回双腿关节角, 否则返回`nullopt`
std::optional<Eigen::VectorXd>

```

- 代码示例

```

C++
// Copyright 2026 UBTECH. All rights reserved.
#include "robot_kinematics_sdk.h"

#include <string>

#include "kinematics_base.h"
#include "math_transform/rigid_transform.h"

int main(int argc, char **argv) {
    /***** CheckComponentIsSupported *****/
    // 返回成功, 支持手臂模型的运动学
    bool res_1 = ubt::robot::kinematics::CheckComponentIsSupported(
        manipulation::kinematics::Model::kS2Arm);
    std::string res_string_1 = res_1 ? "Yes" : "No";
    std::cout << "Support the current model: " << res_string_1 <<
    std::endl;

    // 返回失败, 目前暂不支持手臂模型之外的运动学
    bool res_2 = ubt::robot::kinematics::CheckComponentIsSupported(
        manipulation::kinematics::Model::kS2Head);
    std::string res_string_2 = res_2 ? "Yes" : "No";
    std::cout << "Support the current model: " << res_2 <<
    std::endl;

    /***** CalcEndPose *****/
}

```

```

// 左臂关节角，在关节空间采样的数据，也可以通过从全身关节状态接口
(/mc/sdk/robot_state)中读取当前关节角
Eigen::VectorXd q(7);
q << 0.331215, -0.673191, -1.00882, -0.710871, -0.96685, -
0.317587, 0.5565871;

// 零位偏置修正，若无标定参数修正，可以设置为0
Eigen::VectorXd zero_positions(7);
zero_positions << 0, 0, 0, 0, 0, 0, 0;
std::optional<manipulation::math::RigidTransformd> end_pose_res
=
    ubt::robot::kinematics::CalcEndPose(
        q, zero_positions,
manipulation::kinematics::Location::kLeft,
        manipulation::kinematics::Model::kS2Arm);

if (end_pose_res.has_value()) {
    std::cout << "end_pose_res: " << end_pose_res.value() <<
std::endl;
} else {
    std::cout << "Failed to calc end pose" << std::endl;
    return EXIT_FAILURE;
}

/***** CalcJointAnglesForEndPose
*****/
Eigen::VectorXd initial_q(7);
initial_q << 0.3, -0.6, -1.0, -0.7, -0.9, -0.3, 0.5;
std::optional<Eigen::VectorXd> joint_angles_res =
    ubt::robot::kinematics::CalcJointAnglesForEndPose(
        initial_q, end_pose_res.value(), zero_positions,
        manipulation::kinematics::Location::kLeft,
        manipulation::kinematics::Model::kS2Arm);

if (joint_angles_res.has_value()) {
    std::cout << "joint_angles_res: " <<
joint_angles_res.value().transpose()
        << std::endl;
} else {
    std::cout << "Failed to calc joint angles" << std::endl;
    return EXIT_FAILURE;
}

/***** CalcFloatingBasedKinematics

```

```

*****/
Eigen::VectorXd initial_q_legs(12);
initial_q_legs << -0.01685, -0.00993851, 0.296611, -0.544217,
0.263337,
0.0157838, 0.00978879, -0.00595708, 0.306388, -0.544662,
0.253763,
-0.0101094;
manipulation::math::RigidTransformd X_base_world =
manipulation::math::RigidTransformd{
manipulation::math::RollPitchYawd{Eigen::Vector3d{0, 0,
0}},
Eigen::Vector3d{-1.68371e-09, 0, 0.87}};
manipulation::math::RigidTransformd X_lfoot_world =
manipulation::math::RigidTransformd{
manipulation::math::RollPitchYawd{
Eigen::Vector3d{-0.00914796, -0.00325864,
0.00632319}},
Eigen::Vector3d{-0.00676362, 0.0905028, 2.17913e-05}};
manipulation::math::RigidTransformd X_rfoot_world =
manipulation::math::RigidTransformd{
manipulation::math::RollPitchYawd{
Eigen::Vector3d{0.00298992, 0.000356849,
0.001997550}},
Eigen::Vector3d{-0.000797962, -0.0968128, -1.94652e-
05}};

std::optional<Eigen::VectorXd> leg_joint_angles_res =
ubt::robot::kinematics::CalcFloatingBasedKinematics(
initial_q_legs, X_base_world, X_lfoot_world,
X_rfoot_world);

if (leg_joint_angles_res.has_value()) {
std::cout << "leg_joint_angles_res: "
<< leg_joint_angles_res.value().transpose() <<
std::endl;
} else {
std::cout << "Failed to calc leg joint angles" << std::endl;
return EXIT_FAILURE;
}

return EXIT_SUCCESS;
}

```

结果输出:

```
Support the current model: Yes
ERROR: No support for this model
Support the current model: 0
end_pose_res: xyzrpy = 0.3197; 0.5058; -0.3453; -1.5556; 1.5206; 3.1407
joint_angles_res: 0.314975 -0.666657 -1.05444 -0.722687 -0.935898 -0.323766 0.542592
closedChain initialized successfully from:
closedChain initialized successfully from:
leg_joint_angles_res: -0.0170208 -0.00626681 0.266157 -0.544331 0.28154 0.00787256 0.00976218 -0.00199397 0.275265 -0.545097 0.269455 -0.00677224
```

5.12 ASR

5.12.1 ASR 语音转文字接口

- 说明: ASR 语音转文字 (需要唤醒后才会开启 Asr 识别, 唤醒词“Walker, Walker”)
- 控制方式: topic
- 话题名称: /sys/speech/asr
- 数据定义位置: sys_task_msgs::msg::Asr
- 数据格式:

Plain Text

string text	#文本内容
string language	#语音类型
string emotion	#情感, 暂时未用到
string status	#状态, 暂时未使用
int32 wordnum	#几个汉字或者几个单词

Asr 通过服务: /sys/asr/enable 来控制识别开关, 数据格式为 std_srvs/srv/SetBool

- 代码示例:

```
C++
#include <rclcpp/rclcpp.hpp>
#include <std_srvs/srv/set_bool.hpp>
#include <sys_task_msgs/msg/asr.hpp>

class AsrDemo : public rclcpp::Node
{
public:
    AsrDemo() : Node("asr_demo_node")
    {
        // ASR topic subscriber
        asr_sub_ = this-
```

```

>create_subscription<sys_task_msgs::msg::Asr>(
    "/sys/speech/asr",
    10,
    std::bind(&AsrDemo::asr_callback, this,
std::placeholders::_1));

    // Service client to control ASR enable/disable
    enable_client_ = this-
>create_client<std_srvs::srv::SetBool>("/sys/asr/enable");

    // 等待服务
    if (!enable_client_-
>wait_for_service(std::chrono::seconds(5)))
    {
        RCLCPP_ERROR(this->get_logger(), "ASR enable service
not available.");
        return;
    }

    // 同步调用
    enable_asr(true);

    RCLCPP_INFO(this->get_logger(), "ASR Demo started. Waiting
for ASR messages...");
}

~AsrDemo()
{
    enable_asr(false);
}

private:
    rclcpp::Subscription<sys_task_msgs::msg::Asr>::SharedPtr
asr_sub_;
    rclcpp::Client<std_srvs::srv::SetBool>::SharedPtr
enable_client_;

    void enable_asr(bool enable)
    {
        auto request =
std::make_shared<std_srvs::srv::SetBool::Request>();
        request->data = enable;

        // 直接同步调用

```

```

        auto result_future = enable_client_
>async_send_request(request);

        // 使用 NodeBaseInterface 同步等待
        auto ret_code = rclcpp::spin_until_future_complete(this-
>get_node_base_interface(), result_future);

        if (ret_code == rclcpp::FutureReturnCode::SUCCESS)
        {
            auto response = result_future.get();
            if (response->success)
            {
                RCLCPP_INFO(this->get_logger(),
                    enable ? "ASR Enabled." : "ASR
Disabled.");
            }
            else
            {
                RCLCPP_WARN(this->get_logger(), "Request
failed: %s", response->message.c_str());
            }
        }
        else
        {
            RCLCPP_ERROR(this->get_logger(), "Failed to call
/sys/asr/enable service.");
        }
    }

    void asr_callback(const sys_task_msgs::msg::Asr::SharedPtr
msg)
    {
        RCLCPP_INFO(this->get_logger(),
            "[ASR] text=\"%s\" language=%s wordnum=%d",
            msg->text.c_str(),
            msg->language.c_str(),
            msg->wordnum);
    }
};

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<AsrDemo>();

```

```

rclcpp::spin(node);
rclcpp::shutdown();
return 0;
}

```

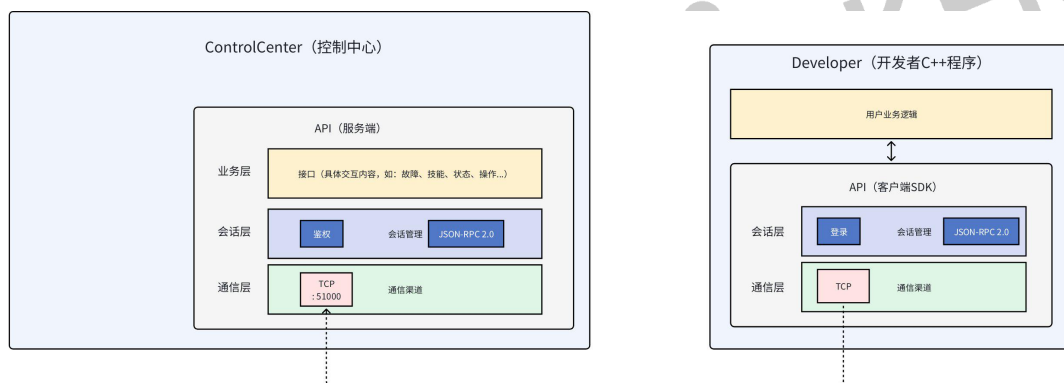
6. 接口说明-应用服务【API】

6.1 使用说明

6.1.1 原理

6.1.1.1 协议栈

如上图所示：



如上图所示，在机器人 Orin 系统中，存在一个名为 ControlCenter 程序（简称：CC）。它是机器人的主控程序，它对外提供统一的 API 操作接口。该 API 接口以 TCP 端口 51000 对外提供 TCP 连接服务。客户程序使用 API SDK，即：ubt_robot 库，与 ControlCenter 建立长连接，实现双向的请求与通知。

客户端 C++ 程序不限定其运行实体。可以是运行在 Orin 系统容器中，也可以运行在客户自己的计算机系统中，如工程机上。只要能通过网络与机器人中 ControlCenter API 提供的 TCP 51000 端口建立连接并进行正常的 TCP 通信即可。

6.1.1.2 关于 api_id 与 token

为了方便机器人对接入的 API 进行合法性校验，每个客户端程序在使用 ubt_robot 库之前都需要指定 api_id 与 token。其中 api_id 用于表明客户端程序的身份，同一时刻机器人端仅允许持有该身份的一个客户程序接入。token 则用于鉴定 api_id 的合法性，防止伪造。

- `api_id`: 用于表明客户端程序的身份。可以填写任意字符串。
- `token`: 用于鉴定合法性, 防止伪造。目前可以不提供 (即留空字符串)。

在 C++ 的 Demo 代码中, 这一配置体现为 `Api::Config` 结构体的赋值:

```
C++
int main()
{
    // ... 略 ...

    ubt::robot::Api::Config cfg;
    cfg.address = "0.0.0.0:51000"; // 指定机器人 API 的服务地址
    cfg.api_id = "demo";           // 填入 APIId (可任意填写)
    cfg.token = "";                // 填入 Token (目前无需提供, 留空即可)

    ubt_api.initialize(cfg);       // 初始化
    // ... 略 ...
}
```

Python 用户注意:

如果你使用 Python 进行开发 (文档 6.1.2 提及 `ubt_api_tiny_python`), 在实例化 `ubt_robot` 库时, 通常也会有类似的参数位置用于传入 `api_id` 和 `token`。根据文档说明, 传入任意字符串作为 `api_id`, 并将 `token` 留空或 `None` 即可。

6.1.1.3 客户端连接互斥性与认证说明

为了帮助开发者更好地理解 `api_id` 的工作机制, 特此补充以下关键说明:

相同 `api_id` 的连接冲突 (互斥性)

- **行为机制:** 机器人端的 ControlCenter (CC) 服务对 `api_id` 采用“独占连接”策略。同一时间, 系统仅允许一个持有特定 `api_id` 的客户端程序接入。
- **冲突结果:** 如果客户端 A 已使用 `api_id="demo"` 成功连接, 此时客户端 B 再次使用相同的 `api_id="demo"` 发起连接请求, 系统将强制断开客户端 A 的连接, 并接受客户端 B 的连接。这通常被称为“踢下线”机制。

6.1.2 API 库使用指引

基于开发者容器的精简版的 SDK 文件 zip:

C++:

[ubt_api_tiny_colcon.0624.tar.xz]

Python:

[ubt_api_tiny_python.0624.tar.xz]

首先使用以下命令将上述 zip 文件传到开发者容器中：

Shell

```
scp -P 2222 所在路径/ubt_api_tiny.202XXXXX.zip  
ubt@192.168.11.3:/debug/  
# 输入密码: Ubtubt@9880
```

然后通过 ssh 进入开发者容器的终端，在 /debug/ 目录下，即可看到上述 zip 文件。
关于进入终端方法，请参考本文档 3.3.1 部分。

其中：

- include 目录，为编译所需头文件；
- lib 目录，为编译所需库文件：
 - aarch64 目录，aarch64 编译所需的库文件；
 - x86_64 目录，x86 编译所需的库文件（暂不开放）；
- examples 目录，为示例程序。

对于 Python 语言用户请安装 ubt_sdk_python 支持包：

```
pip install ubt_robot-1.0.0-cp310-cp310-linux_aarch64.whl
```

里面同样提供了与 C++ api 类似的 example 可供参考,可通过:

```
python3 01_ubt_api_demo.py 运行对应的 demo 程序
```

6.1.2.1 demo 示例

examples/01_ubt_api_demo.cpp 中实现了使用 API 连接机器人的 API 服务，并通过 API 的 requestSync() 方法，向 control_center 发送了 "cc.api.fault.current.get" 的请求，在得到了 control_center 的回复之后，将结果打印出来。

代码内容：

C++

代码块

```
#include <iostream>  
#include <cc_api_client/api.h>
```

```

using namespace std;

int main()
{
    cout << "== UbtApi Demo ==" << endl;
    ubt::robot::Api ubt_api;

    ubt::robot::Api::Config cfg;
    cfg.address = "0.0.0.0:51000";  //!< 指定机器人 API 的服务地址
    cfg.api_id = "demo";          //!< 填入 APIId
    cfg.token = "";               //!< 填入 Token

    ubt_api.initialize(cfg);  //!< 初始化
    ubt_api.start();          //!< 启动 api，内部开始连接机器人服务
    cout << "API start, wait until connect" << endl;
    ubt_api.waitUntilConnected();  //!< 等待连接完成

    cout << "Send request ..." << endl;
    //!< 向机器人服务发送 cc.api.fault.current.get 请求，并等待其回应
    auto rsp = ubt_api.requestSync("cc.api.fault.current.get");
    if (rsp.error.code == 0) {  //!< 如果没有异常，则打印返回内容
        cout << "Success, result:" << rsp.js_result.dump() <<
std::endl;
    } else {  //!< 如果错误，则打印错误码与错误信息
        cout << "Error, code:" << rsp.error.code << ", message:" <<
rsp.error.message << endl;
    }

    ubt_api.stop();  //!< 停止 API 连接服务

    cout << "== End ==" << endl;
    return 0;
}

```

使用对于 sdk 示例代码,可以使用 colcon 构建, 命令为:

Shell

代码块

colcon build #构建

ros2 run ubt_api_tiny 08_get_skill_list_demo #运行

运行效果:

```

ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.aarch64.20260129/ubt_api_tiny$ ./demo
== UbtApi Demo ==
Api start, wait until connect
Send request ...
Success, result:{"fault":[{"code":"07009001","startTime":"1769682408353"}, {"code":"07009002","startTime":"1769682408354"}, {"code":"07019001","startTime":"1769682408354"}, {"code":"07019002","startTime":"1769682408354"}, {"code":"07029001","startTime":"1769682408355"}, {"code":"07039001","startTime":"1769682408355"}, {"code":"07049001","startTime":"1769682408355"}, {"code":"07059001","startTime":"1769682408355"}, {"code":"07209001","startTime":"1769682408355"}]}
== End ==

```

可以看到，程序已获取到故障列表的数据，后续示例也以上面的命令为标准，修改对应文件名即可。

6.1.2.2 连接成功与断开事件监听

如果客户端程序需要在与机器人连接成功与连接断开时执行某些业务相关的操作，可以有通过设置回调的方式实现。

接口定义：

```

C++
代码块
class Api {
    ...
public:
    // 连接和断开(对端断开)回调
    using ConnectCallback = std::function<void()>;
    using DisconnectCallback = std::function<void()>;

    //! 设置连接建立的回调
    void setConnectCallback(ConnectCallback &&cb);
    //! 设置连接断开的回调
    void setDisconnectCallback(DisconnectCallback &&cb);
    ...
};

```

使用示例：

```

C++
代码块
// ...略...

```

```

int main()
{
    //...略...
    ubt_api.initialize(cfg);

    // 连接成功后触发的回调
    ubt_api.setConnectCallback([]{
        std::cout << "connect successful" << std::endl;
    });

    // 对端主动断开连接时触发的回调
    ubt_api.setDisconnectCallback([]{
        std::cout << "peer disconnect" << std::endl;
    });

    ubt_api.start();
    //...略...
}

```

注意：

- setConnectCallback() 应该 start() 之前完成设置。

执行效果：

demo 文件：**examples/02_connect_cb_demo.cpp**

```

• ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.20260203/ubt_api_tiny$ ./demo
== UbtApi Demo ==
connect successful
peer disconnect
== End ==

```

6.1.2.3 异步发送请求

上面 demo 演示了使用 API 的 requestSync() 以同步的方式调用请求，该方式会阻塞性地等待机器人回应。如果想要更为高效的请求，可以采用异常请求的方法 request(method, cb)。

接口定义：

```

C++
代码块
class Api {
    ...
public:
    //! JSONRPC 回复内容

```

```

struct Response {
    Json js_result; /// 结果
    /// 错误相关
    struct {
        int code = 0;          /// 错误码
        std::string message;    /// 错误描述
    } error;
};
/// 请求响应回调
using RequestCallback = std::function<void(const Response &)>;

/// 异步发送 JSONRPC 带参数的请求，并指定收到回复后的回调
void request(const std::string &method, const Json &js_params,
RequestCallback &&cb);
/// 异步发送 JSONRPC，并指定收到回复后的回调
void request(const std::string &method, RequestCallback &&cb);
...
};

```

使用示例:

```

C++
代码块
// ...略...

int main()
{
    // ...略...
    ubt_api.start();
    ubt_api.waitUntilConnected();
    // 异步请求示例(无参数)
    ubt_api.request("cc.api.fault.subscribe",
        [] (const ubt::robot::api::Response &rsp){
            if (rsp.error.code != 0) {
                std::cout << "Subscribe fault error: code = " <<
rsp.error.code << ", message = " << rsp.error.message <<
std::endl;
            } else {
                std::cout << "Subscribe fault success: result = " <<
rsp.js_result.dump() << std::endl;
            }
            p.set_value();
        });
}

```

```

// 异步请求示例(带参数)
ubt::robot::Api::Json js;
js["startIndex"] = 12;
js["endIndex"] = 13;

ubt_api.request("cc.api.fault.history.get_range", js,
    [](const ubt::robot::Api::Response &rsp){
        if (rsp.error.code != 0) {
            std::cout << "Get current fault error: code = " <<
rsp.error.code << ", message = " << rsp.error.message <<
std::endl;
        } else {
            std::cout << "Get current fault success: result = " <<
rsp.js_result.dump() << std::endl;
        }
    });

// 此处 sleep 一段时间模拟业务，等待异步请求完成，防止提前 stop 退出程
序
// ...略...
ubt_api.stop();
return 0;
}

```

注意：

- 请求需在 waitUntilConnected() 之后发起，确保连接完成后再发起请求。

运行效果：

demo 文件：**examples/03_async_request_demo.cpp**

```

ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.20260203/ubt_api_tiny$ ./demo
== UbtApi Demo ==
Subscribe fault success: result = {"result":"accept"}
Get current fault success: result = {"history":[{"code":"07019001","endTime":"1768471531560","fix":3,"index":12,"startTime":"1768446731042"}, {"code":"07019002","endTime":"1768471531562","fix":3,"index":13,"startTime":"1768446731042"}]}
== End ==

```

6.1.2.4 异步发送通知

接口定义：

```

C++
代码块
class Api {

```

```

...
public:
    //! 异步发送 JSONRPC 带参数的通知
    void notify(const std::string &method, const Json &js_params);
    //! 异步发送 JSONRPC 通知, 无参数
    void notify(const std::string &method);
    ...
};

```

使用示例:

```

C++
代码块
// ...略...

int main()
{
    // ...略...
    ubt_api.start();
    ubt_api.waitUntilConnected();

    // 注册接收故障变更的消息服务
    ubt_api.addService("cc.api.fault.current.update",
        [](int int_id, const ubt::robot::Api::Json& js_params) {
            std::cout << "id = " << int_id << ", result = " <<
js_params.dump() << std::endl;
        }
    );

    // 订阅故障变更消息
    ubt_api.notify("cc.api.fault.subscribe");

    // 此处 sleep 一段时间模拟业务, 等待异步请求完成, 防止提前 stop 退出程
序
    // ...略...
    ubt_api.stop();
    return 0;
}

```

运行效果:

demo 文件: **examples/04_notify_demo.cpp**

当故障状态变化时, 即可接收到对应消息:

```

ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.20260203/ubt_api_tiny$ ./demo
== UbtApi Demo ==
id = 0, result = {"fault":[{"code":"00000011","startTime":"1770103379986"}, {"code":"07009001","startTime":"1770102906878"}, {"code":"07009002","startTime":"1770102906878"}, {"code":"07019001","startTime":"1770102906879"}, {"code":"07019002","startTime":"1770102906879"}, {"code":"07029001","startTime":"1770102906879"}, {"code":"07039001","startTime":"1770102906879"}, {"code":"07049001","startTime":"1770102906879"}, {"code":"07059001","startTime":"1770102906880"}, {"code":"07209001","startTime":"1770102906880"}]}
== End ==

```

6.1.2.5 添加服务监听

上面提及，机器人与客户程序之间不像 HTTP 这种单向请求，而是可以进行双向的。机器人有事件也可以主动通知到客户程序。客户程序为了能在机器人主动发起通知或请求时能执行对应的操作，而需要使用 `addService()` 进行注册。

接口定义：

```

C++
代码块
class Api {
    ...
public:
    // 服务响应回调
    using ServiceCallback = std::function<void(int req_id, const
    Json &js_params)>;

    //! 添加 JSONRPC 服务，并指定回调
    void addService(const std::string &method, ServiceCallback
    &&cb);
    //! 删除 JSONRPC 服务
    void removeService(const std::string &method);
    ...
};

```

使用示例：

```

C++
代码块
// ...略...

int main()
{
    // ...略...
    ubt_api.start();
    ubt_api.waitUntilConnected();

    // 注册接收故障变更的消息服务
    ubt_api.addService("cc.api.fault.current.update",

```



```

    [(int int_id, const ubt::robot::Api::Json& js_params) {
        std::cout << "id = " << int_id << ", result = " <<
js_params.dump() << std::endl;
    }
);

// 订阅故障变更消息
ubt_api.notify("cc.api.fault.subscribe");

// 此处 sleep 一段时间模拟业务，等待异步请求完成，防止提前 stop 退出程
序
// ...略...
ubt_api.stop();
return 0;
}

```

注意：

- 要在发起请求前注册好相关的服务，否则可能注册前的请求的结果会丢失。

运行效果：

demo 文件：**examples/05_addservice_demo.cpp**

```

ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.20260203/ubt_api_tiny$ ./demo
== UbtApi Demo ==
id = 0, result = {"fault":[{"code":"00000011","startTime":"1770103379986"}, {"code":"07009001","startTime":"1770102906878"}, {"code":"07009002","startTime":"1770102906878"}, {"code":"07019001","startTime":"1770102906879"}, {"code":"07019002","startTime":"1770102906879"}, {"code":"07029001","startTime":"1770102906879"}, {"code":"07039001","startTime":"1770102906879"}, {"code":"07049001","startTime":"1770102906879"}, {"code":"07059001","startTime":"1770102906880"}, {"code":"07209001","startTime":"1770102906880"}]}
== End ==

```

6.1.2.6 注意事项

- 必须先调用 initialize() 之后再调用 start() ；
- 必须等待连接成功后(waitUntilConnected() 执行完毕)之后再开始发起 request；
- 必须在发起请求前注册好相关的服务，否则会丢失注册前发起的请求结果；
- 确保执行的异步操作全部执行完毕且收到执行结果后再 stop 。

6.2 业务接口

6.2.1 执行技能

6.2.1.1 启动技能任务

技能、任务定义：

技能：一种流程的模板，定义了该流程执行的规则，以及需要传入的参数。

任务：对于某个技能的实例，是对技能的具象化。

比如：

- 技能：将指定的物品，从指定点搬到另一个指定点；
- 任务：将 1#纸箱子从，从 A 点位，搬到 B 点位；

接口定义

技能启动器类：skill.h

```
C++
代码块
//! 技能类，用于启动技能
class Skill {
public:
    using SkillCode = std::string;
    using Json = Work::Json;

    Skill(Api &api);

    //! 启动技能，返回任务实例指针
    std::unique_ptr<Work> launch(const SkillCode &skill_code, const
    Json &js_params);

private:
    Api &api_;
};
```

任务类：work.h

```
C++
代码块
//! 任务类，表示一个技能执行的实例
class Work {
public:
    friend class WorkDispatch;
    friend class Skill;

public:
    using Id = std::string; //!< 任务 id
    using Json = nlohmann::json;

    enum class State {
```

```

    kNone,      //!< 未启动
    kUnderway,  //!< 进行中
    kStoped,    //!< 已停止
    kPaused,    //!< 已暂停
    kFinished   //!< 已结束
};

struct Result {
    enum class Type {
        kSuccess,  //!< 成功
        kFail,     //!< 失败
        kStoped    //!< 被停止
    };
    Type type;  //!< 执行结果类型
    Json js_succ_result;
    Json js_fail_reason;
};

using FinishCallback = std::function<void(const Result &)>;

public:
    ~Work();

public:
    //!< 获取任务 ID
    Id getId() const;
    //!< 获取任务状态
    State getState() const;
    //!< 获取任务结果
    Result getResult() const;
    //!< 设置任务状态（仅供内部使用）
    void setState(State state);
    //!< 设置任务结束回调
    void setFinishCallback(FinishCallback &&cb);

    //!< 停止任务
    bool stop();

    //!< 等待任务结束，或超时返回（阻塞性函数）
    Result waitUntilFinish(std::chrono::milliseconds timeout =
std::chrono::milliseconds(0));

private:

```

```

    explicit Work(Api &api, const Id &id);

private:
    class Impl;
    Impl *impl_;
};

```

使用示例

```

C++
代码块
ubt::robot::Api ubt_api;
// 进行 API 的初始化
// ...
ubt::robot::Skill skill(ubt_api);
ubt::robot::Skill::Json params;
params["pitch"] = 0;
params["yaw"] = 0;

auto work = skill.launch("A000016", params);

if (work) {
    auto result = work->waitUntilFinish();
    if (result.type == ubt::robot::Work::Result::Type::kSuccess) {
        std::cout << "Skill finished successfully, result: " <<
result.js_succ_result.dump() << std::endl;
    } else if (result.type == ubt::robot::Work::Result::Type::kFail)
    {
        std::cout << "Skill failed, reason: " <<
result.js_fail_reason.dump() << std::endl;
    } else if (result.type ==
ubt::robot::Work::Result::Type::kStoped) {
        std::cout << "Skill was stopped." << std::endl;
    }
}
}

```

- 注意：launch 返回的 work 对象在对端拒绝该请求时会为 nullptr。
- 使用 Skill 来执行任务时，无需再使用 addservice("cc.api.work.finished", cb)去获取任务执行结果。

运行效果

demo 文件：**examples/06_skill_work_demo.cpp**

```
ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.20260203/ubt_api_tiny$ ./demo
=== Skill Work API Example ===
Skill failed, reason: "TimeOut"
=== Skill Work API Example End ===
```

6.2.1.2 停止任务

使用示例

```
C++
auto work = skill.launch("A000016", params);
if (work) {
    work->stop();
    auto result = work->waitUntilFinish();
    if (result.type == ubt::robot::Work::Result::Type::kSuccess) {
        std::cout << "Skill finished successfully, result: " <<
result.js_succ_result.dump() << std::endl;
    } else if (result.type == ubt::robot::Work::Result::Type::kFail)
    {
        std::cout << "Skill failed, reason: " <<
result.js_fail_reason.dump() << std::endl;
    } else if (result.type ==
ubt::robot::Work::Result::Type::kStoped) {
        std::cout << "Skill was stopped." << std::endl;
    }
}
```

注意：停止任务需确保任务已经正常启动，TTS 比较特殊，具体查看 TTS 技能部分。

运行效果

demo 文件：**examples/07_work_stop_demo.cpp**

```
ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.20260203/ubt_api_tiny$ ./demo
=== Skill Work API Example ===
Skill was stopped.
=== Skill Work API Example End ===
```

6.2.1.3 获取技能列表

注：该方法需 control center 版本在 0.5.73 以上支持

发起方	客户端
类型	request
方法名	cc.api.skill.list

请求参数	无
返回数据	<pre> JSON { "skills": { "A000001": { "name": "TTS", "description": "语音播报文本", "category": "sound", "version": "0.0.1" }, "A000002": { "name": "NavTo", "description": "导航到指定的位姿或地标", "category": "movement", "version": "0.0.1" }, ...略... } } </pre>

使用示例

C++

代码块

```

auto rsp = ubt_api.requestSync("cc.api.skill.list");
if (rsp.error.code == 0) {  //!< 如果没有异常，则打印返回内容
    cout << "Success, result:" << rsp.js_result.dump() << std::endl;
} else {  //!< 如果错误，则打印错误码与错误信息
    cout << "Error, code:" << rsp.error.code << ", message:" <<
    rsp.error.message << endl;
}

```

运行效果

demo 文件: **examples/08_get_skill_list_demo.cpp**

```
ubt@vision:/debug/export_sdk/ubt_api_tiny$ ./demo-1
== UbtApi Demo ==
Api start, wait until connect
Send request ...
Api connected
Success, result: [{"category": "sound", "description": "Speak the content of the specified text or text from file", "name": "TTS", "version": "0.0.1"}, {"category": "nav", "description": "导航到指定的标记点或指定的位姿", "name": "NavTo", "version": "0.0.1"}, {"category": "nav", "description": "行走相对位姿(无避障)", "name": "FreeWalk", "version": "0.0.1"}, {"category": "motion", "description": "设置腿部模式", "name": "SetLegMode", "version": "0.0.1"}, {"category": "motion", "description": "设置摆臂开关", "name": "SetArmSwingEnable", "version": "0.0.1"}, {"category": "map", "description": "获取地图列表", "name": "GetMapList", "version": "0.0.1"}, {"category": "map", "description": "设置地图", "name": "SetMap", "version": "0.0.1"}, {"category": "nav", "description": "重定位", "name": "Relocation", "version": "0.0.1"}, {"category": "motion", "description": "将箱子从桌上搬起来", "name": "GrabBox", "version": "0.0.1"}, {"category": "motion", "description": "将箱子放到桌面上", "name": "PutBoxSimple", "version": "0.0.1"}, {"category": "motion", "description": "控制头按指定的速度转到指定的方向与俯仰角", "name": "HeadMotion", "version": "0.0.1"}, {"category": "nav", "description": "执行特定的肢体动作", "name": "ExecutePresetLimbAction", "version": "0.0.1"}, {"category": "logistics", "description": "将箱子从起点A搬运到终点B, 包含导航至A点、抓取箱子、导航至B点、放置箱子四个阶段", "name": "CarryBoxFromAtoB", "version": "1.0.0"}, {"category": "debug", "description": "echo params", "name": "Echo", "version": "0.0.1"}, {"category": "motion", "description": "动头的测试示例", "name": "move_head_demo", "version": ""}, {"category": "nav", "description": "导航技能测试", "name": "nav_demo", "version": ""}, {"category": "nav", "description": "导航技能测试", "name": "nav_free_walk_demo", "version": ""}, {"category": "nav", "description": "自由行走演示", "name": "nav_to any pose", "version": ""}, {"category": "nav", "description": "导航到某个位姿测试示例", "name": "set_leg_mode_demo", "version": ""}, {"category": "tts", "description": "语音播报演示", "name": "tts_demo", "version": ""}]]
== End ==
ubt@vision:/debug/export_sdk/ubt_api_tiny$ vi demo-1.cpp
```

6.2.1.4 获取技能 profile

注：该方法需 control center 版本在 0.5.73 以上支持

发起方	客户端
类型	request
方法名	cc.api.skill.profile
请求参数	JSON { "skillCode" : "A000026" }
返回数据	技能描述内容，例如： JSON { "skillCode": "A000026", "name": "ExecutePresetLimbAction", "engine": "atomic", "category": "nav", "description": "执行特定的肢体动作", "version": "0.0.1", "builtin": true, "domain" : ["TYX"], "inputSchema": { ... } }

Profile 内容说明

- skillCode, 技能码, 相对于机器个体必须是唯一的, 用于标识唯一的技能;

- **name**, 英文名称, 用于在编辑工具上、运行记录中展示, 方便理解;
- **nameCn**, 中文名称, 用于在编辑工具上、运行记录中展示, 方便理解;
- **engine**, 执行器类型, 类似于脚本程序里的首行描述, 用于描述该脚本是用什么执行器来执行。

目前执行器有:

- **atomic**, 是原子能力, 由 CC 直接执行;
- **behavior_tree**, 是行为树的组合能力, 由行为树执行;
- **category**, 技能分类。目前的分类有:
 - **scence**, 工作场景;
 - **sound**, 发声类;
 - **motion**, 动作类;
 - **sense**, 感知类;
 - **nav**, 导航;
 - **map**, 地图管理;
- **description**, 技能描述。详细说明该功能的功能, 主要用于用户想详细了解的时候使用;
- **domain**, 应用领域, 搬运、分拣; 技能适用工种和编码:
 - 搬运工种 (编码 BYX)
 - 分拣工种 (编码 FJX)
 - 通用工种 (编码 TYX)
 - 搬运&分拣工种 (BFX)
 - 讲解员 (JJX)
 - 导购员 (编码 DGX)
 - 其他工种 (编码 QTX)
- **file**, 可选, 特指组合能力的实现的主文件, 类似于 python 的 main.py。对于原子能力, 不需要描述;
- **author**, 可选, 作者信息 (仅非内建技能有效)
- **inputSchema**, 可选, 输入参数描述, 用于描述 JSON 参数的格式要求, 以及每

个参数的含义；

- outputSchema, 可选, 输出内容 (可选项, 格式与 inputSchema 一致, 都是 JSON-Schema);
- depends, 可选, 依赖项;
 - engineVersion, 执行器的最低版本要求;
 - skills, 依赖的子技能。注意: **组合技能必须包含此项。**

inputSchema 说明

参考:

它是在描述参数的 JSON 的格式。参数可以是 Object, 也可以是 Array。

- type, 描述参数是 Object 还 Array。
 - object, 对象;
 - array, 数组;
- properties, 具体的 Object 中的子对象;
 - type, 参数的类型, 可选的类型: string, boolean, integer, object, array
 - description, 参数说明;
 - minimum, 最小值;
 - maximum, 最大值;
 - examples, 举例;
 - enum, 枚举可选项;
 - enumDescriptions, 每个枚举值的说明;
- required, 指定哪些参数是必须提供的;
- name, 英文名称;
- nameCn, 中文名称。

使用示例

```
C++  
代码块  
ubt::robot::Api::Json js;
```

```
js["skillCode"] = "A000026";
auto rsp = ubt_api.requestSync("cc.api.skill.profile", js);
if (rsp.error.code == 0) {    //!< 如果没有异常，则打印返回内容
    cout << "Success, result:" << rsp.js_result.dump() << std::endl;
} else {    //!< 如果错误，则打印错误码与错误信息
    cout << "Error, code:" << rsp.error.code << ", message:" <<
    rsp.error.message << endl;
}
}
```

运行效果

demo 文件：**examples/09_get_skill_profile_demo.cpp**

```
ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.20260203/ubt_api_tiny$ ./demo
== UbtApi Demo ==
Success, result:{"builtin":true,"category":"nav","description":"执行特定的肢体动作","domain":["TYX"],"engine":"atomic","inputSchema":{"additionalProperties":false,"properties":{"actionId":{"description":"动作ID","type":"string"},"required":["actionId"],"type":"object"},"name":"ExecutePresetLimbAction","outputSchema":null,"skillCode":"A000026","version":"0.0.1"}
== End ==
ubt@ubt-System-Product-Name:~/下载/ubt_api_tiny.20260203/ubt_api_tiny$
```

6.2.1.5 技能罗列

A000001: TTS

功能：将文本转成语音进行播报。

参数：

- text (必选)：字符串类型，表示要播报的文本内容。
- speed (可选)：整数类型，取值范围 0 ~ 100，默认值为 50。
- volume (音量, 可选)：整数类型，取值范围 0 ~ 100，默认值为 50。

注：使用 API 停止该技能时，需注意：如果停止的不是当前的播报语音，stop 会返回失败，但是仍然会停止成功

示例参数：

```
JSON
{
  "text": "Hello! I am UBTECH Robot Cruzr S2.",
  "speed": 50
}
```

A000002: NavTo

功能：指定坐标或者地点进行导航

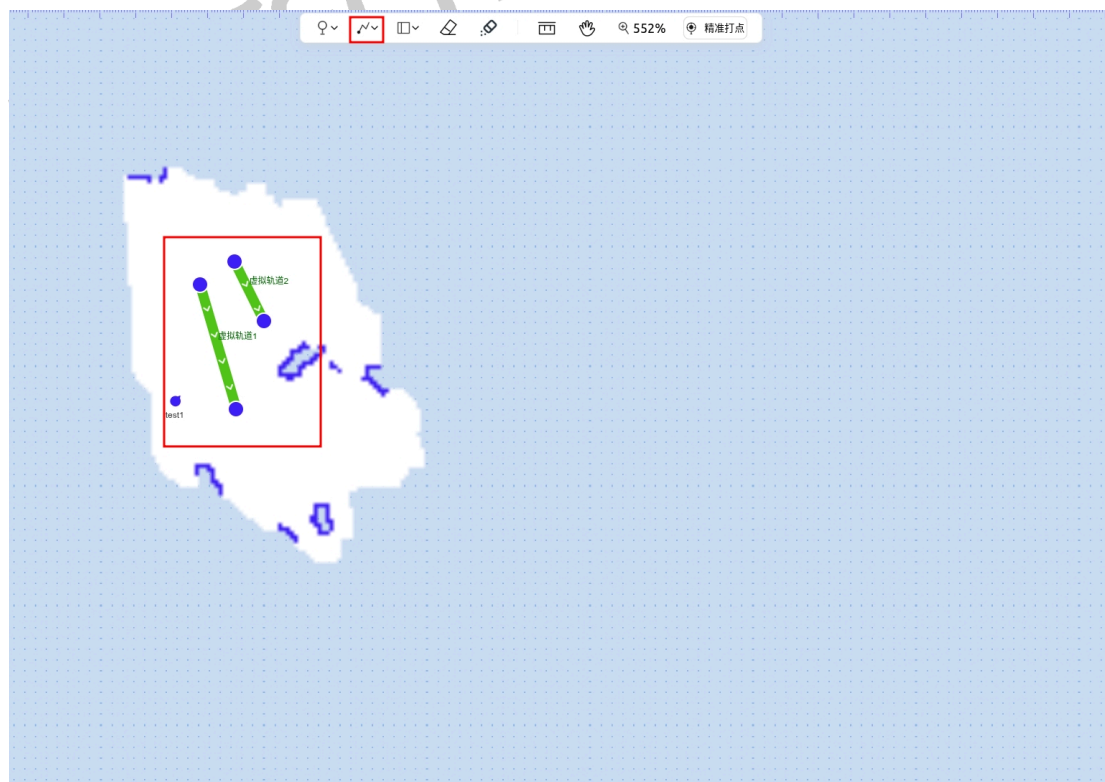
参数：

- **pose** (与 **targetId** 有且必填一个): 数组类型, 包含 3 个元素, 分别表示目标位置的 **x** (单位: m)、**y** (单位: m) 和 **yaw** (单位: rad)。
- **speed** (填了 **pose** 后必填): 数组类型, 包含 3 个非负元素, 分别表示移动速度的 **vx** (单位: m/s)、**vy** (单位: m/s) 和 **vyaw** (单位: rad/s)。
- **targetId** (与 **pose** 有且必填一个): **string** 类型, 地图上打的点位名称, 使用此字段时, 为精定位导航
- **enableTrackNav** (非必填): **bool** 类型, **true**: 使用轨道导航, **false**: 不使用轨道导航, 默认为 **false**

地图参数模板:



轨道导航建图:



注意: 轨道需要在起点和终点 2m 范围内。

示例参数：

```
JSON
{
  "pose": [0.1, 1.0, 2.0],
  "speed": [0.5, 0.3, 0.2]
}
```

```
JSON
{
  "targetId": "A",    //启用此字段时为精定位导航
  "enableTrackNav": true // 不填或者为 false 时为轨道导航，true 时使用
                        轨道导航
}
```

A000012: SetMap

功能：设置地图

参数：

- `mapId` (必填)：字符串类型，表示要设置的地图 ID。

示例参数：

```
JSON
代码块
{
  "mapId": "office_13floor"
}
```

A000013: Relocation

功能：重定位

参数：

- `mode` (必填)：字符串类型，表示重定位模式，取值及要求如下：
 - "global"：全局重定位，不能提供 `pose` 字段。
 - "local"：局部重定位。
 - "force"：强制重定位。
- `pose` (`mode` 非 "global" 时必填)：数组类型，包含 3 个元素，分别表示目标位置

的 x（单位：m）、y（单位：m）和 yaw（单位：rad）。

示例参数：

```
JSON
{
  "mode": "global"
}

{
  "mode": "local",
  "pose": [0.1, 0.1, 0.1]
}
```

A000020: StartMotion

功能：启动运控

参数：为空

示例参数：

```
JSON
{}
```

A000026: ExecutePresetLimbAction

功能：执行特定肢体动作

参数：

- **actionId**（必填）：字符串类型，表示要执行的预设肢体动作 ID，取值参考以下内容：

目前已支持动作列表 (相邻相同色块动作为组合动作)	ID (task_name)	描述	备注
握手	qyh/handshake	握手	启动后需要发 stop 指令关闭
握手回零	qyh/post_handshake	握手 stop 后回零	

脸颊比心	qyh/face_heart	脸颊比心	需要配合脸屏
脸颊比心回零	qyh/post_face_heart	在脸颊比心后使用	
加油四代手	production_movie/cheer_up_s2		需要配备 4 代手
取消加油四代手	production_movie/cheer_down_s2		需要配备 4 代手; 加油的归零动作
加油三代手	production_movie/fist_up_s2		需要配备 3 代手, 会自动回零
胸前比心	production_movie/love		机器人装备 3 代手
胸前比心回零	production_movie/post_love		机器人装备 3 代手

示例参数:

```
JSON
{
  "actionId": "qyh/handshake"
}
```

6.2.2 网络配置

6.2.2.1 设置 WiFi 网络

发起方	客户端
类型	request
方法名	cc.api.network.wifi.client.set
请求参数	<pre>JSON { "ssid": "your_ssid", "password": "your_password"</pre>

	<pre>}</pre>
返回数据	<pre>JSON { "result": "accept" }</pre>

使用示例

C++
代码块

```
ubt::robot::Api::Json js;
js["ssid"] = "your_ssid";
js["password"] = "your_password";
ubt_api.request("cc.api.network.wifi.client.set", js,
  [](const ubt::robot::Api::Response& rsp) {
    if (rsp.error.code == 0) {
      std::cout << "set wifi config success, js_result = " <<
        rsp.js_result.dump() << std::endl;
    } else {
      std::cout << "set wifi config fail, code: " <<
        rsp.error.code
          << ", message: " << rsp.error.message <<
            std::endl;
    }
  }
);
```

6.2.3 状态

6.2.3.1 当前支持的状态列表

状态 ID	说明	数据结构	备注
servoState	伺服电机的温度	<pre>{ 'servoState': [{'id': 1001, 'status': 4663, 'temperature': 54.0}, {'id': 1002, 'status': 4663, 'temperature': 53.0},</pre>	

		<pre>{'id': 2001, 'status': 4663, 'temperature': 25.0}, {'id': 2002, 'status': 4663, 'temperature': 25.0}, }</pre>	
gpuTemperature	GPU 温度	<pre>{ "curr": 75 }</pre>	
navState	导航状态	<pre>{ "state": "navigating" }</pre>	返回字符串对应状态： "navigating": 正在导航 "free_walking": 正在自由行走 "setting_map": 正在设置地图 "relocating": 正在重定位 "mapping": 正在建图 "idle": 空闲中

6.2.3.2 获取指定状态

发起方	客户端
类型	request
方法名	cc.api.status.get
请求参数	<pre>JSON { "list": ["servoState"] }</pre>

返回数据	<pre>JSON {'servoState': [{ 'id': 1001, 'status': 4663, 'temperature': 54.0}, { 'id': 1002, 'status': 4663, 'temperature': 53.0}, { 'id': 2001, 'status': 4663, 'temperature': 25.0}, { 'id': 2002, 'status': 4663, 'temperature': 25.0}, ] }</pre>
------	---

使用示例

```
C++
代码块
ubt::robot::Api::Json js;
js["list"] = {"servoState"};
ubt_api.request("cc.api.status.get", js,
  [](const ubt::robot::Api::Response& rsp) {
    if (rsp.error.code == 0) {
      std::cout << "get status list success, js_result = " <<
rsp.js_result.dump() << std::endl;
    } else {
      std::cout << "get status list fail, code: " <<
rsp.error.code
          << ", message: " << rsp.error.message <<
std::endl;
    }
  }
);
```

6.2.3.3 获取状态列表

发起方	客户端
类型	request
方法名	cc.api.status.list
请求参数	无

返回数据	<pre>JSON { "list": ["servoState", "gpu_temperature", ...] }</pre>
------	---

使用示例

```
C++
代码块
ubt_api.request("cc.api.status.list",
  [](const ubt::robot::Api::Response& rsp) {
    if (rsp.error.code == 0) {
      std::cout << "get status list success, js_result = " <<
rsp.js_result.dump() << std::endl;
    } else {
      std::cout << "get status list fail, code: " <<
rsp.error.code
          << ", message: " << rsp.error.message <<
std::endl;
    }
  }
);
```

6.2.4 人机交互

6.2.4.1 订阅唤醒事件

发起方	客户端
类型	request
方法名	cc.api.sound.wakeup.subscribe
请求参数	无
返回数据	<pre>JSON { "result": "accept" }</pre>

使用示例

C++

代码块

```
ubt_api.request("cc.api.sound.wakeup.subscribe",
    [](const ubt::robot::Api::Response& rsp) {
        if (rsp.error.code == 0) {
            std::cout << "subscribe success, js_result = " <<
rsp.js_result.dump() << std::endl;
        } else {
            std::cout << "subscribe fail, code: " << rsp.error.code
                << ", message: " << rsp.error.message <<
std::endl;
        }
    }
);
```

6.2.4.2、取消唤醒事件的订阅

发起方	客户端
类型	request
方法名	cc.api.sound.wakeup.unsubscribe
请求参数	无
返回数据	JSON<pre>{ "result": "accept" }</pre>

使用示例

C++

代码块

```
ubt_api.request("cc.api.sound.wakeup.unsubscribe",
    [](const ubt::robot::Api::Response& rsp) {
        if (rsp.error.code == 0) {
            std::cout << "unsubscribe success, js_result = " <<
rsp.js_result.dump() << std::endl;
        }
    }
);
```

```
    } else {
        std::cout << "unsubscribe fail, code: " << rsp.error.code
                    << ", message: " << rsp.error.message <<
std::endl;
    }
}
);
```

6.2.4.2 接收唤醒事件

发起方	机器人
类型	notify
方法名	cc.api.sound.wakeup.event
请求参数	无
返回数据	JSON { "direction": 13 // 方向角, -179 ~ 179, 正前方为 0, 左正右负 }

使用示例

```
C++
代码块
...

// 注册接收唤醒事件的回调
ubt_api.addService("cc.api.sound.wakeup.event",
  [](int int_id, const ubt::robot::Api::Json& js_params) {
    std::cout << "id = " << int_id << ", result = " <<
js_params.dump() << std::endl;
  }
);

...
```

6.2.5 地图

6.2.5.1 获取地图列表

发起方	客户端
类型	request
方法名	cc.api.map.list
请求参数	无
返回数据	JSON<pre>{ "mapList": ["xxx1", "xxx2", ...] }</pre>

使用示例

C++

代码块

```
ubt_api.request("cc.api.map.list",  
  [](const ubt::robot::Api::Response& rsp) {  
    if (rsp.error.code == 0) {  
      std::cout << "get map list success, js_result = " <<  
rsp.js_result.dump() << std::endl;  
    } else {  
      std::cout << "get map list fail, code: " << rsp.error.code  
        << ", message: " << rsp.error.message <<  
std::endl;  
    }  
  }  
);
```

6.2.5.2 删除地图

发起方	客户端
类型	request

方法名	cc.api.map.delete
请求参数	JSON 代码块 <pre>{ "mapId": "xxx1" }</pre>
返回数据	JSON <pre>{ "result": "accept" }</pre>

使用示例

C++

代码块

```
ubt::robot::Api::Json js;  
js["mapId"] = "xxx1";  
ubt_api.request("cc.api.map.delete", js,  
  [](const ubt::robot::Api::Response& rsp) {  
    if (rsp.error.code == 0) {  
      std::cout << "delete map success, js_result = " <<  
rsp.js_result.dump() << std::endl;  
    } else {  
      std::cout << "delete map fail, code: " << rsp.error.code  
        << ", message: " << rsp.error.message <<  
std::endl;  
    }  
  }  
);
```

6.2.5.3 普通建图

发起方	客户端
类型	request
方法名	cc.api.map.mapping.start
请求参	JSON

数	代码块<pre>{ "mapId": "xxx1" "mode": "NEW" }</pre>
返回数据	JSON<pre>{ "result": "accept" }</pre>

注意：mapId 长度限制 25，特殊字符目前都支持，没有限制。

使用示例

C++

代码块

```
ubt::robot::Api::Json js;
js["mapId"] = "xxx1";
js["mode"] = "NEW";
ubt_api.request("cc.api.map.mapping.start", js,
    [](const ubt::robot::Api::Response& rsp) {
        if (rsp.error.code == 0) {
            std::cout << "start mapping success, js_result = " <<
rsp.js_result.dump() << std::endl;
        } else {
            std::cout << "start mapping fail, code: " << rsp.error.code
                << ", message: " << rsp.error.message <<
std::endl;
        }
    }
);
```

6.2.5.4 增量建图

发起方	客户端
类型	request
方法名	cc.api.map.mapping.start
请求参	JSON

数	代码块<pre>{ "mapId": "xxx1" "mode": "EXTEND" }</pre>
返回数据	JSON<pre>{ "result": "accept" }</pre>

注意：mapId 长度限制 25，特殊字符目前都支持，没有限制。

使用示例

```
C++  
代码块  
ubt::robot::Api::Json js;  
js["mapId"] = "xxx1";  
js["mode"] = "EXTEND";  
ubt_api.request("cc.api.map.mapping.start", js,  
  [](const ubt::robot::Api::Response& rsp) {  
    if (rsp.error.code == 0) {  
      std::cout << "start mapping success, js_result = " <<  
rsp.js_result.dump() << std::endl;  
    } else {  
      std::cout << "start mapping fail, code: " << rsp.error.code  
        << ", message: " << rsp.error.message <<  
std::endl;  
    }  
  }  
);
```

6.2.5.5 取消建图/保存建图

发起方	客户端
类型	request
方法名	cc.api.map.mapping.stop
请求参	JSON

数	代码块 <pre>{ "isSaveMap": "true" //true:保存建图;false: 取消建图 }</pre>
返回数据	JSON <pre>{ "result": "accept" }</pre>

使用示例

C++

代码块

```
ubt::robot::Api::Json js;  
js["isSaveMap"] = true; //true:保存建图;false: 取消建图  
ubt_api.request("cc.api.map.mapping.stop", js,  
    [](const ubt::robot::Api::Response& rsp) {  
        if (rsp.error.code == 0) {  
            std::cout << "stop mapping success, js_result = " <<  
rsp.js_result.dump() << std::endl;  
        } else {  
            std::cout << "stop mapping fail, code: " << rsp.error.code  
                << ", message: " << rsp.error.message <<  
std::endl;  
        }  
    }  
);
```

6.2.6 运动

6.2.6.1 矢量行走

发起方	客户端
类型	request
方法名	cc.api.motion.cmd_vel
请求参数	JSON 代码块

	<pre>{ "velocity": [x, y, yaw] }</pre>
返回数据	<pre>JSON { "result": "accept" } 或 { "result": "reject", "reason": "not in remote mode" }</pre>

使用示例

C++

代码块

```
ubt::robot::Api::Json js;
js["velocity"] = {0.3, 0.0, 0.0};
while (true) {
    ubt_api.request("cc.api.motion.cmd_vel", js,
        [](const ubt::robot::Api::Response& rsp) {
            if (rsp.error.code == 0) {
                std::cout << "cmd vel success, js_result = " <<
rsp.js_result.dump() << std::endl;
            } else {
                std::cout << "cmd vel fail, code: " << rsp.error.code
<< ", message: " << rsp.error.message <<
std::endl;
            }
        }
    );
    sleep(1);
}
```

范围:

x = [-0.3, 0.8]

y = [-0.3, 0.3]

yaw = [-0.6, 0.6]

规则: 超过 2s 无指令, 速度置零

7. 场景 demo 案例

7.1 半身遥操

本次配置如下：

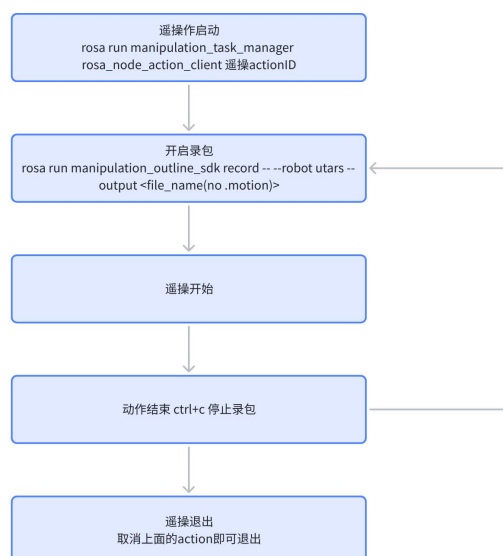
- 四种配置机型：cruzs-s2 带夹板、cruzs-s2 带三代手、cruzs-s2 带四代手、cruzs-s2 带二指夹爪
- 系统大版本：v0.2.0
- 系统环境：需要配置 MC_SCENE=DAC、TELE_DEVICE=pico
- 录制软件：pico 设备
- 遥操操作：参考 Cruzr S2 遥操作 SOP

7.1.1 操作方式

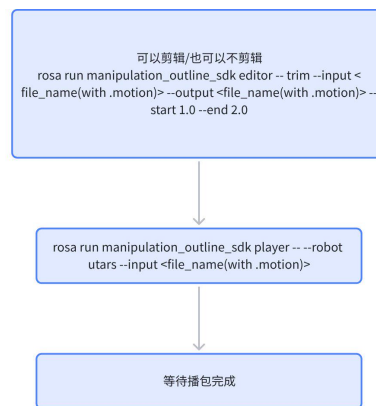
通过 web 或者航模启动机器人后，以下指令都可以在 motion 中的任意容器中执行，`source /opt/walker/setup.bash` 后启动，注意事项：

7.1.1.1 录包流程

1. 遥操作启动
2. 开启录包
3. 遥操开始
4. 动作结束 ctrl+c 停止录包



7.1.1.2 播包流程



7.1.2 遥操作启动

1. 检查 version 板的 drivers 模块是否已包含 signal_server 节点是否存在并运行正常, 不存在则进入 drivers 模块中的任意容器启动 signal_server 节点 (保持界面开启状态):

Plain Text

```
source /opt/walker/setup.bash && rosa run signal_server  
signal_server
```

2. 检查 motion 板的 motion 模块是否已包含 rtm_receiver 节点是否存在并运行正常, 若不存在则进入 motion 模块中的任意容器启动 rtm_receiver 节点 (保持界面开启状态):

Plain Text

```
source /opt/walker/setup.bash && rosa run rtm_receiver  
rtm_receiver
```

3. 启动遥操

Plain Text

1、步骤一:

进入 motion 中的任意容器中执行 `source /opt/walker/setup.bash`

2、步骤 2

主版本遥操启用指令: `rosa run manipulation_task_manager`

`rosa_node_action_client` 遥操 actionID

遥操 actionID 命名规则: 机型_夹具型号_动捕设备_teleoperation

机型: s2/cruzer

夹具型号: clamp/v3/v4/gripper

动捕设备: vr/pico/xsens/exoskeleton

例如:

```
rosa run manipulation_task_manager rosa_node_action_client  
s2_v3_vr_teleoperation
```

```
rosa run manipulation_task_manager rosa_node_action_client  
s2_v4_xsens_teleoperation
```

```
rosa run manipulation_task_manager rosa_node_action_client  
cruzer_clamp_pico_teleoperation
```

```
rosa run manipulation_task_manager rosa_node_action_client  
cruzer_gripper_exoskeleton_teleoperation
```

遥操退出指令: 取消上面的 action 即可退出

3、步骤 3

执行完该遥操器用指令之后, 需要再次 web 端点击开始遥操, 如果 web 端点击开始遥操无响应, 可在本地终端中执行 `pico_control --arm_type clamp/hand/gripper` 命令

7.1.2.1 录包

(需要替换<file_name(no .motion)> 这一部分为期望的具体文件名称)

夹板

Rust

```
rosa run manipulation_outline_sdk record -- --robot utars --output  
<file_name(no .motion)>
```

夹爪

Rust

```
rosa run manipulation_outline_sdk record -- --robot  
utars_with_grip --output <file_name(no .motion)>
```

三代手

Rust

```
rosa run manipulation_outline_sdk record -- --robot  
utars_with_v3_hand --output <file_name(no .motion)>
```

四代手

Rust

```
rosa run manipulation_outline_sdk record -- --robot
```

```
utars_with_v4_hand --output <file_name(no .motion)>
```

7.1.2.2 播包

夹板

```
Rust
rosa run manipulation_outline_sdk player -- --robot utars --input
<file_name(with .motion)>
```

夹爪

```
Rust
rosa run manipulation_outline_sdk player -- --robot
utars_with_grip --input <file_name(with .motion)>
```

三代手

```
Rust
rosa run manipulation_outline_sdk player -- --robot
utars_with_v3_hand --input <file_name(with .motion)>
```

四代手

```
Rust
rosa run manipulation_outline_sdk player -- --robot
utars_with_v4_hand --input <file_name(with .motion)>
```

7.1.2.3 剪辑

取一段时间

```
Rust
rosa run manipulation_outline_sdk editor -- trim --input
<file_name(with .motion)> --output <file_name(with .motion)> --
start 1.0 --end 2.0
```

将提取完之后的文件进行 play 就可以

加/减速播放

```
Rust
rosa run manipulation_outline_sdk editor -- scale --input
<file_name(with .motion)>--output <file_name(with .motion)> --
```

factor 2.0
2.0[放慢]
0.5[加速]

7.2 大模型替换

7.2.1 本地 TTS 替换

抽象化每个模型的输入/输出，隔离具体模型的调用，由具体业务进行解析使用

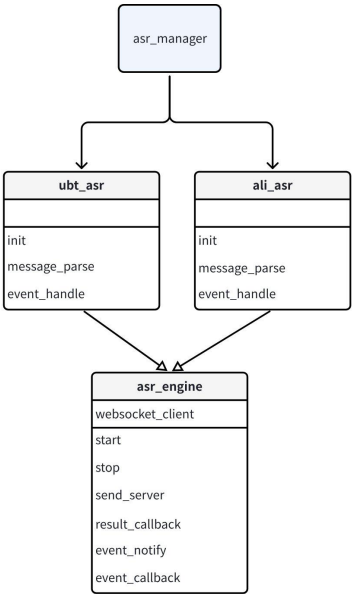
本地模型重要配置和服务

服务名称	服务地址
ASR 服务开关	service: /sys/asr/enable, std_srvs::srv::SetBool
ASR 识别结果 是否传入 NLP	service: /sys/llm_in/enable, std_srvs::srv::SetBool
NLP 结果是否传入 TTS 服务	service: /sys/llm_out/enable, std_srvs::srv::SetBool
TTS 是否播报	service: /sys/tts/enable, std_srvs::srv::SetBool
开启内置麦克风录音功能	service: /sys/run_record/enable, std_srvs::srv::SetBool
降噪音频流监听	topic: /sys/speech/mic_denoise,, std_msgs::msg::Int16MultiArray
唤醒词事件和唤醒解耦开关，唤醒词事件不直接唤醒机器，由业务根据场景判断是否需要唤醒	service: /sys/akdecouple/enable, std_srvs::srv::SetBool
唤醒词事件监听	topic: /sys/speech/kws, std_msgs::msg::String

PS：使用以上 1-4 service 关闭本地模型，使能 5 开启录音功能，监听 6 音频流，上

传云端模型，使用 7 解耦唤醒和唤醒词事件，监听 8 唤醒词事件做业务功能

7.2.1.1 二开设计参考框架



1. 抽象engine类，只包含基础通用能力：
 - 使用websocket和服务器建立连接
 - 发送消息到服务器
 - 通过callback回调服务器消息到业务层
 - 标准接口发布业务事件
2. ubt_asr/ali_asr 是不同大模型的实现，继承engine
3. asr_manager 根据配置选用大模型
4. nlp、tts 同理

7.2.1.2 部分参考代码

开关及 topic 监听

```
C++
// 1. ASR 服务开关
asr_enable_client_ =
common_utils::createClient<std_srvs::srv::SetBool>(
    node, "/sys/asr/enable");

// 2. ASR 识别结果 是否传入 NLP
llm_in_enabled_client_ =
common_utils::createClient<std_srvs::srv::SetBool>(
    node, "/sys/llm_in/enable");

// 3. NLP 结果是否传入 TTS 服务 (service: /sys/llm_out/enable,
std_srvs::srv::SetBool)
llm_out_enabled_client_ =
common_utils::createClient<std_srvs::srv::SetBool>(
    node, "/sys/llm_out/enable");

// 4. TTS 是否播报 (service: /sys/tts/enable,
std_srvs::srv::SetBool)
tts_enabled_client_ =
common_utils::createClient<std_srvs::srv::SetBool>(
```



```

        node, "/sys/tts/enable");

// 5.开启内置麦克风录音功能 (service: /sys/run_record/enable,
std_srvs::srv::SetBool)
run_record_enabled_client_ =
common_utils::createClient<std_srvs::srv::SetBool>(
    node, "/sys/run_record/enable");

// 6. 降噪音频流监听
mic_denoise_reader_ =
common_utils::createMessageReader<std_msgs::msg::Int16MultiArray>(
    node, "/sys/speech/mic_denoise", 10,
std::bind(&cruzs2AsrManager::mic_denoise_callback, this,
std::placeholders::_1));

// 7. 唤醒词事件和唤醒解耦开关
akdecouple_client_ =
common_utils::createClient<std_srvs::srv::SetBool>(
    node, "/sys/akdecouple/enable");

// 8. 唤醒词事件监听
kws_event_reader_ =
    common_utils::createMessageReader<std_msgs::msg::String>(
        node, "/sys/speech/kws", 10,
std::bind(&cruzs2AsrManager::kwsEventCallback, this,
std::placeholders::_1));

// 9. 本地 asr 结果监听
asr_cruzs2_reader_ =
common_utils::createMessageReader<sys_task_msgs::msg::Asr>(
    node, "sys/speech/asr", 10,
std::bind(&cruzs2AsrManager::asrResultCallback, this,
std::placeholders::_1));

// 设置开关
// ros2 命令行 ros2 service call /sys/asr/enable
std_srvs/srv/SetBool "{data: true}" , 其它类似
static std::vector<bool> done(6, false);
auto execute_ops = [&]() {
    if (!done[1]) done[0] = enableS2Env(llm_in_enabled_client_,
false, "llm_in_enabled"); // 关闭 nlp 输入
    if (!done[2]) done[1] = enableS2Env(llm_out_enabled_client_,

```

```

false, "llm_out_enabled"); // 关闭 nlp 输出
    if (!done[3]) done[2] = enableS2Env(tts_enabled_client_,
false, "tts_enabled"); // 关闭 tts 播报
    if (!done[4]) done[3] = enableS2Env(akdecouple_client_, true,
"akdecouple"); // 使能唤醒和唤醒词事件
    if (!done[5]) done[4] =
enableS2Env(run_record_enabled_client_, true,
"run_record_enabled"); // 开启内置麦克风录音
    if (!done[0]) done[5] = enableS2Env(asr_enable_client_, true,
"asr_enable"); // 使能 asr 本地识别, 如无需本地 asr 结果可关闭

    // 检查是否全部完成
    for (bool d : done) {
        if (!d) {
            ULOGW("Not all pre env are configured");
            return false;
        }
    }
    ULOGI("All pre env are configured");
    return true;
};

// ros2 topic echo /sys/speech/mic_denoise
// ros2 topic echo /sys/speech/kws
// 降噪音频流数据回调函数
void
CruzrS2AsrManager::mic_denoise_callback(std_msgs::msg::Int16MultiA
rray::ConstSharedPtr data)
{
    ULOGD("收到降噪数据: %d", data->data.size());
    // 业务根据需求自行处理降噪音频流数据
    ...
    // 发送到云端 asr
    sendMessageToServer(data->data.data(), data->data.size() *
sizeof(data->data[0]), 1);
}

// 唤醒词事件回调函数
void
CruzrS2AsrManager::kwsEventCallback(std_msgs::msg::String::ConstSh
aredPtr msg)
{
    // 业务根据需求自行处理唤醒词事件

```

```
...  
}
```

调用系统扬声器

使用 第三方库 (alsa、portaudio 等) 调用系统扬声器流式播放

示例: 使用 portaudio 库调用系统默认扬声器, 其他使用方法请自行参考官方文档

```
C++  
// 初始化 默认输出设备  
bool startOutputStream(int channel, double rate, PaSampleFormat  
format,  
    unsigned long frames_per_buffer, RawAudioDataCallback  
callback, void *user_data)  
{  
    if (is_playing_) {  
        ULOGE("AudioRecorder is already playing, not need to start  
again.");  
        return true;  
    }  
  
    checkInitialized();  
  
    PaDeviceIndex outputDevice = Pa_GetDefaultOutputDevice();  
    const PaDeviceInfo* outputInfo =  
Pa_GetDeviceInfo(outputDevice);  
    if(outputInfo) {  
        ULOGI("默认输出设备名称: %s", outputInfo->name);  
        ULOGI("默认输出设备最大声道数: %d", outputInfo-  
>maxInputChannels);  
        ULOGI("默认输出设备默认低延迟输入: %f", outputInfo-  
>defaultLowInputLatency);  
        ULOGI("默认输出设备默认低延迟输出: %f", outputInfo-  
>defaultLowOutputLatency);  
        ULOGI("默认输出设备默认高延迟输入: %f", outputInfo-  
>defaultHighInputLatency);  
        ULOGI("默认输出设备默认高延迟输出: %f", outputInfo-  
>defaultHighOutputLatency);  
        ULOGI("默认输出设备默认采样率: %f", outputInfo-  
>defaultSampleRate);  
    }  
}
```

```

PaError err;
err = Pa_OpenDefaultStream(
    &output_stream_,
    0,          // 无输入（麦克风）
    channel,    // 单/双声道输出（喇叭）
    format,     // 32 位浮点采样
    rate,       // 采样率 44.1kHz
    frames_per_buffer, // 每缓冲区 帧
    callback,
    user_data   // 用户数据
);
if (err != paNoError) {
    ULOGE("无法打开喇叭: %s", Pa_GetErrorText(err));
    Pa_Terminate();
    is_playing_ = false;
    return 1;
}

err = Pa_StartStream(output_stream_);
if (err != paNoError) {
    ULOGE("无法启动喇叭: %s", Pa_GetErrorText(err));
    Pa_CloseStream(output_stream_);
    Pa_Terminate();
    is_playing_ = false;
    return 1;
}

is_playing_ = true;
return true;
}

// 写入音频流进行播放
PaError writeStream(void *buffer, unsigned long numFrames)
{
    return Pa_WriteStream(output_stream_, buffer, numFrames);
}

```

7.2.2 本地大模型替换

1. 环境准备

首先，安装 Python 的 OpenAI 依赖库：

Bash

```
pip install openai
```

2. Python 代码示例

在这个示例中，我将配置项放在了一个字典中，你也可以轻松地将其改为从 `.env` 文件或环境变量中读取。

Python

```
from openai import OpenAI
import sys

# ===== 配置区域 =====# 你可以将这些配置放在
# 环境变量或配置文件中
CONFIG = {
    "api_key": "你的 API_KEY 在这里",
    "base_url": "你的云端大模型网址，例如
https://api.openai.com/v1",
    "model": "你的模型名，例如 chat-gpt、qwen 等"
}

# =====def
call_llm_inference(user_query):
    """
    调用大模型进行推理
    :param user_query: 用户文本输入
    :return: 大模型的文本回复
    """# 初始化客户端
    client = OpenAI(
        api_key=CONFIG["api_key"],
        base_url=CONFIG["base_url"]
    )

    try:
        # 发起聊天请求
        response = client.chat.completions.create(
            model=CONFIG["model"],
            messages=[
                {"role": "system", "content": "你是一个乐于助人的 AI
助手。"},
                {"role": "user", "content": user_query}
            ],
            temperature=0.7, # 控制随机性，0.7 比较平衡
            stream=False     # 设置为 True 可以实现打字机效果
```

```

    )

    # 提取回复内容
    answer = response.choices[0].message.content
    return answer

except Exception as e:
    return f"请求出错了, 错误信息: {str(e)}"# --- 测试运行 ---if
__name__ == "__main__":
    # 模拟 ASR 识别后的文本输入
    asr_text = "帮我查一下明天的天气, 顺便写一首关于春天的短诗。"
    print(f"--- 用户输入 (ASR 的结果): ---\n{asr_text}\n")

    result = call_llm_inference(asr_text)

    print(f"--- 大模型回复: ---\n{result}")

```

3. 流式输出 (Streaming) 示例

开启 `stream=True`。这样模型会一边计算一边返回文字, 实现流式输出:

```

Python
# 修改请求部分
stream = client.chat.completions.create(
    model=CONFIG["model"],
    messages=[{"role": "user", "content": user_query}],
    stream=True
)

for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)

```

这样用户就能看到文字逐个蹦出来的效果, 体验更像真实的对话。

7.3 VLA 搬运周转箱子

7.3.1 教程目标

本教程旨在以用例形式, 使用户了解 VLA 模型在优必选 Cruzr-S2 机器人上的使用方法。

用户可通过提供的资料与数据, 自行复现本次提供的 demo 场景: 搬运周转箱。为对齐训练数据, 周转箱尺寸应为 60cm*40cm*22cm, 且需摆放在 1m 高的平台处。

同时。用户也可借鉴此流程，训练并部署自己的 VLA 模型，去处理自定义任务。不同的任务，如 VLA 拆码垛、上下料、抓特定物件等，都可套用此流程，部署对应的模型来适配不同的任务

7.3.2 算法训练

本教程后续代码均使用 Isaac-Groot 1.5 为示例，需改动两个文件才可适配本机所采的数据，做训练与推理

gr00t_finetune.py

data_config.py

用 gr00t_finetune.py 进行训练时，`--data_config` 设成 `utars1`，`--embodiment-tag` 设成 `new_embodiment`

示例数据：

`data/utars_clamp_and_place_large_box_full_data_bio_lerobot_0319`

示例权重文件：`weight/checkpoint-40000`

7.3.3 环境配置

推荐网线连接连接机器人后，进行如下操作

1. 文件拷贝

1. 将 `docker-images` 内的 `vla_control_node_sdk.tar` 拷贝到 `walker@192.168.11.2` 任意位置

a. 后续教程默认为此路径：

`sftp://walker@192.168.11.2/home/walker/vla_control_node_sdk.tar`

2. 将 `codes/motion/s2_vla_scripts` 拷贝到 `walker@192.168.11.2` 任意位置

a. 后续教程默认为此路径：

`sftp://walker@192.168.11.2/home/walker/vla_motion/s2_vla_scripts`

3. 将 `codes/motion/rosa_vla_additional` 拷贝到 `walker@192.168.11.2` 任意位置

a. 后续教程默认为此路径：

`sftp://walker@192.168.11.2/home/walker/vla_motion/rosa_vla_additional`

4. 将 `docker-images` 内的 `vla_inference_node_sdk.tar` 拷贝到 `walker@192.168.11.3` 任意位置

a. 后续教程默认为此路径：

`sftp://walker@192.168.11.3/home/walker/vla_inference_node_sdk.tar`

5. 将 `codes/vision/rosa_vla_additional` 拷贝到 `walker@192.168.11.3` 任意位置

- a. 后续教程默认为此路径：
`sftp://walker@192.168.11.3/home/walker/vla_vision/rosa_vla_additional`
6. 将 `codes/vision/docker_files` 拷贝到 `walker@192.168.11.3` 任意位置
 - a. 后续教程默认为此路径：
`sftp://walker@192.168.11.3/home/walker/vla_vision/docker_files`
7. 将训练权重（或你自己的训练权重）`weight/checkpoint-40000` 拷贝到 `walker@192.168.11.3` 任意位置
 - a. 后续教程默认为此路径：
`sftp://walker@192.168.11.3/home/walker/vla_vision/rosa_vla_additional/checkpoint-40000`

2. Docker 加载

在 192.168.11.2 上：

Terminal 1:

```
Bash
cd ~
docker cp ~/vla_motion/cruze_vla_scripts walker-
motion.manipulation_robot_app-1:/home/ubt/
docker exec -it walker-motion.manipulation_robot_app-1 bash
source /opt/walker/setup.bash
source
/home/ubt/cruze_vla_scripts/auto_load_vla_scripts_new.sh # 然后
一路确认即可
```

Terminal 2:

```
Bash
cd ~
docker load -i vla_control_node_sdk.tar

docker run -it --name vla_control_node_sdk \
  --network host \
  -v
/home/walker/vla_motion/rosa_vla_additional:/home/ubt/addition
al \
  vla_control_node_sdk:latest \
  /bin/bash
```

在 192.168.11.3 上：

Terminal 1:


```

Bash
cd ~
docker load -i vla_inference_node_sdk.tar
docker run --gpus all -it --runtime=nvidia --network=host \
  --name=vla_inference_node_sdk \
  -v
/home/walker/vla_vision/docker_files/gr00t_model:/opt/Isaac-
GR00T \
  -v
/home/walker/vla_vision/rosa_vla_additional:/home/ubt/addition
al \
  vla_inference_node_sdk:latest \
  /bin/bash

```

7.3.4 模型部署与推理

在完成前序步骤后，192.168.11.2（下面简称 motion 板）和 192.168.11.3（下面简称 vision 板）应该分别有一个 docker 环境：vla_control_node_sdk 和 vla_inference_node_sdk

7.3.4.1 Motion 板

ssh walker@192.168.11.2

密码：aa

Terminal 1:

```

Bash
ssh walker@192.168.11.2
# 进入运控容器
docker exec -it walker-motion.manipulation_robot_app-1 bash
# source 环境
source /opt/walker/setup.bash

# 启动运控
rosa run t800_mc_server start_mc_client

# 运控启动结束后，关节角归零
rosa action send_goal /mc/manipulation/action
mc_task_msgs/action/ArmTask
'{"task_name":"cruizr/open_arm_before_home", "yaml_args":""}'

# 把手臂抬到初始位置，这一步视 VLA 模型而定，可以没有

```

```
rosa action send_goal /mc/manipulation/action
mc_task_msgs/action/ArmTask
'{"task_name":"cruzer_bio_vla/cruzer_vla_pick_large_teleop_ready", "yaml_args":""}'
```

Terminal 2:

切换控制器 sdk_controller

注意：需要先将机器人通过遥控器移动到合适位置

```
Bash
# 进入运控容器
ssh walker@192.168.11.2
docker exec -it walker-motion.manipulation_robot_app-1 bash

# source 环境
source /opt/walker/setup.bash

# 需要确定是否开启了运控
rosa run t800_mc_server start_mc_client

# 启动 sdk 控制器
rosa service call /mc/controller_manager/switch_controller
rosa_control_msgs/srv/SwitchController
'{"start_controllers": [\"sdk_controller\"],
 \"stop_controllers\":
 [\"chassis_controller\", \"manipulation_controller\"],
 \"strictness\": 1, \"start_asap\": true, \"timeout\": 1.0}'
```

Terminal 3:

vla 控制节点 node 开启用，主要用来接收 VLA 推理输出的 chunk。

```
Bash
# 进入运控容器
ssh walker@192.168.11.2
docker start vla_control_node_sdk
docker exec -it vla_control_node_sdk bash
source /home/ubt/additional/scripts/setup.sh
cd /home/ubt/additional/vla-
motionx86/src/vla_executor/vla_executor/

# vla 控制节点 node
```

```
python3 executor_node_sdk.py
```

7.3.4.2 Vision 板

ssh walker@192.168.11.3

密码: aa

新机器第一次使用 VLA 功能需要设定时间戳对齐, 后续使用则无需此步骤:

```
Bash
ssh walker@192.168.11.3
sudo sh -c 'echo "echo 1 >
/sys/module/pegasus_cdi_mgr/parameters/ts_type" >>
/usr/local/lib/camera-drv/custom-cam-setting.sh'
```

操作完后需要重启机器

Terminal 4:

进入 vla 推理容器

```
Bash
# 进入 vla 推理容器
ssh walker@192.168.11.3
docker start vla_inference_node_sdk
docker exec -it vla_inference_node_sdk bash
cd /home/ubt/additional/vla-onboard/src/gr00t_control/
source /home/ubt/additional/scripts/setup.sh

python3 gr00t_inference.py
```

```
Set action denoising steps to 4
[inject] modality_transform now reuse model.backbone.*processor
2026-04-14 03:17:25.630 | INFO | model_interface_general_ros2: __init__:135 - Gr00t Model Interface initialized with updated joint names
2026-04-14 03:17:26.074 | INFO | __main__: __init__:156 - Stereo image subscriber initialized on topic: /sensor/camera/stereo/color/raw
2026-04-14 03:17:26.075 | INFO | __main__: __init__:157 - QoS: reliability=2, durability=2
2026-04-14 03:17:26.234 | INFO | __main__: __init__:167 - SDK joint state subscriber initialized on topic: /mc/sdk/robot_state
2026-04-14 03:17:26.235 | INFO | __main__: __init__:168 - QoS: reliability=2, durability=2
2026-04-14 03:17:26.251 | INFO | __main__: __init__:177 - Inference result publisher initialized.
2026-04-14 03:17:26.302 | INFO | __main__: __init__:190 - Inference Action Server initialized: /gr00t/trigger_inference
2026-04-14 03:17:26.303 | INFO | __main__: __init__:191 - Action Server ready! Waiting for goals on /gr00t/trigger_inference...
2026-04-14 03:17:26.303 | INFO | __main__: __init__:193 - Gr00t Inference Action Node initialized
2026-04-14 03:17:26.303 | INFO | __main__: __init__:194 - Sync mode: Closest timestamp
2026-04-14 03:17:26.303 | INFO | __main__: __init__:195 - Timestamp preservation: Original
2026-04-14 03:17:26.304 | INFO | __main__:main:985 - Starting single thread executor...
```

出现此 log 即为启动完成

Terminal 5:

ros2 action send_goal 发送

Bash

进入 vla 容器

ssh walker@192.168.11.3

docker start vla_inference_node_sdk

docker exec -it vla_inference_node_sdk bash

source /home/ubt/additional/scripts/setup.sh

ros2 action send_goal /gr00t/trigger_inference

mc_task_msgs/action/InferenceTask '{"task_id": 2,

"max_inference_duration": 240.0, "end_threshold": 0.060}'

8. FAQ

Q1: 做 SDK 的二开，是否对机器的版本有要求？

A1: 建议在主版本 0.2.0 及以上做二开，不要在低版本或定制版本做二开。

Q2: 底盘关节支持位控？

A2: 底盘关节仅支持速度控制，不支持位控。

若有其它疑问，请通过以下渠道反馈：

- 客服热线：400-6666-700（*仅限中国 北京时间 09:00 - 18:30）
- 微信公众号：优必选服务